

Departamento de TICS

Ciberseguridad

Integrantes:

Santiago Arias Gutierrez 2023016

Isaac Campos Gomez 2023033

Joaquín Chinchilla Bonilla 2023042

Jazmin Montero Naranjo 2023090

Isaac Piedra Chaves 2017441

Jimena Valverde Rodriguez 2020479

Jorge Ignacio Ramirez Brenes

Francisco Alvarado Alvarado

Expotec

2026

Resumen

Los dispositivos IoT domésticos, como cámaras, asistentes de voz y televisores inteligentes, suelen compartir red con los equipos personales, con credenciales débiles y sin actualizaciones, lo que los convierte en una puerta frecuente para botnets y movimiento lateral. Este proyecto presenta Quillnet, un gateway de seguridad con arquitectura Zero Trust construido sobre una Raspberry Pi 4. El gateway segmenta la red del hogar en cuatro VLAN (usuarios, cámaras, asistentes de voz y Smart TV), aplica un firewall de denegación por defecto con nftables e inspecciona el tráfico en línea con Suricata 7.0.10 como sistema de prevención de intrusiones.

Un backend en Flask y PostgreSQL traduce las alertas a lenguaje sencillo mediante un modelo de inteligencia artificial, sin enviarle direcciones IP, y las muestra en una aplicación web con notificaciones push. Desde ella la persona puede bloquear dispositivos; además, el sistema aplica cuarentena automática tras tres bloqueos reales en cinco minutos, el desbloqueo exige verificación en dos pasos y el inicio de sesión se protege con un código enviado por correo.

La investigación combinó una encuesta a 112 hogares costarricenses con pruebas de seguridad lanzadas desde Kali Linux y pruebas automatizadas del backend. Con corte al 5 de noviembre de 2026, se verificó el bloqueo en línea de un beacon real del framework Covenant y que el tráfico MQTT sin cifrar hacia un broker público no salió de la red. También se identificó que el tráfico dentro de una misma VLAN no pasa por el gateway y, por tanto, no es inspeccionado, limitación que se incorporó al plan de mejora junto con las demás pruebas pendientes.

Palabras clave: IoT, Zero Trust, segmentación de red, VLAN, IPS, Suricata, Raspberry Pi, hogar inteligente.

Tabla de contenidos:

Resumen....1
Capítulo I....4
Título del proyecto....4
Objetivo General....4
Objetivos Específicos....4
Justificación....5
Definición del problema....7
Análisis de la problemática....7
Planteamiento del problema y pregunta de investigación....8
Objetivos De Investigación....8
Capítulo II....9
Marco teórico....9
Introducción al IoT....9
Estadísticas actualizadas de la problemática IoT....12
Protocolos IoT y vulnerabilidades específicas....12
Modelo Zero Trust....13
Segmentación de red....14
Purple Team....14
Soluciones del mercado....15
Casos reales documentados....15
Capítulo III....17
Marco Metodológico....17
Tipo de Investigación....17
Diseño de la Investigación....20
Conceptos técnicos del sistema....22
Justificación del stack técnico....25
Arquitectura por Capas....26
Instrumentos y Técnicas de Recolección de Datos....39
Procedimiento General....40
Cronograma Real de Ejecución....41
Alcances y Limitaciones....42
Diseño y Comunicación Visual....44
Gestión Administrativa y Financiera....46
Diagrama de Infraestructura de Conexión y Procesos....53
Red privada....54
Red pública....56
Análisis de seguridad....56
1. Descripción general del proyecto....56
2. Alcance del análisis de seguridad....57
3. Identificación de activos....57
4. Análisis de la tríada CIA....58
5. Identificación de vulnerabilidades....62
6. Identificación de amenazas....63
7. Evaluación de Riesgos....64
8. Medidas de seguridad....66

9. Seguridad de acceso y control de usuarios..... 67
10. Protección de datos y privacidad..... 69
11. Desarrollo seguro..... 70
12. Copias de seguridad..... 71
13. Respuesta ante incidentes..... 72
14. Análisis de resultados..... 73
Imágenes de evidencia..... 77
15. Plan de mejora..... 81
16. Conclusiones..... 82
Capítulo IV..... 83
Conclusiones Generales..... 91
Recomendaciones para la actualidad del proyecto..... 92
Recomendaciones para el desarrollo futuro..... 94
Bibliografía..... 96
Anexos..... 100

Capítulo I

Título del proyecto

QUILLNET

Objetivo General

Implementar un gateway de seguridad físico, basado en Raspberry Pi 4 y diseñado para ser desplegable desde la infraestructura de un proveedor de servicios de internet, que aplique segmentación Zero Trust mediante VLAN y firewall con política de denegación por defecto, detecte y bloquee en línea (modo IPS) el tráfico malicioso de los dispositivos IoT residenciales y se valide mediante ejercicios Purple Team, entregando al usuario final una plataforma web de autogestión que no requiera conocimientos técnicos.

Objetivos Específicos

1. Segmentar la red doméstica bajo el modelo Zero Trust, mediante cuatro VLAN 802.1Q (Usuarios, Cámaras IP, Asistentes de voz y Smart TV), con DHCP/DNS independiente por segmento y un firewall nftables con política forward drop, de modo que ningún dispositivo IoT pueda comunicarse con otro segmento sin una regla explícita.
2. Implementar un sistema de detección y prevención de intrusiones en línea, mediante Suricata 7 en modo IPS (NFQUEUE) con las reglas Emerging Threats Open, servicios de monitoreo en la Raspberry Pi y cuarentena automática de dispositivos, validando su funcionamiento con pruebas controladas de bloqueo y con la metodología Purple Team.
3. Construir una plataforma de autogestión para el usuario final (backend con API REST y frontend web) con mapa de red, centro de alertas traducidas a lenguaje claro mediante inteligencia artificial, bloqueo y desbloqueo de dispositivos y autenticación segura, de modo que comprenda y controle la protección de su hogar sin asistencia técnica.

Justificación

El crecimiento exponencial de los dispositivos IoT en los hogares ha transformado la vida cotidiana, pero también ha creado un vector de ataque sin precedentes. Según Transforma Insights (2026), se proyectan unos 33,700 millones de conexiones IoT a nivel mundial para 2030, lo que convierte al hogar promedio en un ecosistema digital extenso y heterogéneo. Paralelamente, el informe de Netgear y Bitdefender (2025) estima que un hogar promedio tiene 22 dispositivos conectados y recibe 29 ataques diarios, lo que posiciona a los dispositivos IoT como uno de los principales vectores de ataque en entornos domésticos. Sin embargo, la mayoría de estos dispositivos carecen de mecanismos básicos de protección: el Unit 42 IoT Threat Report de Palo Alto Networks (2020) documentó que el 57% de los dispositivos IoT eran vulnerables a ataques de severidad media o alta y que el 98% del tráfico IoT no estaba cifrado, lo que exponía datos sensibles durante su transmisión; aunque la cifra proviene de 2020, ilustra una debilidad estructural que los datos más recientes de este capítulo no contradicen. Esta falta de cifrado y segmentación implica que, una vez comprometido un dispositivo, el atacante puede desplazarse libremente por toda la red doméstica, accediendo a información sensible como credenciales bancarias, comunicaciones personales o imágenes de cámaras de seguridad. En otras palabras, el hogar digital se ha convertido en un espacio sin perímetros, donde la privacidad está expuesta por defecto.

Un agravante crítico es que los usuarios finales carecen de los conocimientos técnicos necesarios para gestionar la seguridad de sus redes IoT. Actividades como la configuración de firewalls, la segmentación de red o el monitoreo de tráfico requieren competencias que escapan al perfil del habitante común. Esta brecha de conocimiento tiene consecuencias graves: la mayoría de los hogares no aplican medidas de protección básicas, como el cambio de contraseñas por defecto o la activación de cifrado, lo que deja sus dispositivos y datos expuestos. Esta situación no es un problema menor de falta de actualización, sino un problema estructural que afecta a millones de hogares. La seguridad no puede seguir dependiendo de la iniciativa individual, porque el costo de un error es la exposición total de la vida privada de las personas. Por ello, resulta imperativo trasladar la responsabilidad de la seguridad a un nivel superior: el del proveedor de servicios de

internet (ISP), que ya gestiona la conectividad del hogar y puede, desde esa posición, ofrecer protección sin requerir acción técnica por parte del usuario.

Por todo lo expuesto, el proyecto Quillnet se justifica en tres pilares fundamentales. El primero es la relevancia del problema: la magnitud de dispositivos conectados, su alto índice de vulnerabilidad y la exposición de datos sensibles evidencian una amenaza real y generalizada que requiere atención inmediata, lo cual lleva hacia el segundo pilar, la urgencia de actuar: los usuarios carecen de formación técnica para protegerse, y las soluciones existentes en el mercado no están diseñadas para ser desplegadas desde el ISP ni para ofrecer una protección integral que incluya simulación de ataques, segmentación y alertas inteligentes, dejando un vacío funcional que este proyecto busca cubrir. El tercer pilar es la viabilidad y oportunidad de la implementación: el ISP cuenta con la infraestructura, el alcance y la posición estratégica para desplegar este sistema como un servicio gestionado, sin que el usuario final necesite realizar configuración alguna. Este proyecto no se limita a analizar el problema, sino que materializa una solución operativa: diseñar, construir y desplegar un sistema funcional que integre segmentación Zero Trust y simulaciones de ataque desde una red que simula al ISP, protegiendo así los hogares de forma masiva y automatizada. En un mundo donde el hogar es cada vez más inteligente, la seguridad no puede seguir siendo un lujo ni una responsabilidad individual: debe ser un servicio básico garantizado desde la red.

En el mundo del IoT ya existen soluciones de seguridad a los dipsositivos Enterprise IoT Security, que combina identificación automática de dispositivos mediante aprendizaje automático con políticas Zero Trust aplicadas en firewalls de nueva generación. Cisco propone una plataforma, mediante TrustSec y Cyber Vision. Fortinet ofrece FortiNAC, un controlador de acceso a red que soporta múltiples fabricantes y aplica segmentación dinámica con cuarentena automática. Zscaler implementa un modelo cloud-native con su Zero Trust Exchange. Cómo se logra ver las soluciones de empresas grandes existen pero su limitante principal es que son muy costosas y poco amigables con usuarios poco experimentados. Esto abre una gran brecha de seguridad para el sector doméstico y las soluciones domésticas son poco usuales o están descontinuadas. Por eso Quillnet abre una ventana de posibilidad para proteger las redes en los hogares.

Definición del problema

Los dispositivos IoT del hogar (cámaras IP, asistentes de voz, televisores inteligentes) se conectan habitualmente a la misma red plana que los equipos personales de los usuarios, sin aislamiento entre ellos, sin inspección del tráfico y, con frecuencia, con credenciales por defecto, firmware sin actualizar y comunicaciones sin cifrar. Cuando uno de estos dispositivos es comprometido, el atacante puede moverse lateralmente hacia el resto de la red, exfiltrar información privada o usar el dispositivo como parte de una botnet. El usuario promedio no puede detectar ni resolver esta situación por sí mismo, y no existe una solución doméstica accesible que combine segmentación Zero Trust, prevención de intrusiones y una interfaz comprensible para personas no técnicas.

Análisis de la problemática

Causas técnicas. Redes planas sin segmentación; contraseñas por defecto (el 35 % de los dispositivos mantiene “admin” como usuario, WifiTalents, 2026); vulnerabilidades CVE sin parchear por más de dos años en el 60 % de los dispositivos (JumpCloud, 2025); 75 % sin actualización automática y tráfico sin cifrar.

Causas humanas. Falta de conocimiento y de cultura de ciberseguridad; la protección depende de la iniciativa individual y la configuración de firewalls y VLAN excede las competencias del usuario común.

Causas de mercado. Las soluciones empresariales son costosas y complejas; las domésticas son limitadas, descontinuadas o no aplican segmentación estricta.

Efectos. Movimiento lateral del atacante, robo de credenciales e imágenes de cámaras, espionaje, participación involuntaria del hogar en botnets y ataques DDoS (Aisuru, BadBox 2.0) y exposición de la privacidad familiar.

Afectados. Hogares con dispositivos IoT, en particular usuarios sin conocimientos técnicos; indirectamente, los proveedores de internet y la infraestructura de la red.

Brecha identificada. No existe una solución doméstica accesible que integre segmentación Zero Trust, IPS en línea, respuesta automática y explicación de alertas en lenguaje sencillo.

Planteamiento del problema y pregunta de investigación

¿Es factible diseñar, desde la infraestructura de un proveedor de servicios de Internet, un sistema de seguridad basado en los principios de segmentación Zero Trust y ejercicios colaborativos de Purple Teaming, que reduzca significativamente el riesgo de accesos no autorizados a datos generados y transmitidos por dispositivos IoT residenciales, y cuáles son las condiciones técnicas y operativas que determinan su efectividad?

Objetivos De Investigación

1. Identificar las principales amenazas a los dispositivos IoT en el ambiente doméstico que pueden afectar a la información privada de los usuarios por medio de una recopilación de datos de inteligencia frente amenazas con el fin de entender las amenazas a los que los dispositivos conectados a internet enfrentan en el hogar
2. Enlistar los principales vectores de ataques que afectan a los dispositivos de Internet de las cosas en los hogares mediante investigaciones en páginas confiables de información de IoT y ciberseguridad.
3. Definir la superficie de ataque o los puntos de vulnerabilidad en los dispositivos conectados a internet que están menos protegidos para determinar el grado de protección que tienen estos dispositivos.
4. Escoger los sistemas de protección más seguros para proteger las redes de IoT que puedan mantener un control del tráfico y evitar ataques comunes de estos dispositivos.
5. Evaluar, mediante pruebas controladas, la efectividad de la solución implementada para detectar, bloquear y explicar amenazas, e identificar sus limitaciones.

Capítulo II

Marco teórico

Introducción al IoT

La tecnología ha llegado a revolucionar la vida del ser humano, con sus logros podemos garantizar que trabajan de mejor manera conectados a internet, estos dispositivos que se conectan a una nube virtual y se comunican entre sí se les conoce como IoT. Según Microsoft Azure, el Internet de las Cosas (IoT) es una red de dispositivos físicos como cámaras, automóviles y camas de hospital, equipados con sensores, software y conectividad a internet que les permite detectar e intercambiar datos. Los dispositivos IoT recopilan, transmiten y procesan datos en tiempo real para ayudar a las organizaciones a mejorar la eficiencia operativa, la toma de decisiones y la experiencia del cliente en diversos sectores, como la sanidad, la industria manufacturera y el transporte.
Los dispositivos de IoT siguen una serie de protocolos que merecen ser analizados para poder entender su funcionamiento. Los dispositivos IoT siguen cuatro pilares esenciales como lo explica McRae en su artículo:

Dispositivo. El primero de los cuatro pilares del IoT es el dispositivo. Este dispositivo puede ser cualquier cosa, desde un teléfono hasta aparatos médicos utilizados en hospitales, automóviles y otros electrodomésticos. Se trata de cualquier dispositivo de hardware capaz de transferir datos de un punto a otro a través de internet.

Datos. Uno de los principales objetivos y propósitos del IoT es la recopilación y el almacenamiento de datos. Estos datos procesados ayudarán a mejorar la aplicación y la funcionalidad de diferentes dispositivos y software.

Analítica. Permite analizar de forma eficiente y precisa cada dato recopilado, convirtiendo al dispositivo en una herramienta potente y valiosa en la vida de cualquier persona.

Conectividad. El último de los cuatro pilares, que permite que los otros tres funcionen en armonía. La conectividad ininterrumpida es fundamental para mantener el flujo de procesamiento y análisis de datos en tiempo real.

Este sistema permite a los objetos comunes conectarse a internet y mejorar su eficiencia. Aunque estos dispositivos llegaron para facilitarle la vida a los usuarios, conllevan un gran riesgo: si

alguno de estos dispositivos es comprometido por un agente malicioso, la información privada del usuario queda expuesta, lo cual supone una ciberamenaza que, de no ser tratada, podría llegar a dañar a las empresas y a los hogares por igual.

Riesgos y ataques de seguridad

Los riesgos de seguridad del IoT provienen de muchas vulnerabilidades técnicas en el hardware, el software y las comunicaciones de red de los dispositivos. Entre ellas se incluyen métodos de autenticación deficientes o débiles, la transmisión de datos en texto claro, el uso de una versión antigua del firmware y la exposición de servicios de red innecesarios. Los dispositivos vulnerables también ofrecen la posibilidad de explotar contraseñas predeterminadas, puertos abiertos y la falta de actualizaciones de seguridad del software oportunas, lo que crea un punto de entrada para los atacantes (SentinelOne, 2025).

Entre los vectores de ataque más comunes contra dispositivos IoT se encuentran:

Ataques de denegación de servicio distribuido (DDoS). Intento de dejar un dispositivo o red inaccesible para sus usuarios legítimos mediante la saturación con tráfico o solicitudes de información. Los dispositivos IoT pirateados pueden usarse para sobrecargar servidores y provocar interrupciones del servicio.

Ataques de intermediario (man-in-the-middle). Un atacante intercepta la comunicación entre dos dispositivos o redes y suplanta la identidad de uno de ellos para acceder a información confidencial.

Ataques de contraseñas. Intentos de acceder a un dispositivo o red adivinando o descifrando la contraseña mediante fuerza bruta.

Malware. Software diseñado para dañar o inhabilitar un dispositivo o red; los dispositivos IoT son particularmente vulnerables debido a que suelen tener menos seguridad que los dispositivos tradicionales.

Ataques físicos. Manipulación directa de dispositivos IoT poco protegidos contra manipulaciones, permitiendo a un atacante eludir medidas de seguridad y obtener acceso a datos o control del dispositivo.

Actualizaciones de firmware maliciosas. Reemplazo del firmware legítimo con código malicioso que puede usarse para tomar control del dispositivo o robar datos almacenados.

Conexiones inalámbricas no seguras. Conexiones que, al no estar debidamente protegidas, pueden ser aprovechadas para obtener acceso a la red o a los dispositivos conectados a ella.

Ataques de ingeniería social. Explotan debilidades humanas en lugar de vulnerabilidades técnicas, mediante suplantación de una entidad legítima para obtener acceso a información sensible.

Ataques de escuchas telefónicas (eavesdropping). Interceptación de comunicaciones entre dispositivos para obtener información confidencial, como contraseñas o números de tarjetas de crédito (Ip_Admin, 2024).

Los puertos que transportan la información de estos dispositivos normalmente son 8883 (MQTT seguro), 5671/5672 (AMQP/S), y los puertos web 443 (HTTPS/TLS) y 80 (HTTP). Como se puede esperar, la superficie de ataque también se ha expandido con el IoT, por lo que es fundamental desarrollar y seguir las mejores prácticas de seguridad para proteger dispositivos, usuarios y red de ciberamenazas (Fortinet, s.f.).

Superficie de ataque del IoT

Según IBM, una superficie de ataque es la suma de las vulnerabilidades, las vías o los métodos (a veces denominados vectores de ataque) que los atacantes pueden utilizar para obtener acceso no autorizado a la red o a los datos confidenciales, o para llevar a cabo un ciberataque. Los dispositivos IoT, al ser tan vulnerables, tienen una gran superficie de ataque que abarca gran parte de los vectores en los cuales pueden ser vulnerados. Según ManageEngine, estos vectores incluyen: hardware y dispositivos conectados; software y aplicaciones; puntos de entrada digital (puertos abiertos, servicios en ejecución, interfaces públicas); usuarios y credenciales; y red y conexiones (VPNs, Wi-Fi, firewalls mal configurados).

Muchos dispositivos IoT no vienen configurados de forma segura por defecto al ser entregados por los fabricantes y, debido a su naturaleza a menudo integrada, no se les aplican parches ni se protegen periódicamente una vez en producción (Jouly, s.f.). Un paso crítico para proteger los dispositivos del IoT es fortalecerlos mediante protección de endpoint, conectando

vulnerabilidades en puertos de alto riesgo (TCP, UDP), conexiones inalámbricas y comunicaciones no cifradas. También es vital una puerta de enlace web segura (SWG), que incluya control de aplicaciones, inspección profunda de HTTPS/SSL, y filtrado de URL. La seguridad de la red comienza con la configuración de un firewall seguro, complementado con autenticación multifactor (MFA) y monitoreo continuo de la actividad de la red. El cifrado, tanto simétrico como asimétrico, es fundamental para proteger los datos en movimiento entre dispositivos o hacia internet (Fortinet, s.f.).

Estadísticas actualizadas de la problemática IoT

Actualizando el panorama de la problemática de seguridad en dispositivos IoT, información más reciente confirma y agrava el diagnóstico presentado: se estima un promedio de 22 dispositivos IoT por hogar (Netgear/Bitdefender, 2025), con un registro de 29 ataques diarios por hogar, una cifra que triplica la reportada en 2024 (Bitdefender, 2025). A nivel global, IoT Analytics (2025) reporta una base de 21.1 mil millones de dispositivos IoT conectados a nivel mundial al cierre de 2025 (+14 % frente a 2024) y proyecta unos 39 mil millones hacia 2030 (cifra distinta de la de Transforma Insights por diferencias de metodología).

Otros datos relevantes muestran que un 35% de los dispositivos IoT continúan utilizando “admin” como usuario predeterminado (WifiTalents, 2026), que un 60% presenta vulnerabilidades conocidas (CVE) sin parchear desde hace más de dos años (JumpCloud, 2025), y que un 75% carece de actualización automática (WifiTalents, 2026). En cuanto a incidentes de gran escala, Cloudflare (2026) informó que en noviembre de 2025 mitigó el ataque de denegación de servicio distribuido (DDoS) más grande registrado hasta ese momento, con un pico de 31.4 terabits por segundo, asociado a las redes de bots Aisuru y Kimwolf, compuestas en gran parte por dispositivos domésticos comprometidos, principalmente televisores con Android.

Protocolos IoT y vulnerabilidades específicas

Además de los vectores de ataque generales descritos anteriormente, resulta pertinente detallar las vulnerabilidades específicas de los protocolos de comunicación más utilizados por los dispositivos IoT del entorno doméstico.

MQTT (Message Queuing Telemetry Transport). Utilizado por asistentes de voz y sensores inteligentes, no cuenta con autenticación por defecto ni control de acceso sobre sus canales de comunicación (topics), lo que permite ataques de suscripción no autorizada, inyección de comandos y denegación de servicio contra el intermediario (broker) de mensajes.

Zigbee. Utilizado en bombillas inteligentes, cerraduras y sensores de movimiento, transmite la clave de red sin cifrar durante el emparejamiento de dispositivos, lo que permite a un atacante capturar dicha clave y descifrar posteriormente todas las comunicaciones del dispositivo.

BLE (Bluetooth Low Energy). Utilizado en dispositivos ponibles, cerraduras inteligentes y dispositivos médicos, presenta vulnerabilidades conocidas en sus mecanismos de emparejamiento, entre ellas los ataques de tipo BIAS (Bluetooth Impersonation Attacks).

CoAP (Constrained Application Protocol). Utilizado en sensores con recursos limitados, opera sobre UDP sin estado y sin autenticación nativa en sus implementaciones básicas, haciéndolo vulnerable a ataques de intermediario, denegación de servicio por amplificación y falsificación de comandos.

Modelo Zero Trust

El modelo Zero Trust o «Confianza Cero» es un paradigma de ciberseguridad que parte del principio de que ningún usuario, dispositivo o servicio debe ser considerado confiable por defecto, independientemente de si se encuentra dentro o fuera del perímetro de la red. Cada solicitud de acceso debe ser autenticada, autorizada y verificada de forma continua (NIST, 2020).

Aplicado al contexto IoT, Zero Trust implica que cada dispositivo conectado al hogar debe ser tratado como potencialmente inseguro. Empresas como Palo Alto Networks han desarrollado soluciones que aplican este paradigma mediante la identificación dinámica de dispositivos a partir de múltiples atributos comportamentales, generando políticas de seguridad automatizadas (Palo Alto Networks, 2024). Este enfoque resulta particularmente adecuado para entornos donde los dispositivos no admiten la instalación de agentes de seguridad propios.

Segmentación de red

Otro método fundamental para proteger los dispositivos IoT es la segmentación. Según Cisco (2026), la segmentación es la práctica de dividir una red en partes más pequeñas llamadas segmentos, para mejorar la seguridad, el rendimiento, la capacidad de gestión y el cumplimiento normativo. Su efectividad radica en que aísla las redes y dispositivos en segmentos lógicos independientes, ampliamente utilizada en laboratorios para simular ataques y defensas.

Como señala Bitdefender (2025), segmentar la red consiste en dividirla en subredes aisladas, de modo que los dispositivos dentro de un segmento puedan comunicarse entre sí, pero las interacciones con otros segmentos estén estrictamente controladas o prohibidas. Esto garantiza que, en caso de una brecha de seguridad en un área, no se vean comprometidos todos los dispositivos simultáneamente, limitando el movimiento lateral del atacante. Los firewalls desempeñan un papel fundamental en esta estrategia, actuando como un amortiguador que separa la red de las amenazas externas, evitando el tráfico malicioso mientras permite el intercambio legítimo de datos, y para una protección más avanzada se recomienda el uso de firewalls con inspección profunda de paquetes (Nozomi Networks, 2025).

Purple Team

En ciberseguridad, los equipos tradicionales se dividen en Red Team (ofensivo, simula ataques) y Blue Team (defensivo, detecta y responde). El enfoque Purple Team integra ambas funciones en un proceso colaborativo donde los hallazgos de los ataques simulados alimentan directamente la mejora de las defensas (MITRE, 2024).

Esta metodología es particularmente útil en proyectos de seguridad IoT, ya que permite validar empíricamente que las medidas de protección funcionan ante escenarios reales. La organización MITRE mantiene el marco ATT&CK, una base de conocimiento estructurada sobre tácticas y técnicas de adversarios, ampliamente adoptada por la industria. Herramientas como MITRE Caldera permiten automatizar campañas de ataque mapeadas a este marco, ejecutándose contra entornos controlados para medir la capacidad de detección de las defensas (MITRE Caldera, 2024).

Soluciones del mercado

Las principales empresas del sector enterprise de ciberseguridad han desarrollado soluciones específicas para proteger dispositivos IoT. Palo Alto Networks ofrece Enterprise IoT Security, que combina identificación automática de dispositivos mediante aprendizaje automático con políticas Zero Trust aplicadas en firewalls de nueva generación (Palo Alto Networks, 2024). Cisco propone una arquitectura basada en la red como plataforma, mediante TrustSec y Cyber Vision (Cisco, 2024). Fortinet ofrece FortiNAC, un controlador de acceso a red que soporta múltiples fabricantes y aplica segmentación dinámica con cuarentena automática ante anomalías (Fortinet, 2024). Zscaler implementa un modelo cloud-native con su Zero Trust Exchange (Zscaler, 2024), y Akamai, mediante Guardicore Segmentation, se especializa en microsegmentación pura del tráfico lateral interno (Akamai, 2026).

Para el ámbito doméstico, las opciones son significativamente más limitadas. Productos como Bitdefender BOX (descontinuado en 2021), Firewalls Purple/Gold, Fingbox, o las funciones de seguridad integradas en routers de NETGEAR (Armor), TP-Link (HomeShield) y ASUS (AiProtection) representan los intentos más cercanos al consumidor, aunque ninguno ofrece simulación de ataques integrada ni segmentación dinámica al nivel de las soluciones enterprise (NETGEAR, 2025).

Esta brecha entre la sofisticación de las soluciones enterprise y la simplicidad de las soluciones consumer es precisamente el espacio que busca abordar el presente proyecto, traduciendo principios técnicos validados al contexto del hogar inteligente.

Casos reales documentados

Con el fin de sustentar empíricamente la relevancia del problema abordado por Quillnet, se documentan tres casos reales recientes de compromisos de seguridad en dispositivos IoT domésticos. Botnets Aisuru-Kimwolf (2025): Cloudflare reportó la mitigación, en noviembre de 2025, de un ataque DDoS de 31.4 terabits por segundo, el mayor registrado hasta ese momento, asociado a redes de bots compuestas en gran parte por dispositivos domésticos comprometidos, principalmente televisores con Android (Cloudflare, 2026). BadBox 2.0 (2025): en junio de 2025 el FBI advirtió que esta red de bots había comprometido millones de dispositivos conectados (como dispositivos de streaming para televisión, proyectores digitales, marcos digitales y sistemas de

infoentretenimiento para vehículos), muchos de ellos dentro de hogares y operando sin que sus propietarios lo supieran (FBI IC3, 2025). CVE-2023-1389 (TP-Link): vulnerabilidad de inyección de comandos sin autenticación (CVSS 8.8) en la interfaz web de los routers TP-Link Archer AX21, explotada de forma pública desde abril de 2023 por una variante de la botnet Mirai e incluida después en el catálogo de vulnerabilidades explotadas conocidas (KEV) de CISA (Trend Micro, 2023; CISA, s.f.).

Capítulo III

Marco Metodológico

Tipo de Investigación

El proyecto Quillnet se enmarca dentro de la investigación aplicada, un tipo de investigación que se caracteriza por utilizar el conocimiento científico y técnico ya existente con el propósito de resolver problemas concretos, en contraposición a la investigación básica, cuyo fin es la generación de conocimiento nuevo sin una aplicación inmediata definida. En el caso de Quillnet, el equipo no partió de la necesidad de descubrir nuevos principios de ciberseguridad, sino de la necesidad de aplicar principios ya consolidados en la industria, segmentación de red mediante VLANs, arquitectura de confianza cero (Zero Trust) y metodologías de validación ofensiva-defensiva conocidas como Purple Team, a un contexto que, hasta el momento, no contaba con una solución accesible: el hogar doméstico costarricense con dispositivos conectados a internet.

Esta orientación aplicada se sostiene en un diagnóstico previo, desarrollado durante la fase inicial del proyecto, en el cual el equipo investigó el panorama de soluciones de ciberseguridad ya disponibles en el mercado. Dicho diagnóstico permitió identificar que las soluciones empresariales existentes, ofrecidas por compañías como Palo Alto Networks, Fortinet, Checkpoint, Watchguard, Zscaler, Cisco y Akamai, si bien son robustas y efectivas, están diseñadas para entornos corporativos, requieren de personal técnico especializado para su configuración y mantenimiento, y tienen un costo que las vuelve inaccesibles para un hogar promedio. Por otro lado, las soluciones domésticas actualmente disponibles, como los routers con firewall básico de marcas como TP-Link o ASUS, ofrecen cierto nivel de protección, pero no implementan políticas de segmentación estrictas ni un modelo de seguridad basado en confianza cero. Es precisamente en ese vacío identificado entre ambos extremos del mercado donde Quillnet plantea su aporte: trasladar principios de seguridad de nivel empresarial a un dispositivo pensado para ser instalado y comprendido por un usuario sin conocimientos técnicos avanzados.

Enfoque del Proyecto

Dada la naturaleza multidimensional del proyecto, que combina el desarrollo de infraestructura de red, la validación experimental mediante pruebas de penetración, el desarrollo de software y la gestión de un producto pensado como servicio, Quillnet no puede analizarse bajo un único enfoque metodológico. Por ello, el proyecto adopta un enfoque mixto, que integra de manera complementaria los enfoques cualitativo y cuantitativo, cada uno aplicado a distintas dimensiones del desarrollo.

El enfoque cualitativo se hace presente en dos dimensiones del proyecto. Por un lado, en decisiones de diseño, percepción del usuario e interpretación de necesidades, como la construcción de la identidad visual, la paleta de colores y la experiencia de usuario de cada pantalla. Por otro lado, bajo un diseño documental y descriptivo, aplicado a la construcción del marco teórico: mediante fuentes confiables de inteligencia frente a amenazas, se describen los conceptos relevantes del IoT y los tipos de ataque que pueden vulnerarlo, fundamentando así el problema que Quillnet busca resolver.

El enfoque cuantitativo, por su parte, se hace presente en aquellas dimensiones del proyecto que sí pueden medirse de forma objetiva y numérica. Ejemplos de esta dimensión son los registros generados por el sistema de detección de intrusos Suricata, los resultados obtenidos durante las pruebas de penetración (por ejemplo, si un ataque simulado fue detectado y bloqueado o no), los tiempos de respuesta del sistema ante una alerta, y los datos financieros asociados al costo del prototipo y a su eventual modelo de negocio.

Justificación del Enfoque Combinado

La decisión de adoptar un enfoque metodológico combinado, en lugar de uno único, responde a que el desarrollo de Quillnet integra tres procesos de naturaleza claramente distinta, los cuales ocurren de manera simultánea y no de forma secuencial excluyente entre sí.
En primer lugar, el proyecto contiene un componente de investigación aplicada, presente desde su fase inicial, en la cual se estudiaron tanto las soluciones existentes en el mercado como los protocolos de comunicación utilizados por los dispositivos IoT (entre ellos MQTT, Zigbee, Bluetooth Low Energy y CoAP), con el objetivo de comprender sus vulnerabilidades características y fundamentar técnicamente la necesidad del proyecto.

En segundo lugar, el proyecto contiene un componente de desarrollo iterativo, dado que el sistema no se construyó de una sola vez, sino a través de fases sucesivas, cada una sujeta a revisión, corrección de errores y ajuste con base en los resultados obtenidos en la fase anterior. Es importante aclarar que, si bien las defensas académicas del proyecto (primera, segunda y tercera defensa) constituyen hitos formales de evaluación y funcionan como marcadores temporales del avance esperado, el trabajo técnico interno del equipo no depende exclusivamente de esas fechas: el desarrollo continúa de manera constante entre una defensa y otra, y los bloqueos técnicos, como el que enfrentó el equipo con la instalación del sistema de detección de intrusos sobre OpenWRT (resuelto al migrar a Raspberry Pi OS Lite), se documentan y resuelven de forma continua, independientemente del calendario de defensas.

En tercer lugar, el proyecto contiene un componente de diseño experimental, particularmente visible en la fase de validación de seguridad mediante la metodología Purple Team. Esta metodología consiste en plantear escenarios de ataque controlado sobre el propio sistema, ejecutarlos, observar si las medidas de defensa implementadas logran detectarlos y neutralizarlos, y ajustar la configuración del sistema en función de los resultados obtenidos. Este componente del proyecto se asemeja metodológicamente a un diseño experimental clásico, en el cual existe una variable manipulada (el tipo de ataque simulado) y una variable de respuesta observada (si el sistema de detección logra identificar y bloquear dicho ataque).

La combinación de estos tres componentes metodológicos permite que el proyecto sea evaluado no únicamente por el producto final entregado, sino por la solidez, coherencia y trazabilidad del proceso que condujo a dicho producto, criterio que resulta central dentro de los parámetros de evaluación de Expotec, donde se espera que el equipo pueda explicar y defender el razonamiento detrás de cada decisión tomada.

Diseño de la Investigación

Fases del Proyecto

El desarrollo de Quillnet se organizó en cinco fases consecutivas, cada una con objetivos particulares y resultados esperados definidos con anticipación por el equipo.

La primera fase, denominada fase de investigación y definición, se desarrolló entre febrero y marzo de 2026, y se encuentra completada. Durante este período, el equipo se concentró en la definición precisa del problema que el proyecto buscaría resolver: la falta de soluciones de ciberseguridad accesibles para dispositivos IoT dentro del hogar. Como parte de esta fase, se realizó un estudio del panorama completo de soluciones de ciberseguridad empresarial disponibles en el mercado, con el fin de comprender qué tecnologías existían y en qué punto se encontraba el vacío que el proyecto podría cubrir. De manera paralela a esta investigación técnica, el equipo trabajó en la construcción de la identidad del proyecto, definiendo el nombre Quillnet, la mascota Quill, un erizo cuyo mecanismo de defensa natural se adoptó como metáfora central del enfoque de seguridad del proyecto y el eslogan que representa su filosofía de trabajo. Asimismo, en esta fase se estableció la arquitectura inicial de cinco capas del sistema y se investigó el fundamento técnico relacionado con los protocolos de comunicación IoT y el modelo de seguridad Zero Trust.

La segunda fase, denominada fase de documentación y primera defensa, se desarrolló entre abril y mayo de 2026, y se encuentra completada. En esta fase se elaboró el trabajo escrito inicial del proyecto y se preparó al equipo para su presentación ante el panel evaluador, logrando la aprobación de la primera defensa en mayo de 2026. Dentro de esta fase ocurrió una decisión metodológica de gran relevancia para el rumbo del proyecto: el pivote desde un enfoque de laboratorio virtual, en el cual el sistema dependería únicamente de máquinas virtuales y entornos simulados, hacia un enfoque de dispositivo físico real. Esta decisión se tomó con base en el criterio de que un prototipo físico, operando sobre hardware real y dispositivos IoT reales, constituye una demostración más contundente y creíble de la funcionalidad del sistema frente a un panel evaluador, en comparación con una simulación completamente virtual.

La tercera fase, denominada fase de construcción del sistema, se desarrolló entre junio y julio de 2026 y se encuentra completada. Durante esta fase se configuró el hardware inicial, se seleccionó el access point utilizado para distribuir la red inalámbrica y se identificó que la versión

25.12 de OpenWRT, al usar el gestor de paquetes apk, impedía la instalación de Suricata. Ante ello, el equipo reemplazó OpenWRT por Raspberry Pi OS Lite (Debian), lo que permitió completar la instalación del sistema de detección y prevención de intrusiones.

La cuarta fase, denominada fase de backend y frontend, se desarrolló entre agosto y septiembre de 2026 y se encuentra completada. En ella se construyeron el backend (Flask y PostgreSQL, desplegado en Render), los servicios del gateway, el frontend web y la activación de Suricata en modo IPS, respaldados por pruebas automatizadas del backend (238 pruebas al corte del 4 de noviembre de 2026). Su hito de evaluación intermedio fue la segunda defensa del proyecto, en agosto de 2026.

La quinta fase, denominada fase de pulido y Expotec, se desarrolla entre octubre y noviembre de 2026 y se encuentra en progreso. Comprende el refinamiento final del sistema en todas sus capas, de cara a su presentación pública. Su primer hito fue la tercera defensa del proyecto, realizada entre el 12 y el 16 de octubre de 2026. Desde el 1 de noviembre de 2026 el código se encuentra congelado (solo se admiten correcciones críticas) para priorizar la estabilidad, y la fase cierra con la presentación final en Expotec, del 13 al 15 de noviembre de 2026.

Resulta pertinente aclarar, como parte del diseño metodológico de esta investigación, que las fechas de las defensas académicas no determinan de manera estricta el ritmo de avance técnico del proyecto. Dichas defensas constituyen hitos formales de evaluación externa, pero el equipo continúa trabajando de manera constante entre una y otra, ajustando el ritmo de trabajo según la disponibilidad de cada integrante y según la complejidad de los obstáculos técnicos que se presentan durante el desarrollo.

Población y Contexto de Aplicación

Quillnet está diseñado para operar dentro del contexto del hogar doméstico costarricense, específicamente en aquellos hogares que cuentan con dispositivos conectados a internet de tipo IoT, tales como cámaras de seguridad, asistentes de voz y televisores inteligentes. El perfil de usuario para el cual está pensado el sistema es una persona sin conocimientos técnicos

especializados en redes o ciberseguridad, capaz de instalar y utilizar el sistema sin requerir asistencia técnica externa constante.

El prototipo desarrollado para su presentación en Expotec 2026 constituye una prueba de concepto funcional a escala reducida. Dicho prototipo simula las condiciones de una red doméstica real mediante la conexión de dispositivos IoT de prueba a través del access point configurado por el equipo, permitiendo demostrar el funcionamiento del sistema de segmentación, detección y respuesta ante amenazas en un entorno controlado, pero representativo de las condiciones reales de uso del producto.

Conceptos técnicos del sistema

Para comprender el funcionamiento interno del sistema Quillnet, es necesario definir previamente algunos conceptos técnicos fundamentales sobre los que se apoya su arquitectura.

API (interfaz de programación de aplicaciones). Conjunto de reglas y funciones que permite que distintos componentes de software se comuniquen entre sí: recibe una solicitud, la transmite al componente correspondiente y devuelve una respuesta. En Quillnet se utilizan API con responsabilidades diferenciadas: una desarrollada en Python que recibe los registros generados por Suricata y los envía al backend, y una externa de inteligencia artificial que interpreta esa información técnica y la convierte en una explicación comprensible para el propietario de la vivienda.

Flask. Framework ligero para desarrollar aplicaciones web con Python, que provee las herramientas necesarias para crear rutas, formularios, sesiones y conexiones con bases de datos (Pallets Projects, 2025). En Quillnet se utiliza para desarrollar el backend (API REST de 27 rutas), servido con Gunicorn en Render, que conecta el frontend con la base de datos PostgreSQL, el servicio de traducción de alertas con inteligencia artificial y los comandos hacia el gateway.

Suricata. Herramienta de ciberseguridad de código abierto que puede funcionar simultáneamente como sistema de detección de intrusos, sistema de prevención de intrusos y herramienta de monitoreo de red. Analiza el tráfico de la red y utiliza reglas para identificar eventos sospechosos, generando alertas y registros en formato JSON mediante su sistema EVE.

Autenticación de dos factores (2FA). Mecanismo de seguridad que exige dos métodos distintos de verificación de identidad, pertenecientes a categorías diferentes. En Quillnet, el inicio de

sesión exige correo, contraseña y un código de 6 dígitos enviado por correo, y el desbloqueo de un dispositivo exige además un código temporal enviado por correo, de modo que una sesión abierta no baste para retirar una protección.

Backend y frontend. El frontend es la parte visual de la aplicación con la que interactúa directamente el usuario. El backend es la parte interna encargada de recibir las solicitudes del frontend, aplicar la lógica del sistema, consultar la base de datos y comunicarse con los demás componentes. Aunque ambos se ejecuten sobre el mismo equipo durante la etapa de prototipo, constituyen componentes lógicamente diferenciados.

Servidor. Equipo, programa, o combinación de ambos, que proporciona información o funciones a un cliente. En la versión final de Quillnet el backend se ejecuta en un servidor en la nube (Render), mientras que el gateway (Raspberry Pi) actúa como cliente: siempre inicia la conexión hacia el servidor, porque se encuentra detrás de NAT y no puede recibir conexiones entrantes.

PostgreSQL. Sistema de gestión de bases de datos relacional de código abierto (PostgreSQL Global Development Group, 2025). En Quillnet reemplazó a SQLite, usado en el prototipo local inicial, y almacena cuentas, dispositivos, alertas y las traducciones generadas por la inteligencia artificial, con los datos de cada hogar separados mediante un identificador de gateway (arquitectura multi-tenant).

VLAN e IEEE 802.1Q. Una VLAN (Virtual Local Area Network) es un dominio de difusión lógico creado sobre la misma infraestructura física; el estándar IEEE 802.1Q define el etiquetado de las tramas Ethernet con un identificador de VLAN (IEEE, 2022). En Quillnet separa los dispositivos en cuatro segmentos.

nftables. Framework de filtrado de paquetes del kernel de Linux, sucesor de iptables, que organiza las reglas en tablas y cadenas con políticas por defecto (Netfilter Project, s.f.). En Quillnet se usa con política forward drop: se deniega todo lo que no esté explícitamente permitido.

NAT, DHCP, DNS y dnsmasq. NAT traduce direcciones privadas a una dirección pública para salir a Internet (Srisuresh & Egevang, 2001); las direcciones privadas siguen el RFC 1918 (Rekhter et al., 1996). DHCP asigna automáticamente direcciones IP a los clientes (Droms, 1997) y

DNS resuelve nombres de dominio; dnsmasq presta ambos servicios con un ámbito independiente por VLAN (Kelley, s.f.).

IPS y NFQUEUE. Un IDS detecta y alerta; un IPS además bloquea el tráfico malicioso en línea. NFQUEUE es una cola del kernel de Linux que entrega los paquetes a un programa en espacio de usuario, como Suricata, para decidir si se aceptan o se descartan (Netfilter Project, s.f.; Open Information Security Foundation, 2025). Las reglas Emerging Threats Open son firmas comunitarias de amenazas (Proofpoint, s.f.). El modo bypass (fail-open) mantiene el tráfico si el motor deja de responder, priorizando la disponibilidad.

JWT y multi-tenant. Un JSON Web Token es un estándar de tokens firmados para transmitir identidad y permisos entre partes (Jones et al., 2015). En Quillnet cada solicitud lleva el identificador del gateway, con lo que un mismo sistema atiende a varios hogares con datos aislados (multi-tenant).

scrypt y limitación de tasa. scrypt es una función de derivación de claves costosa en memoria, diseñada para dificultar la fuerza bruta contra contraseñas almacenadas (Percival & Josefsson, 2016). La limitación de tasa (rate limiting) restringe el número de solicitudes por intervalo para mitigar intentos repetidos de acceso. Quillnet almacena las contraseñas con scrypt (librería werkzeug.security, parámetros scrypt:32768:8:1) y una sal aleatoria por contraseña. Los códigos de verificación se guardan como HMAC-SHA256, un código de autenticación basado en hash con una clave secreta del servidor.

SSE (Server-Sent Events). Mecanismo HTTP por el cual el servidor envía eventos al navegador en tiempo real (WHATWG, s.f.); Quillnet lo usa para mostrar alertas al instante.

Web Push y PWA. Web Push es el estándar que permite a un servidor enviar notificaciones al dispositivo de la persona aunque la página esté cerrada, mediante claves VAPID y un service worker. Una PWA (aplicación web progresiva) es una página web que se instala como aplicación. Quillnet implementa ambas; en iPhone la notificación requiere instalar la PWA desde el navegador.

Modelo de lenguaje (LLM) y Groq. Un LLM es un modelo de inteligencia artificial entrenado para comprender y generar texto. Quillnet usa Llama 3.3 70B (Meta, 2024) mediante la API de Groq (Groq, s.f.) para traducir alertas técnicas a lenguaje sencillo. Al modelo solo viajan la firma de la regla, su categoría, la acción y el protocolo; nunca direcciones IP, direcciones MAC ni nombres de

dispositivos. Si el modelo principal falla, se reintenta con un modelo de respaldo (llama-3.1-8b-instant) y, si también falla, la alerta se guarda con un texto de respaldo según su severidad.

Raspberry Pi 4 Modelo B. Computadora de placa única de bajo costo con procesador ARM de cuatro núcleos y puerto Ethernet Gigabit (Raspberry Pi Ltd, 2025), suficiente para ejecutar de forma simultánea el sistema operativo, el firewall y Suricata.

Justificación del stack técnico

La selección de cada tecnología dentro de la arquitectura de Quillnet respondió a criterios técnicos específicos, y no a una elección arbitraria.

Suricata frente a Snort. Se seleccionó Suricata como sistema de detección de intrusos, en lugar de Snort, principalmente porque Suricata es multi-hilo, lo que le permite aprovechar los cuatro núcleos de procesamiento de la Raspberry Pi 4, mientras que Snort ejecuta su análisis en un solo hilo, resultando más lento ante el tráfico de múltiples dispositivos. Adicionalmente, Suricata cuenta con mejor soporte nativo para protocolos IoT modernos, es compatible con las reglas ya existentes de Snort, y admite un modo de prevención activo.

Raspberry Pi OS Lite frente a OpenWRT. Inicialmente se seleccionó OpenWRT como sistema operativo de red por ser una de las opciones más estables para arquitectura ARM, por su soporte de VLANs bajo el estándar 802.1Q y por incluir un firewall nftables integrado. Sin embargo, su versión 25.12 incorporó el gestor de paquetes apk y no permitió instalar Suricata, por lo que se reemplazó por Raspberry Pi OS Lite de 64 bits (Debian 13), que ofrece mayor compatibilidad con Suricata, Python y el resto de tecnologías del proyecto, además de amplia documentación para configurar funciones de router, switch y gateway.

VLANs frente a reglas de firewall únicamente. Se optó por complementar las reglas de firewall con segmentación mediante VLANs porque estas crean un aislamiento a nivel de red: un dispositivo conectado a una VLAN determinada no puede ver el tráfico correspondiente a otra VLAN. Esto constituye una barrera de aislamiento físico-lógica, adicional y complementaria a las reglas de firewall nftables aplicadas sobre el gateway.

Arquitectura por Capas

Capa Física

La capa física del sistema está compuesta por una Raspberry Pi 4 Modelo B, con 4 gigabytes de memoria RAM, la cual funciona como el gateway central de toda la red doméstica. La selección de este hardware respondió a criterios específicos de costo, disponibilidad en el mercado local y capacidad de procesamiento suficiente para ejecutar de manera simultánea el sistema operativo de red, el firewall y el sistema de detección de intrusos, sin comprometer el rendimiento general del dispositivo.

Dentro de esta capa, el gateway recibe Internet por su interfaz inalámbrica (wlan0), conectada al hotspot de un teléfono que sustituye al proveedor de internet durante la demostración, y entrega la red segmentada por su puerto Ethernet nativo (eth0) hacia el access point TP-Link EAP110, mediante un enlace con etiquetas 802.1Q. Esta distinción es de particular importancia metodológica, ya que garantiza que todo el tráfico del hogar, tanto entrante como saliente, deba pasar obligatoriamente a través de la Raspberry Pi antes de llegar a cualquier dispositivo, lo cual constituye la base física que hace posible la aplicación de las políticas de segmentación y de seguridad Zero Trust.

Capa de Segmentación y Detección

Sobre la capa física descrita anteriormente se instaló Raspberry Pi OS Lite de 64 bits (Debian 13) como sistema del gateway, con nftables como firewall, NetworkManager para las VLAN, dnsmasq para DHCP/DNS y Suricata como sistema de prevención de intrusiones. Previamente se había probado OpenWRT, descartado por las razones expuestas en la justificación del stack técnico.

Proceso de instalación y configuración, documentado paso a paso:

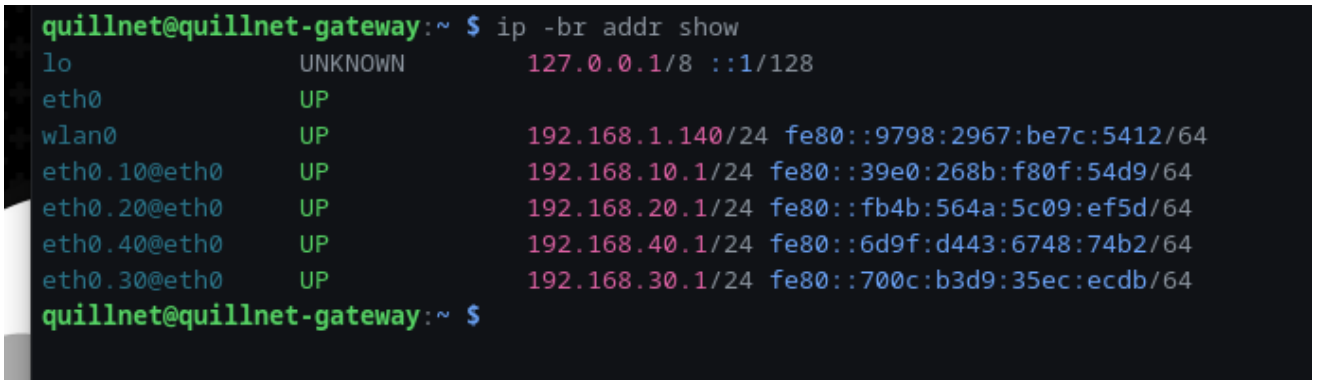
Se instaló Raspberry Pi OS Lite de 64 bits en la tarjeta microSD de la Raspberry Pi 4 y se actualizó el sistema, dejando el gateway listo para su configuración de red.

Se configuró la salida a Internet por la interfaz inalámbrica wlan0, conectada al hotspot del teléfono, y se habilitó el reenvío de paquetes con traducción de direcciones (NAT) hacia esa interfaz.

Se configuró el access point TP-Link EAP110 como punto de distribución inalámbrico, conectado al puerto Ethernet de la Raspberry Pi mediante un enlace con etiquetas 802.1Q, publicando un SSID por cada VLAN.

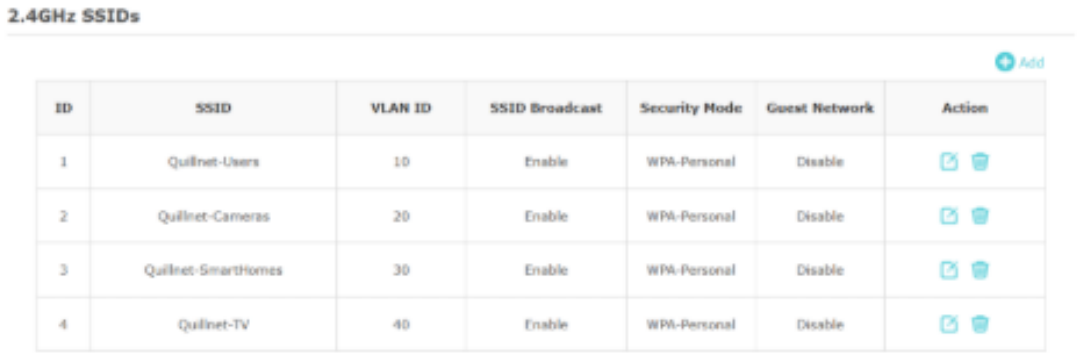
Con la conectividad básica establecida, se crearon las VLAN con NetworkManager (nmcli) y se asignó a cada una su propio servicio DHCP/DNS con dnsmasq, para separar los distintos tipos de dispositivos de la red, quedando distribuidas de la siguiente manera:

- VLAN 10: Usuarios (192.168.10.0/24)
- VLAN 20: Cámaras IP (192.168.20.0/24)
- VLAN 30: Asistentes de voz (192.168.30.0/24)
- VLAN 40: Smart TV (192.168.40.0/24)



(Fuente: Propia del equipo)

El access point TP-Link EAP110 (hardware v4.0, firmware 5.0.4 de 2022) publica cuatro SSID, uno por VLAN: Quillnet-Users (VLAN 10), Quillnet-Cameras (VLAN 20), Quillnet-SmartHomes (VLAN 30, asistentes de voz) y Quillnet-TV (VLAN 40). Las cuatro redes usan WPA2-PSK con cifrado AES y una contraseña distinta por VLAN, de modo que conocer la clave de una red no da acceso a las demás; la red de invitados está deshabilitada y la radio opera en 2.4 GHz (canal 11, 802.11 b/g/n). El AP se administra en la VLAN 10 (192.168.10.119). Su puerto LAN negocia a 100 Mbps full dúplex, por lo que el enlace troncal 802.1Q entre la Raspberry Pi y el AP está limitado a Fast Ethernet.



(Fuente: Propia del equipo)

Una vez creadas las VLAN, se configuró el firewall nftables bajo el modelo Zero Trust, con política forward drop: solo se permite el tráfico explícitamente autorizado. Primero se permitió que todas las VLAN pudieran acceder a internet, para que los dispositivos funcionarán con normalidad. Después se bloqueó el acceso desde las VLAN de cámaras, asistentes de voz y Smart TV hacia la VLAN de usuarios, de forma que, si alguno de esos dispositivos llegara a verse comprometido, no pudiera acceder a la red donde están los equipos de los usuarios. También se permitió que la VLAN de usuarios pudiera comunicarse con las demás para administrar los dispositivos IoT cuando fuera necesario. Por último, se bloqueó la comunicación entre las VLAN de dispositivos IoT entre sí, para que cada una permaneciera aislada de las otras.

Como parte de esta misma capa se instaló Suricata 7.0.10 en modo IPS inline, integrado con nftables mediante NFQUEUE y con bypass (fail-open) para no dejar el hogar sin Internet si el motor falla. Tiene cargadas 52,246 reglas (Emerging Threats Open y reglas propias de demostración), sin ninguna fallida, de las cuales 12,860 operan en modo drop, es decir, bloquean el tráfico de forma real. Las reglas propias son cinco: 9000001 (herramienta sqlmap), 9000002 (ruta /quillnet-ataque), 9000010 (control remoto), 9000011 (tráfico sospechoso) y 9000020 (MQTT sin cifrar). Por diferencia, 52,241 reglas provienen de Emerging Threats Open y 5 son propias. El obstáculo técnico identificado inicialmente con OpenWRT quedó resuelto al migrar a Raspberry Pi OS Lite.

```
quillnet@quillnet-gateway:~ $ sudo grep -c '^drop' /var/lib/suricata/rules/suricata.rules
12860
quillnet@quillnet-gateway:~ $ sudo suricata -T -c /etc/suricata/suricata.yaml -v 2>&1 | grep -E "rules (successfully )?loaded|rules failed|rules skipped"
Info: detect: 2 rule files processed. 52249 rules successfully loaded, 0 rules failed, 0
quillnet@quillnet-gateway:~ $
```

(Fuente: Propia del equipo)

Suricata se inicia mediante un archivo de sobrescritura de systemd (override.conf) con la opción -q 0, que lo conecta a la cola NFQUEUE 0 del kernel; esto lo habilita como IPS en línea (puede descartar paquetes) y no solo como IDS pasivo. Los tres servicios propios del gateway (quillnet-agente, quillnet-vigia y quillnet-control) son unidades de systemd con reinicio automático (Restart=always). quillnet-control corre como root porque necesita manipular nftables, mientras que quillnet-agente y quillnet-vigia corren con el usuario sin privilegios quillnet, en aplicación del principio de mínimo privilegio.

```
quillnet@quillnet-gateway:~ $ systemctl status quillnet-agente quillnet-vigia quillnet-control --no-pager
● quillnet-agente.service - Quillnet - Agente de alertas Suricata
     Loaded: loaded (/etc/systemd/system/quillnet-agente.service; enabled; preset: enabled)
    Drop-In: /etc/systemd/system/quillnet-agente.service.d
             └─unbuffered.conf
     Active: active (running) since Thu 2026-11-05 15:35:07 UTC; 10min ago
 Invocation: 429ee27bcd4741cb9fd94f3ba47127ae
   Main PID: 886 (python3)
      Tasks: 1 (limit: 3967)
        CPU: 916ms
     CGroup: /system.slice/quillnet-agente.service
             └─886 /usr/bin/python3 /home/quillnet/agente/agente.py

Nov 05 15:35:07 quillnet-gateway systemd[1]: Started quillnet-agente.service - Quillnet - Agente de alertas Suricata.
Nov 05 15:35:08 quillnet-gateway python3[886]: Agente Quillnet iniciado, vigilando /var/log/suricata/eve.json

● quillnet-vigia.service - Quillnet - Vigia de dispositivos nuevos
     Loaded: loaded (/etc/systemd/system/quillnet-vigia.service; enabled; preset: enabled)
    Drop-In: /etc/systemd/system/quillnet-vigia.service.d
             └─unbuffered.conf
     Active: active (running) since Thu 2026-11-05 15:35:07 UTC; 10min ago
 Invocation: f8f88198d3a14420816d5f03bb260987
   Main PID: 883 (python3)
      Tasks: 1 (limit: 3967)
        CPU: 9.772s
     CGroup: /system.slice/quillnet-vigia.service
             └─883 /usr/bin/python3 /home/quillnet/vigia/vigia_dispositivos.py

Nov 05 15:42:46 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:43:03 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:43:20 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:43:38 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:43:55 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:44:13 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:44:30 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:44:48 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:45:05 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200
Nov 05 15:45:23 quillnet-gateway python3[883]: Presencia: 2 conectados (192.168.20.122, contthi) -> HTTP 200

● quillnet-control.service - Quillnet - API de control de acceso (bloqueo por MAC)
     Loaded: loaded (/etc/systemd/system/quillnet-control.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-11-05 15:35:06 UTC; 10min ago
 Invocation: 859663ecb1234c75b418aff010de3632
   Main PID: 857 (python3)
      Tasks: 1 (limit: 3967)
        CPU: 3.388s
     CGroup: /system.slice/quillnet-control.service
             └─857 /usr/bin/python3 -u /home/quillnet/vigia/control_acceso.py

Nov 05 15:35:06 quillnet-gateway systemd[1]: Started quillnet-control.service - Quillnet - API de control de acceso (bloqueo por MAC).
Nov 05 15:35:07 quillnet-gateway python3[857]: Reconciliando 6 dispositivo(s) bloqueado(s)...
Nov 05 15:35:07 quillnet-gateway python3[857]: Regla reaplicada: 0e:7b:62:03:ed:34
```

(Fuente: Propia del equipo)

Capa de Validación y Pentesting

La validación de las medidas de seguridad implementadas en las capas anteriores se realiza mediante la aplicación de la metodología Purple Team, un enfoque de ciberseguridad que combina la perspectiva ofensiva propia de un equipo Red Team, encargado de simular ataques reales contra el sistema, con la perspectiva defensiva propia de un equipo Blue Team, encargado de detectar y responder ante dichos ataques.

Selección de tecnologías. Para correr la máquina virtual de Kali Linux se evaluaron inicialmente dos hipervisores de escritorio: VirtualBox y VMware Workstation Pro; se utilizó VMware por su mayor rendimiento y menor sobrecarga.

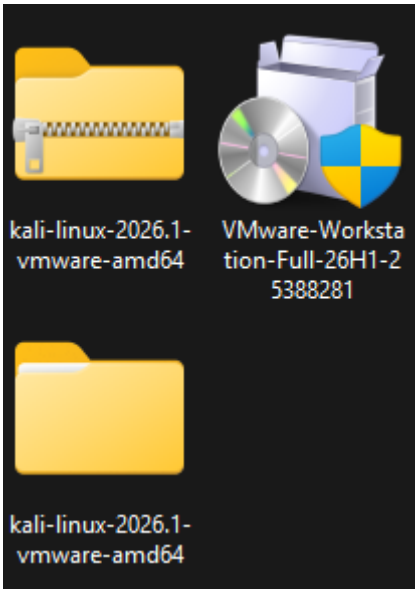
En cuanto al sistema operativo de pruebas, se escogió Kali Linux antes que otras alternativas debido a que es el estándar de la industria para el hacking ético y el pentesting, y entre sus mayores ventajas está el ser una caja de herramientas bastante completa y ya configurada

.

Proceso de instalación y configuración, documentado semana a semana:

En primer lugar, se descargó el instalador original de VMware para Windows de 64 bits, junto con la carpeta de la máquina virtual de Kali Linux, y se procedió a instalar el Workstation de VMware.

Nota: las capturas siguientes corresponden a la etapa inicial del laboratorio con VMware Workstation en Windows; el entorno vigente corre sobre KVM/QEMU (virt-manager) en Fedora.

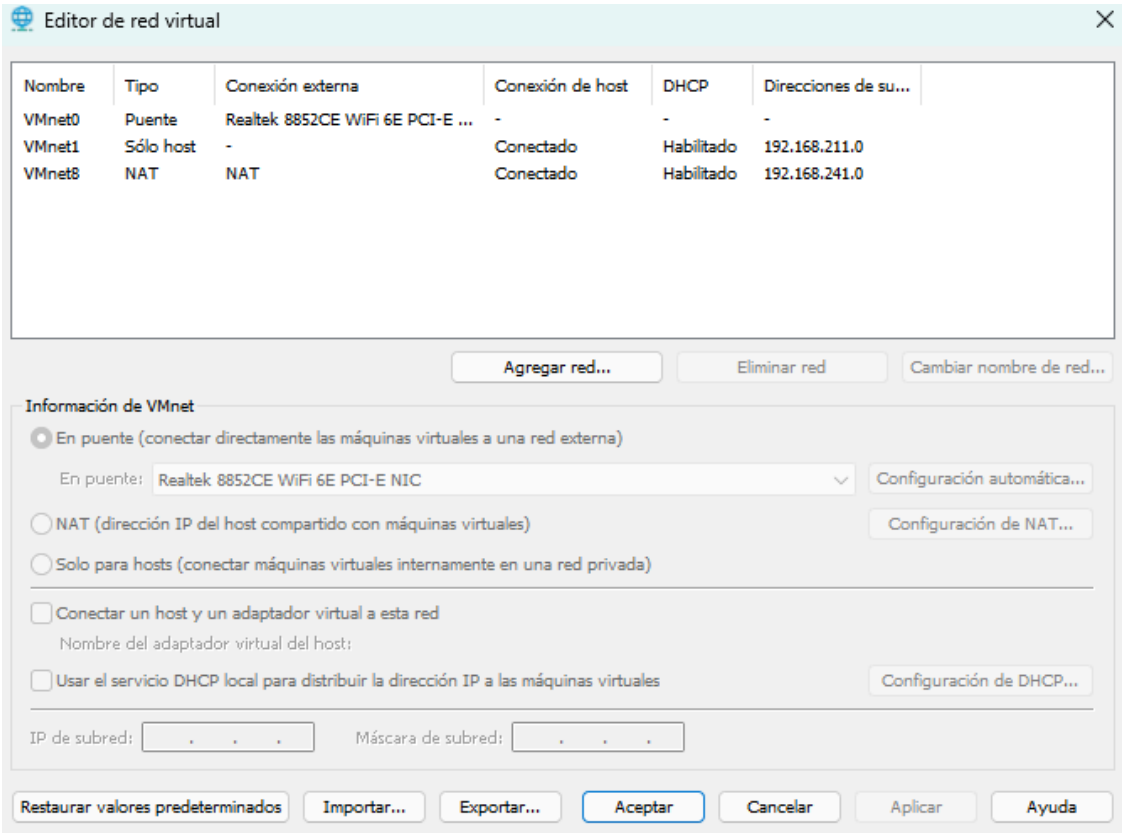


(Fuente: Propia del equipo)

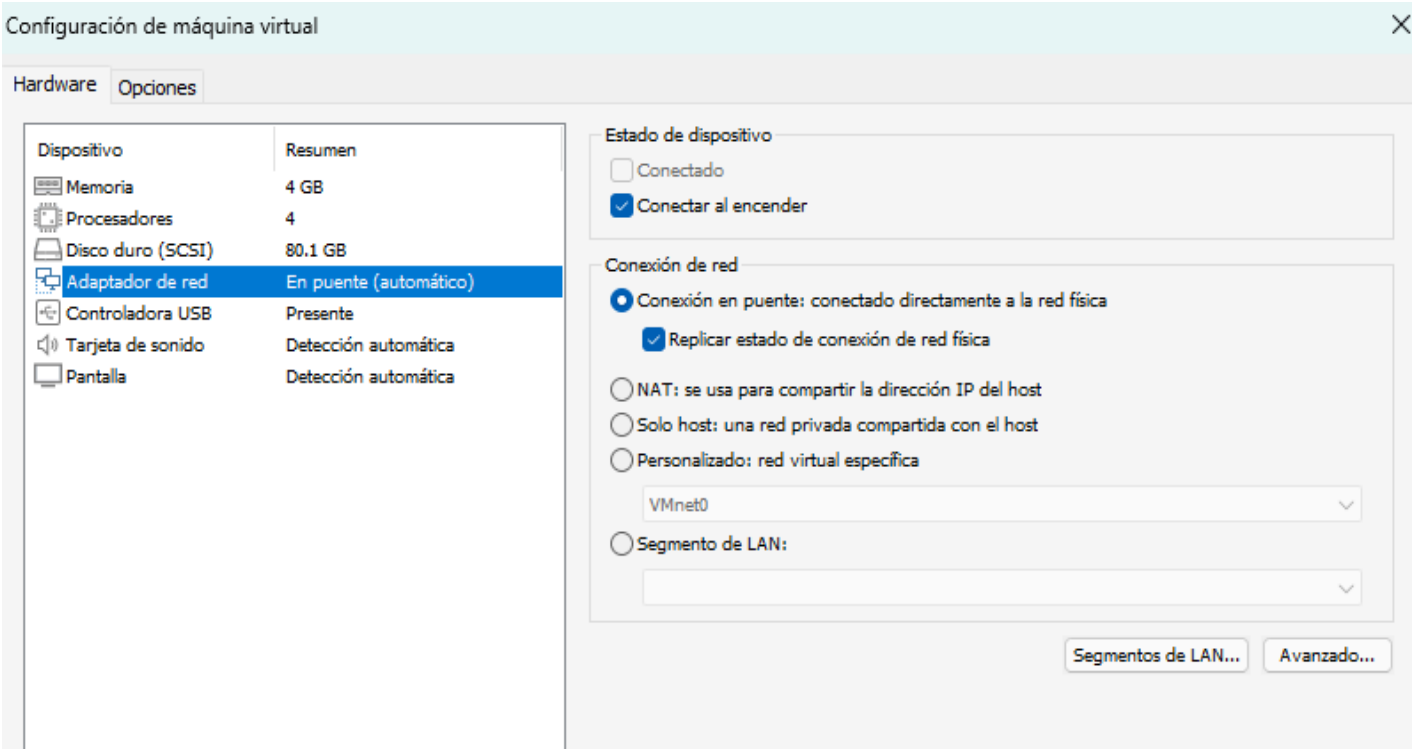
Posteriormente, se dedicó una etapa a la instalación de la máquina virtual dentro del Workstation de VMware y al ajuste de los recursos consumidos por esta.



(Fuente: Propia del equipo)

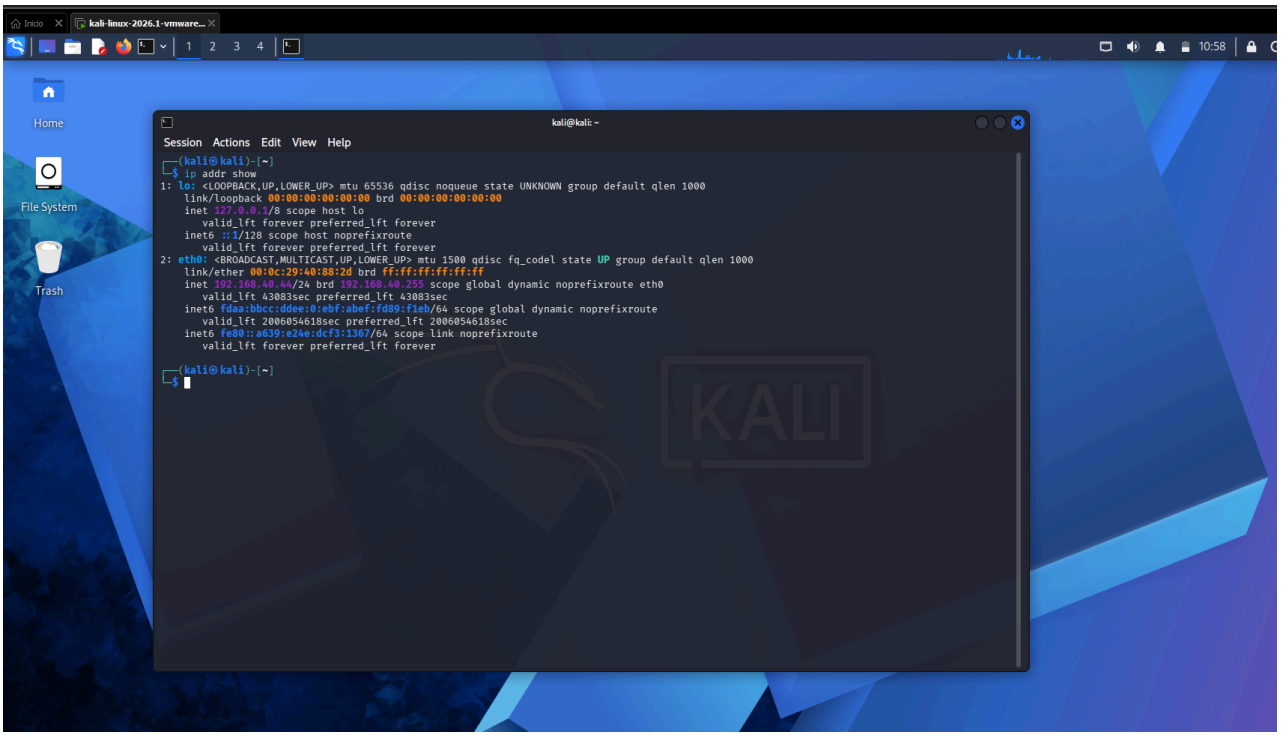


(Fuente: Propia del equipo)

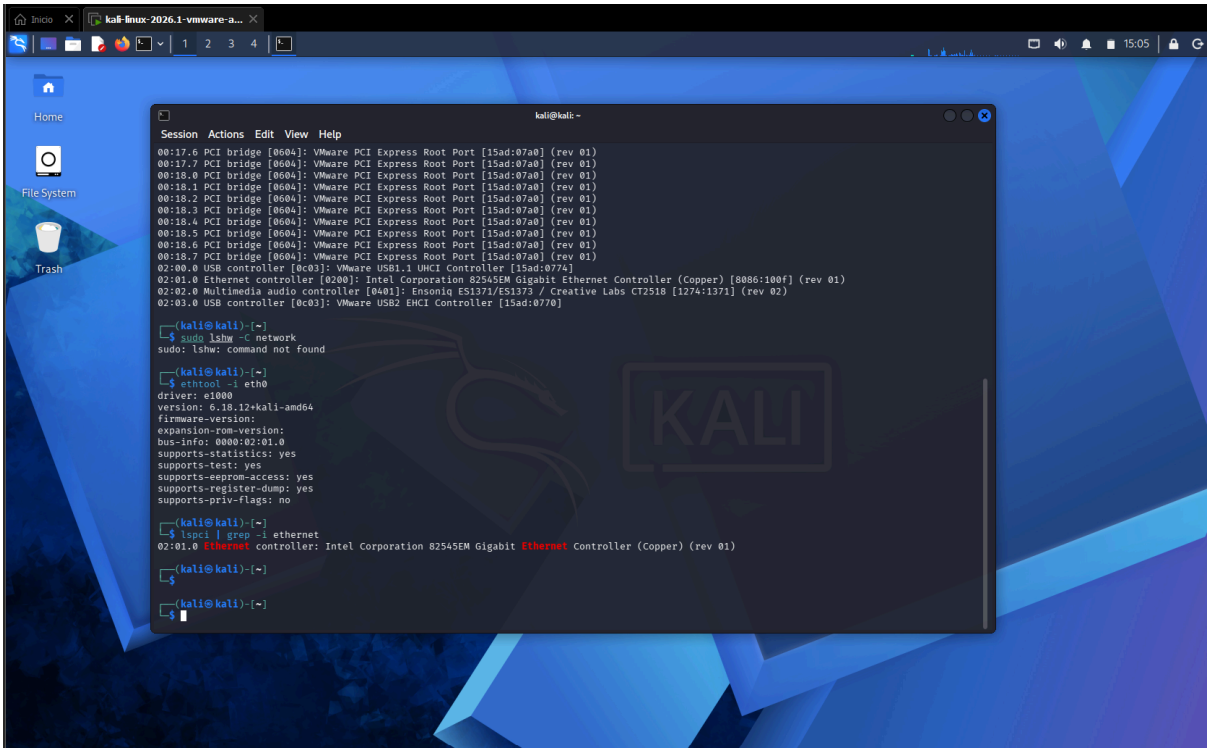


(Fuente: Propia del equipo)

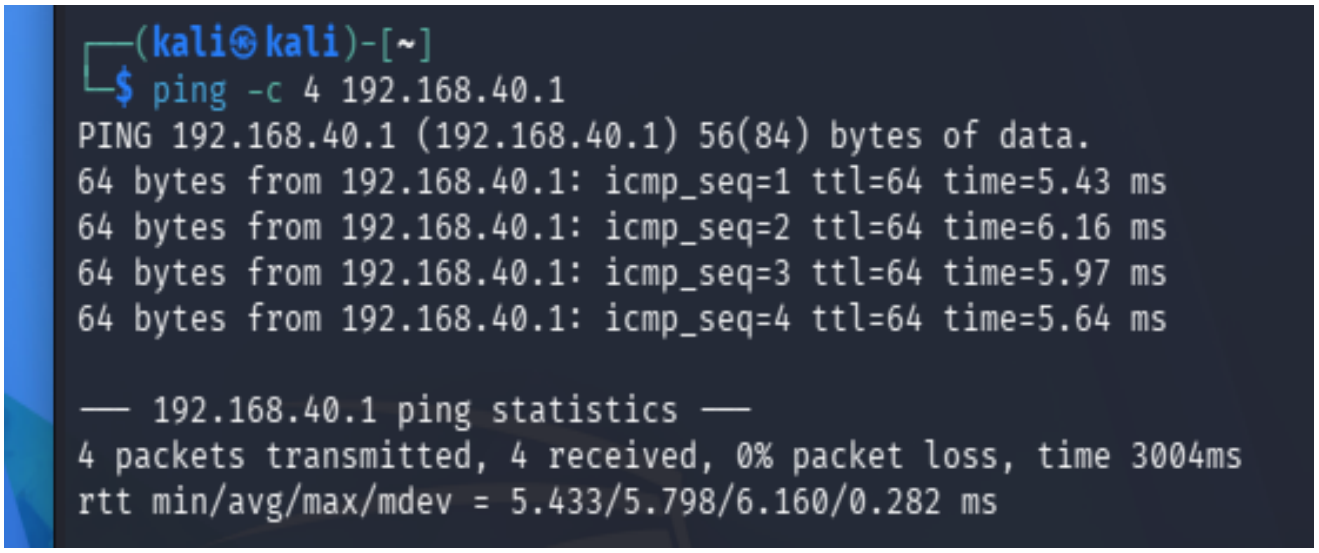
Complementando esta configuración, y con el fin de que Kali Linux apareciera como un dispositivo independiente dentro de la red, se configuró y aseguró que la interfaz utilizada fuera específicamente la interfaz WiFi de la laptop.



(Fuente: Propia del equipo)



(Fuente: Propia del equipo)



(Fuente: Propia del equipo)

A la fecha de redacción de este documento, el entorno de pruebas está completo y se ejecutaron pruebas manuales de detección y bloqueo (Feodo Tracker, firma NSIS_Inetc, check-in de troyano, bloqueo por dirección MAC y persistencia tras reinicio y corte de energía), descritas en el análisis de seguridad. La batería formal de pruebas con Kali por cada VLAN, la verificación de la detección en las VLAN 20, 30 y 40, la prueba de VLAN hopping y la prueba de rendimiento bajo carga simultánea permanecen pendientes. Corte de verificación: 5 de noviembre de 2026. Las pruebas manuales, todas lanzadas desde la VLAN 10 con una Kali (192.168.10.171), demuestran el bloqueo en línea de un beacon real del framework Covenant (firma ET MALWARE Covenant Framework HTTP Beacon, sid 2027793) y que el tráfico MQTT sin cifrar hacia un broker público no salió de la red; no sustituyen la batería formal de pentesting.

Capa de Backend

El backend del sistema Quillnet está compuesto por tres elementos que operan de manera coordinada. Los servicios Python del gateway (quillnet-agente, quillnet-vigia y quillnet-control) leen los registros de Suricata, vigilan qué dispositivos están realmente conectados (cada 15 segundos) y consultan cada 5 segundos los comandos pendientes para ejecutarlos con nftables. Un servidor Flask 3.1 con Gunicorn, desplegado en Render, expone 27 rutas. Una base de datos PostgreSQL (10 tablas y 13 migraciones Alembic) almacena la información, aislada por hogar mediante un identificador de gateway incluido en el JWT. La Raspberry Pi siempre inicia la conexión hacia el servidor, por estar detrás de NAT.

Proceso de construcción de la base de datos con traducción de alertas, documentado durante su desarrollo:

El primer paso consistió en crear una base de datos con un traductor de inteligencia artificial integrado. Se realizaron pruebas con archivos en formato JSON: si el identificador de una alerta no se encuentra registrado en la base de datos, el sistema envía el texto completo del archivo a la API de Groq, la cual lo traduce a lenguaje simple; ese texto traducido se le muestra al usuario y, además, se guarda el identificador junto con la traducción dentro de la base de datos. De esta manera, si la misma alerta vuelve a ocurrir, el sistema únicamente busca el identificador ya existente en la base de datos, en lugar de volver a consultar a la inteligencia artificial.

En una etapa posterior, la base de datos se migró de SQLite a PostgreSQL, se versionó su esquema mediante migraciones y se avanzó en la definición de los requerimientos funcionales del sistema completo.

A la fecha de redacción de este documento, el backend, la base de datos con el traductor de inteligencia artificial (modelo llama-3.3-70b-versatile, sin enviar direcciones IP), la cuarentena automática (3 bloqueos en 5 minutos), el envío de correos y los eventos en tiempo real se encuentran implementados y probados mediante 238 pruebas automatizadas del backend (pytest), ejecutadas contra una base PostgreSQL local descartable y nunca contra producción, con el correo y el DNS simulados. Las notificaciones push (Web Push) están implementadas en el backend y cubiertas por pruebas automáticas.

Clasificación de alertas por tipo de ataque. Cada alerta se clasifica en uno de 13 tipos: Malware, Control remoto, Conexión peligrosa, Escaneo de red, Fuerza bruta, Ataque web, Intento de intrusión, Robo de datos, Denegación de servicio, Criptominería, Navegación riesgosa, Dispositivo desconocido y Tráfico sospechoso. Primero se aplican reglas fijas y deterministas (el tipo del evento, palabras de fuerza bruta, el prefijo de la regla de Emerging Threats como ET MALWARE o ET SCAN, la categoría de Suricata y, por último, palabras clave); solo si esas reglas no deciden, la inteligencia artificial elige de una lista cerrada, de modo que no puede inventar categorías. Esta decisión surgió de dos versiones previas: una basada únicamente en palabras clave, donde casi todo caía en «Otros», y otra donde la IA creaba categorías libres, lo que dispersaba nombres equivalentes (por ejemplo, «Malware» y «Software malicioso»).

Mecanismos de seguridad implementados en esta capa. (1) La Raspberry Pi genera su propio secreto de autenticación (32 bytes aleatorios) y lo entrega una sola vez al registrarse; el backend guarda únicamente su hash SHA-256, por lo que no puede volver a revelarlo. (2) Las contraseñas se almacenan con hash scrypt (werkzeug.security, parámetros scrypt:32768:8:1, con sal aleatoria por contraseña) y solo pueden cambiarse mediante un enlace de un solo uso enviado por correo: el token es aleatorio, se guarda como SHA-256, vence en 30 minutos y la respuesta es idéntica exista o no la cuenta, para evitar la enumeración de usuarios. (3) Las sesiones utilizan JWT con vigencia de 7 días y el cambio de contraseña invalida las sesiones anteriores. Además, el inicio de sesión tiene verificación en dos pasos: con correo y contraseña correctos no se entrega la sesión, sino que

llega un código de 6 dígitos al correo y la sesión se entrega solo cuando la persona lo escribe. El código vence a los 10 minutos, admite 5 intentos, sirve una sola vez y se guarda únicamente como HMAC-SHA256 con clave del servidor; un código de desbloqueo no sirve para entrar y, con una contraseña incorrecta, no se envía ningún correo. Si el correo no puede enviarse, el servidor responde 502 y no se concede acceso; la variable LOGIN_2FA=0 en Render apaga el código como interruptor de emergencia. Los demás secretos usan SHA-256 (secreto de la Pi y token de restablecer contraseña). (4) Existe limitación de tasa en el inicio de sesión, el registro, la recuperación de contraseña y el desbloqueo. (5) Desbloquear un dispositivo exige verificación en dos pasos (contraseña y un código de 6 dígitos enviado por correo, protegido con HMAC-SHA256, con vigencia de 10 minutos y máximo 5 intentos), mientras que bloquear es una acción directa, porque bloquear reduce el riesgo y desbloquear lo reabre. (6) Toda consulta se filtra por el identificador del gateway (aislamiento multi-tenant) y un identificador ajeno responde 404 para no confirmar su existencia. (7) El backend envía cabeceras de seguridad (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy y HSTS) y limita CORS a una lista explícita de orígenes.

Capa de Frontend

La capa de frontend del sistema se desarrolla como una aplicación web construida con HTML, CSS y JavaScript, sin utilizar un framework adicional, diseñada para ser accesible desde cualquier navegador de internet moderno, sin requerir la instalación de software adicional por parte del usuario final.

En su versión final, la aplicación incluye un mapa isométrico de la red con una losa por VLAN y el erizo Quill al centro; una vista ampliada por VLAN donde los dispositivos aparecen como luciérnagas con su estado escrito (Nuevo o Alerta); un Panel de estadísticas; un Centro de alertas con explicaciones en lenguaje simple; la gestión de dispositivos (aprobar, bloquear y desbloquear con verificación en dos pasos) y los Ajustes. Funciona como aplicación web progresiva (PWA) con tema claro y oscuro, recibe alertas en tiempo real mediante Server-Sent Events y cumple criterios de accesibilidad como mostrar siempre el estado en texto (no solo en color), respetar la preferencia de reducir movimiento y mantener un contraste mínimo de 4,5:1.

Metodología de definición de pantallas. Antes de programar una sola línea de código, cada una de las doce pantallas que componen la aplicación se definió siguiendo un mismo formato de cinco elementos: propósito (qué problema resuelve), flujo del usuario (qué ve y hace, en orden), contrato con el backend (qué se pide y qué se recibe), qué puede salir mal (errores esperables), y el porqué (justificación de las decisiones técnicas). Este proceso permitió que el equipo supiera exactamente qué se estaba construyendo y por qué, antes de iniciar la programación.

A modo de ejemplo representativo de este proceso, se documentan a continuación tres de las doce pantallas definidas:

Inicio de sesión (Login). Su propósito es confirmar que quien está frente a la pantalla es un usuario autorizado del hogar. Ante credenciales incorrectas, se decidió mostrar un mensaje genérico (“Usuario o contraseña incorrectos”), sin especificar cuál de los dos falló, para no confirmarle a un posible atacante que el usuario sí existe. La validación de la contraseña vive completamente en el backend, nunca en el frontend, ya que el código del navegador es inspeccionable por cualquier persona.

Panel principal (Dashboard). Su propósito es responder, de un vistazo y sin tecnicismos, si todo está bien en la red del hogar en ese momento. En la versión implementada, el estado del hogar se calcula con cuatro niveles, siempre acompañados de texto y no solo de color: «protegida»; «en atención» (hay alertas nuevas o un dispositivo en cuarentena desde hace menos de 15 minutos); «en riesgo» (hay una alerta de severidad alta abierta) y «sin conexión» (el gateway lleva más de 3 minutos sin enviar señal). Como al sistema solo llegan alertas de severidad media y alta, el tráfico normal no genera alarma. El Panel muestra además los indicadores de las últimas 24 horas, las alertas por día de la última semana, un gráfico de tipos de ataque y una matriz de ataques por zona (VLAN) y tipo.

Verificación “¿Sos vos?” (2FA ante comportamiento sospechoso). A diferencia del inicio de sesión normal, esta verificación se activa cuando el sistema detecta comportamiento inusual en una cuenta ya conectada. Se estableció como regla sin excepción que la sesión se cierra localmente de inmediato en cuanto aparece este aviso, sin permitir que la persona conectada siga navegando mientras decide si fue ella o no. Si confirma que fue ella, debe ingresar un código adicional enviado por correo antes de abrir una nueva sesión; si indica que no fue ella, el sistema

bloquea el acceso de inmediato, sin pasos intermedios. En la versión implementada, este código de verificación por correo se exige para desbloquear un dispositivo.

Las nueve pantallas restantes (Registro/vinculación de cuentas, Mapa de red, Nuevo dispositivo detectado, Detalle de dispositivo, Centro de alertas, Estadísticas y Configuración, entre otras) siguieron el mismo proceso de definición documentada antes de su desarrollo visual.

La decisión de usar un dispositivo físico responde a un mejor manejo de los dispositivos reales del hogar: un dispositivo físico que trabaje de la mano del router resulta más apto que un laboratorio virtual no tangible para el usuario, y facilita la instalación gracias a sus dos interfaces claramente diferenciadas.

La arquitectura completa del proyecto está construida sobre las reglas de Zero Trust, un modelo de ciberseguridad que opera bajo la premisa de “nunca confiar, siempre verificar”. Quillnet no confía en ninguno de los dispositivos conectados a la red, por lo que los segmenta en VLANs y evita que su comunicación sea abierta. Este modelo, a pesar de parecer rígido, es una solución viable para la protección de las redes de los dispositivos IoT.

Requerimientos del Sistema

Como parte de la metodología de desarrollo del backend y del frontend, el equipo definió de manera previa un conjunto de requerimientos funcionales y no funcionales, con el propósito de guiar la construcción del sistema antes de iniciar su implementación técnica.

Requerimientos funcionales :

ID	Requerimiento	Estado
ReF01	Registro de usuario mediante correo electrónico, contraseña y número de teléfono	Cumplido con ajuste: el registro pide código de vinculación, correo y contraseña; no solicita número de teléfono; el inicio de sesión incorpora verificación en dos pasos por correo (pantalla «Revisá tu correo» con cuenta regresiva y reenvío)
ReF02	Inicio de sesión seguro (JWT) y verificación en dos pasos (2FA) para desbloquear dispositivos	Cumplido
ReF03	API de análisis de seguridad en Python — procesa registros de Suricata, detecta amenazas y envía resultados al backend	Cumplido
ReF04	API de respuesta en red — aislamiento de dispositivos por	Cumplido

ID	Requerimiento	Estado
	VLAN, bloqueo de dispositivos por dirección MAC (nftables), aplicación de reglas de seguridad	
ReF05	Generación de reportes automáticos con resultados del análisis, riesgos y recomendaciones	Parcial: el Panel ofrece un resumen del estado de la red y estadísticas de 24 h y 7 días; no existe reporte exportable
ReF06	Clasificación de amenazas en niveles medio y alto (la severidad baja se descarta como ruido) y en 13 tipos de ataque	Cumplido
ReF07	Notificaciones al usuario ante amenazas críticas o actividad sospechosa	Parcial: tiempo real (SSE) y correo listos; push implementado, entrega en celular pendiente de prueba
ReF08	Panel de control (dashboard) con estado de red, dispositivos y alertas	Cumplido
ReF09	Recomendaciones de seguridad (cambio de contraseñas, actualizaciones, etc.)	Cumplido
ReF10	Historial de eventos y análisis de seguridad	Cumplido (retención de 90 días)
ReF11	Cierre de sesión seguro, eliminando accesos activos	Cumplido: Ajustes > Cuenta incluye cerrar sesión

Requerimientos no funcionales :

ID	Requerimiento	Estado
ReNF01	Rendimiento — el análisis de red no debe afectar significativamente su funcionamiento	Pendiente (sin pruebas de carga)
ReNF02	Usabilidad — interfaz intuitiva y fácil de usar para cualquier tipo de usuario	Parcial: sin prueba formal con usuarios
ReNF03	Privacidad — los datos del usuario no se comparten sin autorización	Cumplido
ReNF04	Tiempo de respuesta cercano al tiempo real	Cumplido

Instrumentos y Técnicas de Recolección de Datos

Instrumentos Técnicos

La recolección de datos técnicos del proyecto se realiza principalmente a través de dos fuentes. La primera son los registros generados de manera automática por el sistema de detección de intrusos Suricata, los cuales documentan el tráfico de red analizado y las amenazas identificadas. La segunda fuente corresponde a los resultados obtenidos durante la ejecución de las

herramientas utilizadas en la fase de validación mediante pentesting, los cuales permiten determinar de manera objetiva si una amenaza simulada fue correctamente detectada y neutralizada por el sistema.

Instrumentos Documentales

Como instrumento complementario de recolección y trazabilidad de información, el proyecto mantiene una bitácora de desarrollo centralizada en la plataforma Notion. Dentro de esta bitácora, cada integrante del equipo documenta de manera individual e independiente sus tareas asignadas, su registro cronológico de actividades realizadas, los problemas técnicos encontrados durante su trabajo y las posibles soluciones identificadas, así como sus próximos pasos planificados. Este instrumento documental resulta de particular relevancia metodológica, ya que permite reconstruir con precisión, en cualquier momento del proyecto, la secuencia completa de decisiones tomadas, los obstáculos enfrentados por cada área de trabajo y las soluciones aplicadas a lo largo de todo el proceso de desarrollo, constituyendo así la principal fuente de evidencia del proceso de prueba y error documentado a lo largo de este capítulo.

Validación con Actores Externos

De manera adicional a los instrumentos técnicos y documentales descritos anteriormente, el proyecto cuenta con un mecanismo de validación externa, proporcionado por los asesores académicos designados para el proyecto, quienes acompañan el avance del equipo y retroalimentan sus decisiones técnicas y documentales en los hitos formales del proyecto. Este mecanismo se complementa con la verificación propia del equipo mediante pruebas controladas, particularmente valiosa cuando surgen bloqueos técnicos, como ocurrió con el conflicto relacionado con la instalación del sistema de detección de intrusos sobre la versión inicial de OpenWRT utilizada en el proyecto. Dicho bloqueo fue resuelto por el equipo al reemplazar OpenWRT por Raspberry Pi OS Lite.

Procedimiento General

Con el fin de mantener coherencia metodológica entre las distintas áreas de trabajo del proyecto como la parte técnica, de diseño y administrativa, el equipo aplicó de manera transversal el siguiente procedimiento general en el desarrollo de cada componente del sistema:

En primer lugar, se procede a la definición precisa del requerimiento o problema específico que se busca resolver dentro de ese componente particular del sistema. En segundo lugar, se realiza el diseño conceptual de la solución propuesta, incluyendo la descripción del flujo esperado de funcionamiento y la identificación anticipada de los posibles puntos de falla. En tercer lugar, se procede a la implementación técnica del componente, siguiendo el diseño conceptual previamente acordado. En cuarto lugar, se somete el componente implementado a pruebas de funcionamiento bajo condiciones controladas. En quinto lugar, se documentan de manera detallada los resultados obtenidos durante dichas pruebas, incluyendo cualquier problema o error identificado. En sexto lugar, se realizan los ajustes o correcciones necesarias con base en los resultados documentados. Finalmente, este ciclo completo se repite para el desarrollo del siguiente componente o para la siguiente iteración de mejora del mismo componente.

Este procedimiento no se aplicó de manera exclusiva al desarrollo técnico del sistema, sino que fue adoptado también en el desarrollo de la identidad visual del proyecto y en la planificación administrativa y financiera del mismo, lo cual explica la coherencia metodológica presente en las distintas áreas de trabajo descritas en los apartados siguientes de este capítulo.

Cronograma Real de Ejecución

Con el propósito de reflejar de manera honesta el proceso metodológico real del proyecto, y no únicamente una planificación idealizada, se presenta a continuación el cronograma de ejecución del proyecto, contrastando los hitos ya cumplidos con aquellos que aún se encuentran pendientes de realización a la fecha de redacción de este documento.

Durante mayo de 2026 se aprobó la primera defensa académica, cerrando la segunda fase. En julio de 2026 se identificó el bloqueo de instalación de Suricata sobre OpenWRT, resuelto al migrar a Raspberry Pi OS Lite. Entre agosto y septiembre se construyeron el backend, el frontend y el IPS, y en agosto se realizó la segunda defensa.

De cara a lo que resta, la tercera defensa se realizó entre el 12 y el 16 de octubre de 2026 y el proyecto será presentado ante el público general y el panel evaluador en el marco de Expotec, del 13 al 15 de noviembre de 2026. A continuación se presenta el resumen del cronograma real de ejecución del proyecto:

Fase	Periodo	Estado
Fase 1 — Investigación y definición	Febrero-marzo 2026	Completada
Fase 2 — Documentación y primera defensa	Abril-mayo 2026	Completada
Fase 3 — Construcción del sistema	Junio-julio 2026	Completada
Reunión de seguimiento	Julio 2026	Realizada; bloqueo resuelto con Raspberry Pi OS
Fase 4 — Backend y frontend	Agosto-septiembre 2026	Completada
Segunda Defensa	10-13 de agosto de 2026	Realizada
Fase 5 — Pulido y Expotec	Octubre-noviembre 2026	En progreso
Tercera Defensa	12-16 de octubre de 2026	Realizada
Presentación final en Expotec	13 al 15 de noviembre de 2026	Pendiente

Alcances y Limitaciones

Resulta importante delimitar con claridad, dentro del marco metodológico de este proyecto, cuál es el alcance real del prototipo desarrollado y cuáles aspectos quedan explícitamente fuera de dicho alcance.

El prototipo desarrollado para su presentación en Expotec 2026 constituye una prueba de concepto funcional, y no representa un producto comercial completamente terminado y listo para su distribución masiva. Dentro de los elementos que quedan fuera del alcance del prototipo actual se encuentra la implementación de un sistema de gestión de información y eventos de seguridad, conocido en la industria como SIEM. Si bien este tipo de sistema fue considerado en etapas iniciales del proyecto, el equipo determinó reposicionarlo como una línea de escalabilidad futura del proyecto, y no como un componente que forme parte del prototipo actualmente en desarrollo, debido a la complejidad adicional que su implementación representaría dentro del plazo disponible.

De manera similar, es necesario aclarar que, si bien el equipo ha utilizado herramientas de acceso remoto como recurso interno de colaboración durante la construcción del prototipo, dichas herramientas no forman parte de la arquitectura del producto final. Quillnet opera con el gateway dentro de la red local del hogar y un backend en la nube (Render); el gateway siempre inicia la

conexión hacia el backend y la traducción de alertas depende de la API de Groq, por lo que requiere conexión a Internet mientras esa dependencia externa se mantenga.

Adicionalmente, el prototipo presenta limitaciones propias de su etapa de desarrollo. Su funcionamiento depende de herramientas y servicios externos (Suricata, Flask, PostgreSQL, Render, Groq y Resend), por lo que cualquier cambio en su disponibilidad puede afectarlo. En particular, el inicio de sesión con verificación en dos pasos depende de que Resend entregue el código: con el remitente de prueba (onboarding@resend.dev) solo se entrega al correo dueño de la cuenta de Resend, por lo que antes de la presentación debe confirmarse la entrega o desactivarse el código con LOGIN_2FA=0. El prototipo está orientado inicialmente a una única instalación, utiliza un solo gateway sin redundancia y su rendimiento depende de la capacidad de la Raspberry Pi. La base de datos PostgreSQL del plan gratuito de Render venció el 27 de octubre de 2026; el equipo la renovó con un plan de pago de la base de datos de 6$ al mes.Además, el gateway aún no cuenta con una cadena input en nftables que proteja sus propios servicios.

Otras limitaciones conocidas son: el limitador de tasa guarda sus contadores en la memoria de un único proceso, por lo que no es compartido; la conexión de tiempo real (SSE) envía el token en la URL, que podría quedar en registros de acceso; las sesiones solo se invalidan por cambio de contraseña o por vencimiento, sin revocación explícita; y, mientras no se verifique un dominio propio en el servicio de correo, los mensajes solo se entregan al titular de la cuenta de ese servicio. Tampoco se han realizado pruebas de carga.

Limitaciones de inspección identificadas en las pruebas: el tráfico entre equipos de una misma VLAN se conmuta sin pasar por el gateway, por lo que Suricata no lo inspecciona (un sqlmap lanzado desde la Kali hacia otro equipo de la VLAN 10 no generó alerta); el IPS tampoco puede aplicar firmas de contenido al tráfico cifrado con TLS; y las firmas informativas de severidad baja se descartan por diseño para evitar la fatiga de alertas. Estas limitaciones justifican la segmentación por VLAN y dejan como mejoras el aislamiento de clientes en el access point, reglas basadas en metadatos TLS (JA3/SNI) y el filtrado DNS. Además, el enlace entre la Raspberry Pi y el access point es de 100 Mbps, por limitación del puerto del EAP110.

Finalmente, la principal limitación metodológica identificada a la fecha es que la validación experimental formal mediante pentesting (batería con Kali por cada VLAN, detección en las VLAN

20, 30 y 40, prueba de VLAN hopping y rendimiento bajo carga simultánea) sigue pendiente. Las pruebas manuales realizadas demuestran el bloqueo de amenazas reales, pero no sustituyen esa validación, cuyo avance se documentará en las actualizaciones posteriores de este trabajo escrito.

Diseño y Comunicación Visual

Proceso de Construcción de Identidad de Marca

La identidad visual del proyecto Quillnet no se concibió como un elemento estético añadido posteriormente al desarrollo técnico, sino como un componente definido desde la fase inicial de investigación del proyecto, siguiendo un proceso metodológico propio. Este proceso incluyó la definición del nombre del proyecto, la creación de una mascota conceptual, un erizo llamado Quill, cuyo mecanismo natural de defensa consiste en erizar sus púas únicamente ante una amenaza real, sin necesidad de atacar de manera activa y la definición de un eslogan que sintetiza esta filosofía: “Defensa que no necesita atacar”. Esta metáfora no es meramente decorativa, sino que refleja de manera directa la filosofía de seguridad Zero Trust aplicada por el sistema: una postura defensiva, pasiva por defecto, que únicamente reacciona ante una amenaza identificada.

Criterios de Selección de Paleta de Colores

La definición de la paleta de colores del proyecto siguió, al igual que los componentes técnicos descritos anteriormente en este capítulo, un proceso metodológico documentado, y no una selección arbitraria. Este proceso inició con una etapa de investigación sobre psicología del color, orientada a comprender qué emociones y percepciones se asocian comúnmente a distintas combinaciones cromáticas. A partir de dicha investigación, se identificaron inicialmente dos combinaciones de paleta candidatas, cada una asociada a un conjunto distinto de valores y percepciones que el proyecto podría transmitir a sus usuarios y evaluadores.

Como parte de este proceso, el equipo definió explícitamente cuáles eran los valores y percepciones que el producto debía transmitir a quien lo observa por primera vez, entre ellos: una sensación de confianza, tranquilidad, elegancia y sofisticación, y una imagen de innovación tecnológica que resultara atractiva y memorable. Con base en estos criterios, se seleccionó como paleta definitiva del proyecto la siguiente combinación de colores:

- Negro (#000000), utilizado para representar el entorno digital y las amenazas invisibles que existen dentro de una red doméstica, transmitiendo además seriedad y profesionalismo.
- Azul marino (#000080), utilizado para representar la protección y el estado seguro de los dispositivos monitoreados, transmitiendo estabilidad y tranquilidad al usuario.
- Morado eléctrico (#BF00FF, usado como morado decorativo, mientras que los botones y elementos interactivos usan #9D00FF), utilizado para representar la innovación tecnológica y la inteligencia del sistema, en referencia directa al hecho de que el sistema no solo protege pasivamente, sino que analiza el tráfico de red, detecta comportamientos sospechosos y responde ante posibles amenazas. Este color conecta conceptualmente, además, con la metodología Purple Team aplicada en el proyecto, la cual combina precisamente una perspectiva ofensiva y una defensiva para fortalecer la seguridad del sistema.
- Blanco (#FFFFFF), utilizado para representar la claridad y transparencia de la información entregada al usuario, reflejando el principio de que el usuario debe poder comprender con claridad, mediante gráficas y alertas en lenguaje simple, qué está ocurriendo en su propia red doméstica.

En cuanto a la tipografía definitiva del proyecto, se seleccionó la familia tipográfica Noto Sans Telugu, aplicada de manera jerárquica según el tipo de contenido a mostrar dentro de la interfaz: se utiliza el peso Semibold (600) para los títulos, el peso Regular (400) para los subtítulos, y el peso Thin (100) para los textos generales de la aplicación, con el fin de establecer una jerarquía visual clara entre los distintos niveles de información presentados al usuario. En la implementación del frontend, los títulos, las cifras grandes y el logo en texto usan Rowdies, y el cuerpo usa Noto Sans Telugu (Google Fonts, con respaldo system-ui). Un peso Thin (100) en texto corrido no cumple el contraste mínimo de 4,5:1 declarado en la capa de frontend, por lo que para el texto corrido conviene usar Regular (400).

Definición de Pantallas del Frontend

Tal como se describió previamente en el apartado correspondiente a la capa de frontend dentro de este capítulo, la definición de cada pantalla de la aplicación siguió un proceso documentado de manera sistemática y aplicado de forma consistente a lo largo de todo el proyecto. Para cada pantalla, se definió y documentó de manera previa a su desarrollo técnico su propósito específico, su flujo de interacción esperado, su contrato de intercambio de datos con el backend, y los posibles puntos de falla que podrían presentarse. Este mismo proceso metodológico, originalmente aplicado al desarrollo técnico de la interfaz, fue liderado en estrecha colaboración con el área de diseño y comunicación del proyecto, con el fin de garantizar que las decisiones funcionales y las decisiones estéticas de cada pantalla respondieron a una misma lógica coherente.

Proceso Iterativo de Mockups

El desarrollo visual de las distintas pantallas del sistema se realizó siguiendo un proceso iterativo, en el cual el equipo partió de definiciones puramente conceptuales de cada pantalla descritas en términos de propósito y flujo funcional y avanzó de manera progresiva hacia representaciones visuales cada vez más elaboradas y refinadas. Este enfoque iterativo permitió que los ajustes de estilo visual pudieran realizarse de manera independiente, sin necesidad de modificar la lógica funcional de cada pantalla, la cual ya había sido acordada y documentada previamente por el equipo.

Gestión Administrativa y Financiera

Distribución de Responsabilidades y Roles

El equipo de trabajo de Quillnet distribuyó sus responsabilidades en seis roles complementarios entre sí, cada uno documentado de manera individual dentro de la bitácora del proyecto: un rol de liderazgo y coordinación general del proyecto, encargado de la toma de decisiones estratégicas y de la supervisión de la arquitectura completa del sistema; un rol de infraestructura de red, encargado del diseño, instalación y configuración de la red física y de los componentes de segmentación y detección; un rol de pentesting, encargado de las pruebas de

penetración y de la documentación de las evidencias obtenidas durante dichas pruebas; un rol de desarrollo web, encargado de la construcción del backend, la base de datos y el frontend del sistema; un rol de diseño, mercadeo y comunicación, encargado de la identidad visual del proyecto, la interfaz gráfica de la aplicación y la comunicación con actores externos; y un rol de administración financiera y de recursos, encargado de la planificación general del proyecto, la valoración de sus costos y su viabilidad como servicio.

Esta distribución de roles permite que cada integrante documente de manera independiente su propio proceso metodológico de trabajo, mientras que el rol de liderazgo mantiene la coherencia general del proyecto y asegura que las distintas áreas de trabajo avancen de manera coordinada entre sí.

Proceso de Toma de Decisiones de Compra

Las decisiones relacionadas con la adquisición de hardware para el proyecto siguieron, al igual que las decisiones técnicas y de diseño descritas anteriormente en este capítulo, un proceso de evaluación de alternativas previo a la selección final, en lugar de una selección arbitraria del primer producto disponible.

Un ejemplo concreto de este proceso, documentado dentro de la bitácora del proyecto, corresponde a la selección del access point utilizado para distribuir la red inalámbrica del prototipo. Inicialmente, el equipo había contemplado la adquisición del modelo TP-Link EAP615, sin embargo, dicho modelo no se encontró disponible dentro del mercado local en el momento en que se requería su adquisición. Ante esta limitación, el equipo evaluó alternativas disponibles y seleccionó finalmente el modelo TP-Link EAP110, decisión que se tomó considerando tanto la disponibilidad inmediata del producto como su ajuste al presupuesto general definido para el proyecto.

Manejo de Presupuesto

Como parte de la gestión administrativa y financiera del proyecto, se elaboró un plan de negocio que incluyó, entre otros elementos, un estudio de mercado de las soluciones de ciberseguridad ya existentes y una proyección detallada de los costos asociados al prototipo desarrollado.

La inversión total en hardware requerida para el desarrollo del prototipo asciende a ciento veintisiete mil novecientos colones costarricenses, monto que se distribuye entre la adquisición de la Raspberry Pi 4 Modelo B, con un costo de ciento diez mil quinientos colones, y la adquisición del access point TP-Link EAP110, con un costo de diecisiete mil cuatrocientos colones.

Como parte de la proyección de escalabilidad del proyecto más allá de su etapa de prototipo académico, se planteó un modelo de negocio proyectado en el cual un proveedor de servicios de internet (ISP) ofrecería Quillnet como servicio mediante una suscripción mensual de tres mil quinientos colones (₡3,500) por hogar. Este escenario es hipotético y no implica un acuerdo con ninguna empresa. El mercado meta estimado es de 430,000 personas interesadas en proteger su red digital.

Este precio se contrastó con la disposición a pagar observada en la encuesta (entre 5 y 10 dólares adicionales al mes, Capítulo IV), que con un tipo de cambio de referencia de ₡453.43 por dólar (fecha: 05/11/2026) equivale a ₡2,267.15 – ₡4,534.30 mensuales. El precio proyectado de ₡3,500 se encuentra dentro de ese rango, por lo que el modelo no requiere un subsidio del ISP para ser aceptable para los hogares encuestados. Esta conclusión se limita a la muestra de 112 hogares, que no es probabilística.

Curva de adopción realista (primeros 18 meses)

Para un producto nuevo, se proyecta una tasa de conversión del 2.5 % del mercado meta (430,000 personas) al cabo de año y medio, lo que equivale a 10,750 clientes conectados. La curva parte de cero clientes y suma usuarios mes a mes gracias a campañas de telemercadeo, la aplicación del ISP y publicidad.

• Meses 1 a 3 (lanzamiento y educación): adopción lenta, con unos 300 clientes nuevos por mes. Al cierre del trimestre hay 900 clientes que generan ₡3,150,000 al mes.

• Meses 4 a 6 (tracción comercial): el ritmo sube a 500 clientes nuevos por mes. Al mes 6 hay 2,400 clientes activos que generan ₡8,400,000 mensuales.

• Meses 7 a 12 (maduración del producto): crecimiento constante de 700 clientes nuevos por mes. Al mes 12 hay 6,600 clientes activos que facturan ₡23,100,000 al mes.

• Meses 13 a 18 (consolidación): se mantienen 700 altas mensuales hasta alcanzar el tope de 10,750 clientes en el mes 18, con una facturación de ₡37,625,000 mensuales.

Mes	Clientes activos	Ingresos del mes	Ingresos acumulados
3	900	₡3,150,000	₡6,300,000
6	2,400	₡8,400,000	₡26,250,000
12	6,600	₡23,100,000	₡128,100,000
14	8,000	₡28,000,000	₡181,650,000
18	10,750	₡37,625,000	₡317,975,000

La inversión inicial proyectada es de ₡172,000,000. Con una suscripción de ₡3,500 mensuales, los ingresos acumulados superan esa inversión en el mes 14 (₡181,650,000 acumulados; al mes 13 eran ₡153,650,000). Este plazo se calcula sobre ingresos y no sobre ganancia: no descuenta los costos de operación en la nube, el soporte técnico ni el mantenimiento.

Gestión de Relación con Actores Externos

La gestión de la relación del proyecto con sus actores externos, particularmente con los asesores académicos designados para el proyecto, se documenta de manera centralizada dentro de la bitácora del proyecto, incluyendo las fechas de las reuniones sostenidas, los acuerdos alcanzados durante dichas reuniones y los próximos pasos definidos de manera conjunta entre el equipo y dichos actores externos.

Instrumentos de recopilación de datos

En el proyecto se creó un formulario en el cual los encuestados rellenaban información sobre sus dispositivos e información básica de sus hogares. Las preguntas del cuestionario pueden dividirse en tres categorías:

Información general del hogar: ¿En qué provincia se encuentra su hogar viviendo actualmente?, ¿Cuántas personas viven en su hogar?, ¿En qué rango de edad se encuentra usted actualmente?, ¿Cuántos son los ingresos mensuales de su hogar? La información general sirve para tener datos del tipo de población al que el proyecto quillnet está hecha, obteniendo información fundamental de la muestra que tomamos.

Información sobre dispositivos: Cantidad de Celulares, asistentes inteligentes (como parlantes), computadoras, tabletas y televisores inteligentes. Este conocimiento es relevante para poder comprender la cantidad de dispositivos promedio de un hogar, no solo para tener datos reales del uso de dispositivos IoT en los hogares sino también para mejorar quillnet a una amplia gama de dispositivos.

Información sobre el conocimiento del usuario: ¿Están usted y su hogar conscientes de que todos esos dispositivos pueden o están siendo vulnerados por algún atacante? ¿Sabían usted y su hogar que en Costa Rica hay alrededor de 255 millones de ataques a dispositivos como los que tiene en su casa? Si existiera un servicio que pudiera proteger sus redes de estos ataques y darle información relevante del estado de sus conexiones integrado a su internet por un pequeño valor agregado, ¿lo usaría? Si su respuesta anterior fue afirmativa. ¿Cuánto consideran que sería un buen precio para ese valor agregado? Estas preguntas están hechas con el propósito de entender el estado de conocimiento de los encuestados en materia de ciberseguridad aplicada a sus dispositivos en el hogar.

Nota metodológica: la cifra de 255 millones de ataques en Costa Rica mencionada en una de las preguntas del cuestionario no pudo verificarse en una fuente pública (la búsqueda del equipo

no la localizó), por lo que no se utiliza como dato del marco teórico ni de las conclusiones de este trabajo.

1. Dispositivos

N.º	Dispositivo	Especificación / función	Cantidad
1	Raspberry Pi 4 Modelo B	4 GB de RAM, ARM de 4 núcleos. Gateway central: enrutamiento, VLAN, firewall e IPS.	1
2	Punto de acceso TP-Link EAP110	Distribuye la red inalámbrica con un SSID por VLAN.	1
3	Tarjeta microSD	Almacena Raspberry Pi OS Lite y la configuración del gateway.	1
4	Fuente USB-C y cables Ethernet	Alimentación y enlace con el access point.	1
5	Teléfono móvil (hotspot)	Proporciona la WAN; sustituye al proveedor de internet en la demostración.	1
6	Dispositivos IoT de prueba	Cámara IP, asistente de voz y Smart TV, uno por VLAN.	3
7	Computadora portátil con Fedora Linux y Kali Linux (máquina virtual)	Estación de pruebas ofensivas (Red Team): Kali Linux 2026.2 en KVM/QEMU (VMware), con Nmap, sqlmap, curl y tmNIDS.	1

Componente	Versión / detalle	Uso en Quillnet
Raspberry Pi OS Lite 64-bit	Debian 13 “trixie”	Sistema operativo del gateway

Componente	Versión / detalle	Uso en Quillnet
NetworkManager (nmcli), nftables, dnsmasq	Política forward drop; un ámbito DHCP/DNS por VLAN	VLAN 802.1Q, firewall Zero Trust y servicios de red
Suricata	7.0.10, IPS inline (NFQUEUE) con bypass; 52,246 reglas cargadas, 12,860 en modo drop, 5 propias de demostración	Detección y bloqueo de tráfico malicioso
Servicios Python del gateway	quillnet-agente v4 (alertas de Suricata), quillnet-vigia v4 (presencia cada 15 s) y quillnet-control (comandos cada 5 s, bloqueo por MAC)	Reporte de eventos, vigilancia de dispositivos y ejecución de comandos
Flask 3.1 + Gunicorn en Render	27 rutas	Backend (API REST)
PostgreSQL + Alembic	10 tablas, 13 migraciones	Base de datos multi-tenant
Groq (llama-3.3-70b-versatile, respaldo llama-3.1-8b-instant), Resend y Web Push (VAPID)	—	Traducción de alertas, envío de correos y notificaciones push
HTML, CSS y JavaScript	Sin frameworks	Frontend: mapa de red, alertas, tema claro/oscuro
Kali Linux, Nmap, sqlmap, curl, tmNIDS, mosquitto-clients, jq, Notion y Git/GitHub	—	Pruebas ofensivas, análisis de eve.json, bitácora y control de versiones

Software y servicios

2.Conceptos

Concepto	Costo (₡)	Observación
Raspberry Pi 4 Modelo B (4 GB)	110,500	Gateway central
Punto de acceso TP-Link EAP110	17,400	Sustituyó al EAP615, no disponible en el mercado local
Total de hardware principal	127,900	Inversión base del prototipo
microSD, fuente y cables	0	Accesorios
Software (Suricata, nftables, Flask, PostgreSQL, Debian)	0	Software libre y de código abierto
Servicios en la nube (Render, Groq, Resend)	6$ al mes de la BD.	Render: servicio web ; base PostgreSQL de

Concepto	Costo (₡)	Observación
		Render renovada con plan de pago; Groq y Resend con planes gratuitos

Descripción de la solución o prototipo

Quillnet es un gateway de seguridad físico que se coloca entre Internet y la red del hogar. Todo el tráfico de los dispositivos debe atravesarlo, lo que permite segmentarlo, inspeccionar y, si es malicioso, bloquearlo antes de que llegue a su destino. La solución se compone de tres partes que cooperan: el gateway (Raspberry Pi 4 con VLAN, nftables y Suricata IPS), el backend en la nube (Flask y PostgreSQL en Render) y el frontend web de autogestión.

Cuando Suricata detecta o bloquea tráfico malicioso, el servicio quillnet-agente (que solo reenvía alertas de severidad media y alta) envía el evento al backend, que lo guarda, lo clasifica en uno de 13 tipos de ataque, traduce la alerta a lenguaje sencillo con inteligencia artificial (sin enviar direcciones IP, MAC ni nombres) y la muestra en tiempo real en el panel; además, notifica por Web Push y, cuando la alerta es de severidad alta y no hay una suscripción push activa, por correo. Es importante distinguir los casos: un beacon de comando y control como el de Covenant es bloqueado en línea; las reglas de demostración de detección (como la de sqlmap) generan alerta sin bloquear y el tráfico entre equipos de una misma VLAN no pasa por el gateway, por lo que Suricata no lo inspecciona. Si un mismo dispositivo acumula 3 bloqueos en 5 minutos, el sistema lo pone en cuarentena automáticamente; para desbloquearlo, el propietario debe confirmar con verificación en dos pasos. La Raspberry Pi consulta cada 5 segundos la cola de comandos del backend y siempre inicia la conexión, por lo que no es necesario abrir puertos en el módem del hogar.

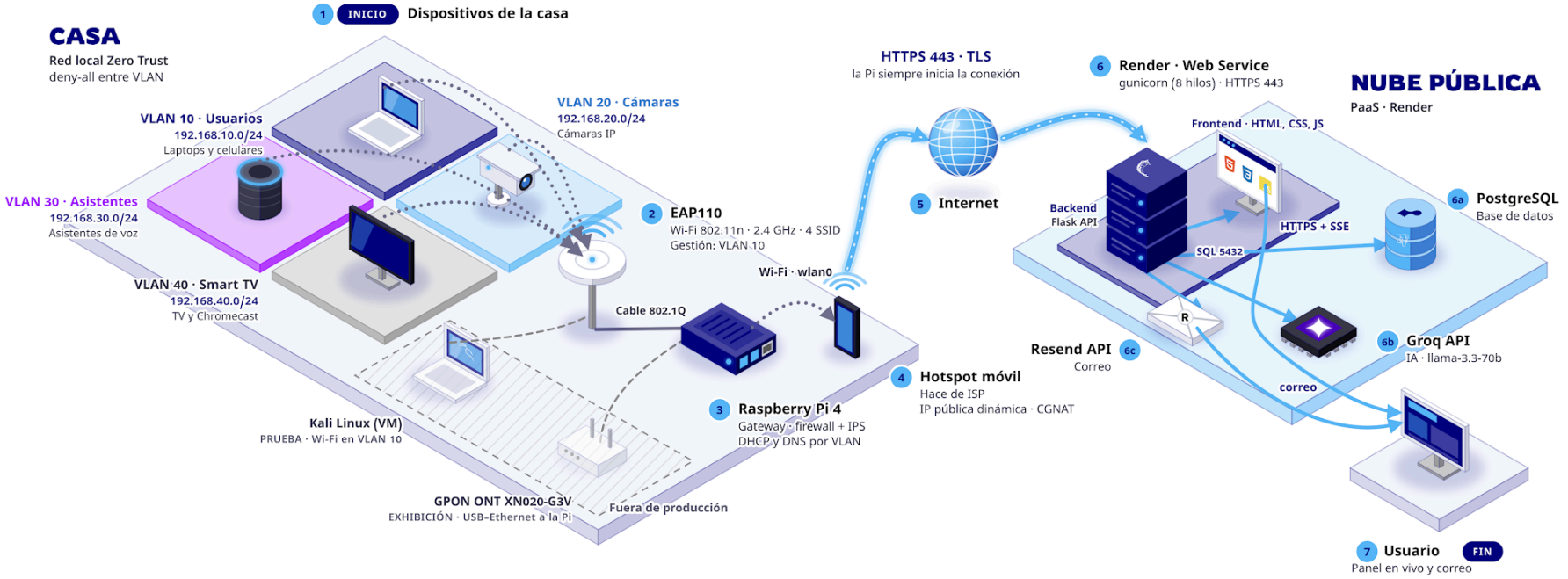
La filosofía del proyecto está representada por su mascota, el erizo Quill, que solo eriza sus púas ante una amenaza real: “Defensa que no necesita atacar”. El detalle de cada capa (física, segmentación y detección, validación, backend y frontend) se describe en la sección Arquitectura por Capas.

Diagrama de Infraestructura de Conexión y Procesos

Infraestructura de red · Quillnet

Gateway IoT Zero Trust en Raspberry Pi 4 · del dispositivo de la casa (inicio) al usuario en la nube (fin)

1 Dispositivos › 2 EAP110 › 3 Pi › 4 Hotspot › 5 Internet › 6 Render › 7 Usuario



Direccionamiento IP

SEGMENTO	RED PRIVADA	GATEWAY
VLAN 10 Usuarios	192.168.10.0/24	.10.1
VLAN 20 Cámaras	192.168.20.0/24	.20.1
VLAN 30 Asistentes	192.168.30.0/24	.30.1
VLAN 40 Smart TV	192.168.40.0/24	.40.1

RFC 1918 · DHCP .100–.200 (dnsmasq)

PUNTO	IP PÚBLICA / WAN
wlan0 · Pi	Dinámica, la asigna el hotspot
Hotspot · ISP	Pública dinámica · CGNAT
Render	quillnet.onrender.com

Conexiones

TRAMO	TIPO	PROTOCOLO · VELOCIDAD
1 → 2	Inalámbrica	Wi-Fi 802.11n · ≤ 300 Mbps
2 → 3	Física	Ethernet + 802.1Q · 10/100/1000
3 → 4	Inalámbrica	Wi-Fi 802.11 (wlan0) · variable
4 → 6	Internet	HTTPS / TLS 1.2+ · 443
6 → 6a	Interna	PostgreSQL · TCP 5432
6 → 6b·6c	Internet	HTTPS · API de IA y correo
6 → 7	Internet	HTTPS + SSE · 443
GPON	Fibra	ITU-T G.984 · 2.5/1.25 Gbps

Procesos

3 Raspberry Pi

nftables · firewall Zero Trust
Suricata · IPS · dnsmasq · DHCP + DNS
agente · vigía · control (Python)

6 Render · Flask 3.1 + gunicorn (8 hilos)

/api/alerts · /api/devices/presencia
/api/commands/pending · /ack
/api/gateways/register
6a usuarios, dispositivos, alertas, comandos
6b explica alertas sin datos personales
6c recuperación de contraseña y 2FA

Servicios y puertos

SERVICIO	TRANSPORTE	PUERTO
HTTPS panel + API	TCP · TLS	443
DNS (dnsmasq)	UDP / TCP	53
DHCP (dnsmasq)	UDP	67 / 68
SSH administración	TCP	22
PostgreSQL	TCP	5432
MQTT (previsto)	TCP · TLS opc.	1883 / 8883

Sentido: la Pi siempre inicia (saliente).
Desde Internet no entra nada a la casa.

Leyenda

- Cable (físico)
- Inalámbrico (Wi-Fi)
- HTTPS casa → nube
- Llamadas dentro de la nube
- Fuera de producción

GPON ONT XN020-G3V · exhibición
Sin fibra real · USB-Ethernet a la Pi

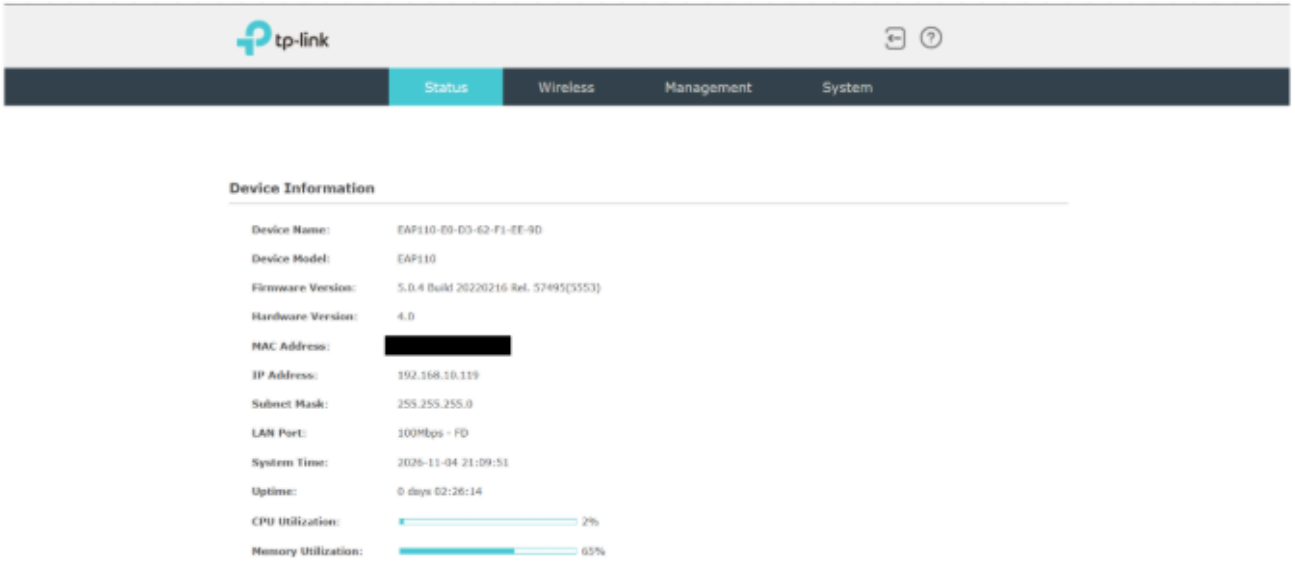
Kali Linux (VM) · prueba
KVM/QEMU bridged · Wi-Fi en VLAN 10

Red privada

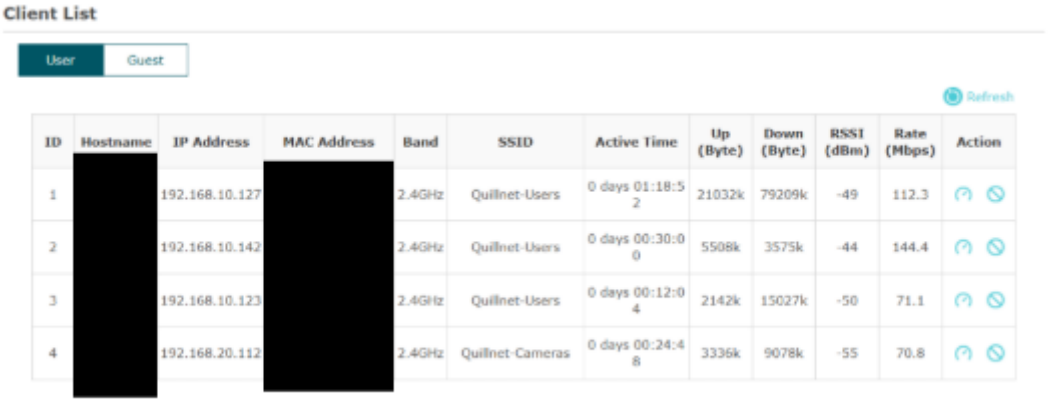
La red privada es la LAN segmentada del hogar, formada por cuatro VLAN con direcciones de los rangos privados del RFC 1918 (Rekhter et al., 1996), no enrutables en Internet:

VLAN	Segmento	Subred	Dispositivos
10	Usuarios	192.168.10.0/24	Celulares y computadoras de las personas del hogar
20	Cámaras IP	192.168.20.0/24	Cámaras de videovigilancia
30	Asistentes de voz	192.168.30.0/24	Parlantes y asistentes inteligentes
40	Smart TV	192.168.40.0/24	Televisores y decodificadores

Direccionamiento verificado en el gateway: la puerta de enlace de cada VLAN es la dirección .1 de su subred (por ejemplo, 192.168.10.1 para Usuarios) y dnsmasq entrega direcciones del rango .100 al .200 con un tiempo de concesión de 24 horas en cada VLAN. Las cuatro VLAN viajan etiquetadas (802.1Q) sobre la interfaz eth0 hacia el access point TP-Link EAP110, que se conecta a la VLAN 10 (gestión) y se excluye de los reportes de alertas y presencia para no generar ruido de infraestructura.



(Fuente: Propia del equipo)



(Fuente: Propia del equipo)

La política del firewall es Zero Trust (forward drop): todas las VLAN pueden salir a Internet bajo inspección de Suricata; no existen reglas inter-VLAN, por lo que ninguna VLAN puede iniciar comunicación hacia otra: el tráfico entre VLAN lo corta el firewall sin generar alerta de Suricata (informe de pruebas del 5 de noviembre de 2026).

```
quillnet@quillnet-gateway:~ $ sudo nft list ruleset
table inet filter {
        chain forward {
                type filter hook forward priority filter; policy drop;
                ether saddr 62:87:69:f2:28:40 drop
                ether saddr 5e:6c:f5:65:93:ff drop
                ether saddr c6:09:e8:8b:4d:bf drop
                ether saddr 5a:46:4b:13:5a:8a drop
                ether saddr 22:56:e2:e5:fa:7b drop
                ether saddr 0e:7b:62:03:ed:34 drop
                iifname { "eth0.10", "eth0.20", "eth0.30", "eth0.40" } oifname "wlan0" accept
                ct state established,related accept
        }

        chain IPS {
                type filter hook forward priority filter + 10; policy accept;
                queue flags bypass to 0
        }

        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;
                oifname "wlan0" masquerade
        }
}
quillnet@quillnet-gateway:~ $
```

(Fuente: Propia del equipo)

Red pública

Red de acceso (WAN): el gateway obtiene Internet por su interfaz inalámbrica (wlan0), conectada al hotspot de un teléfono que sustituye al proveedor de internet en la demostración. Al estar detrás de NAT, el gateway no recibe conexiones entrantes: él inicia todas las conexiones hacia el backend.

Backend en Render: servicio con dirección pública, protegido por HTTPS con HSTS, cabeceras de seguridad (CSP, X-Frame-Options), CORS con lista explícita de orígenes, autenticación JWT para las personas, token propio para la Raspberry Pi y limitación de tasa; es el único componente de Quillnet visible desde Internet. Los servicios externos Groq (traducción de alertas) y Resend (correo) son consumidos únicamente por el backend. El frontend se accede desde cualquier navegador mediante la URL pública.

Análisis de seguridad

1. Descripción general del proyecto

- Nombre: Quillnet
- Problema que resuelve: dispositivos IoT domésticos (cámaras, asistentes de voz, smart TVs) suelen conectarse a la misma red plana que las computadoras/celulares de los usuarios, sin ningún control de tráfico entre ellos, convirtiéndose en puerta de entrada para ataques. Casos reales como la botnet Aisuru-Kimwolf (31.4 Tbps, formada en gran parte por dispositivos domésticos comprometidos) o BadBox 2.0 (millones de smart TVs comprometidos) muestran el impacto real de este problema.
- Usuarios: hogares con dispositivos IoT.
- Tecnologías: Raspberry Pi 4 (gateway), Raspberry Pi OS Lite (Debian 13), nftables, dnsmasq, Suricata 7.0.10 (IPS inline), VLANs 802.1Q, Flask + Gunicorn y PostgreSQL en Render (backend), Groq API (traducción de alertas), Resend (correo) y frontend HTML/CSS/JS.
- Tipo de información que procesa: metadatos de tráfico de red (IPs, puertos, protocolos, alertas de Suricata), no contenido de comunicaciones. También datos de cuenta (credenciales con hash scrypt), dispositivos y alertas en PostgreSQL.

- Dispositivos/servicios que forman parte: RPi4 gateway, AP TP-Link EAP110, dispositivos IoT reales por VLAN, VM Kali Linux (pentesting), API externa de Groq, WAN vía hotspot celular.

2. Alcance del análisis de seguridad

Se evalúa:

- Red local (segmentación VLAN, firewall nftables, detección Suricata)
- Backend Flask/PostgreSQL
- API de IA (Groq) como integración externa
- Frontend (dashboard)
- AP TP-Link EAP110

No se evalúa (y por qué):

- Seguridad del hotspot celular como proveedor de WAN — es infraestructura de terceros usada como stand-in de ISP para la demo, fuera del control del equipo.
- Seguridad interna del firmware de los dispositivos IoT mismos — el proyecto los trata como no confiables por diseño; busca contenerlos, no parchearlos.
- Infraestructura de Groq — solo se evalúa cómo Quillnet la consume, no su seguridad interna.

3. Identificación de activos

Activo	Importancia	Responsable	Consecuencia si se compromete
RPi4 (gateway)	Crítica	Segmentación y seguridad de la red del hogar.	Pérdida total de segmentación y detección — la red vuelve a estar plana
Reglas nftables / config VLAN	Crítica	Dispositivos, aislados en categorías y seguros.	Pérdida del aislamiento entre dispositivos
Logs de Suricata (eve.json)	Alta	Reportes de ataques realizados	Sin evidencia de ataques ni capacidad de auditoría

Base de datos PostgreSQL (alertas y cuentas)	Alta	Alertas de ataques realizados a la red	Pérdida de historial; alteración podría ocultar un ataque real
API key de Groq	Alta	Traducción de alertas	Abuso de cuota o uso no autorizado a nombre del proyecto
Código fuente (repo)	Media-Alta	Lógica de detección	Filtración de lógica de detección o credenciales hardcodeadas
Credenciales del AP TP-Link	Alta	Configuración de aislamiento	Reconfiguración maliciosa que anula el aislamiento
Documentación (Notion, manuales)	Media	Avances organizados	Afecta continuidad del proyecto
Imagen y reputación del proyecto	Media	Identidad de proyecto	Daño de credibilidad frente al jurado
Secretos de operación (gateway_secret, JWT_SECRET_KEY, claves de Groq y Resend, contraseña de la base)	Crítica	Autenticación de la Pi y de las personas; acceso a servicios externos	Suplantación del gateway o de usuarios, abuso de cuota o acceso a la base
Archivo de identidad del gateway (gateway.json)	Crítica	Autenticar a la Pi ante la nube	Quien lo copie podría enviar alertas y comandos como si fuera la Pi

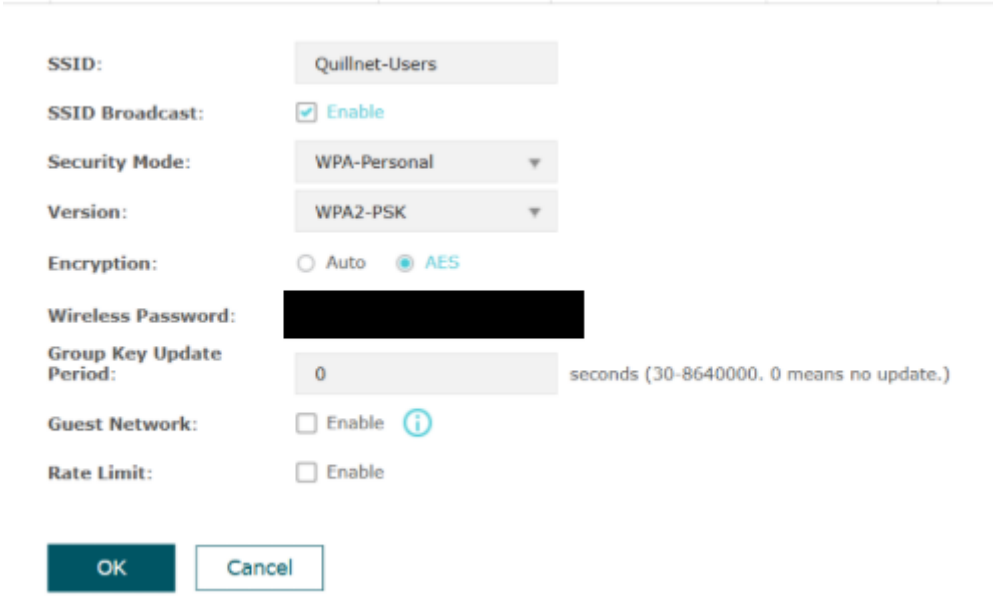
4. Análisis de la tríada CIA

Confidencialidad

- **¿Qué es privada?** Alertas de Suricata (revelan qué dispositivos existen y su comportamiento), API key de Groq, credenciales del AP y del dashboard.
- **¿Quién accede?** El equipo técnico al gateway (vía SSH); usuario final al dashboard.
- **¿Cómo se controla?** Acceso SSH al RPi4, segmentación por VLAN limita quién puede ver tráfico de quién.
- ¿Contraseñas/datos sensibles protegidos? Las contraseñas de usuarios se almacenan con hash scrypt; la API key de Groq y demás secretos se leen de variables de entorno y no del código. El inicio de sesión exige además un código de 6 dígitos enviado por correo

(almacenado solo como HMAC-SHA256). Los secretos expuestos durante el desarrollo deben rotarse.

- ¿Cifrado? El backend en Render se sirve por HTTPS, por lo que las conexiones del navegador y del gateway hacia el backend viajan cifradas. En la red inalámbrica, los cuatro SSID usan WPA2-PSK con cifrado AES y una contraseña distinta por VLAN, por lo que comprometer la red de cámaras no da acceso a la red de usuarios. Como limitaciones, WPA2-PSK usa una clave compartida por red y el access point no rota la clave de grupo (periodo de actualización en 0).



(Fuente: Propia del equipo)

- Controles adicionales implementados: el secreto del gateway se guarda en el backend solo como hash SHA-256; los códigos de verificación se guardan como HMAC; los tokens de recuperación se guardan como SHA-256 y viajan en el fragmento (#) de la URL para no quedar en los registros del servidor; y a la IA nunca se le envían IPs, MAC ni nombres.

Cumplimiento: la segmentación VLAN impide que un dispositivo en VLAN 20 vea tráfico de VLAN 10, aunque compartan la misma red física. Problema: sin autenticación fuerte en el acceso administrativo, un compromiso de esa credencial anula toda la arquitectura. Mejora: autenticación por llave SSH (no contraseña) + API key en variable de entorno. Problema adicional: el gateway aún no tiene una cadena input en nftables, por lo que sus propios servicios (por ejemplo SSH)

podrían ser alcanzables desde los segmentos IoT. Mejora: cadena input con política drop y SSH solo por llave desde la VLAN de usuarios.

Integridad

- ¿Quién modifica datos? Solo Suricata (en el gateway) escribe en eve.json; solo el backend escribe en PostgreSQL.
- **¿Cómo se evita alteración?** En la documentación técnica no consta evidencia de control de integridad (permisos de solo-lectura, hashes) sobre los logs una vez escritos.
- **¿Se registran cambios?** Los logs de Suricata registran eventos de red, pero no hay evidencia de logging de cambios administrativos al propio sistema (ej. quién modificó una regla de nftables y cuándo). En la base de datos se conservan registros de auditoría: la tabla de desafíos de verificación, los tokens de recuperación y la cola de comandos (con su origen manual o automático).
- ¿Validaciones de datos? El backend valida y sanea los datos de entrada; además, la inteligencia artificial recibe únicamente el texto técnico de la alerta, sin direcciones IP.
- ¿Copias de seguridad? La base de datos es un servicio gestionado en Render. Se realizó un respaldo completo con pg_dump (26 de octubre de 2026). No consta una copia de la configuración del gateway fuera de la microSD original; generarla es una mejora pendiente del plan.

Cumplimiento: el pipeline de detección (Suricata → eve.json → backend) tiene una sola fuente de verdad. Problema: no hay control de integridad sobre los logs una vez generados. Mejora: permisos de solo-lectura sobre eve.json y backup periódico de la configuración de nftables/VLANs.

Disponibilidad

- **¿Qué tumbaría el proyecto?** Falla del RPi4 (hardware único, sin redundancia), pérdida de conexión WAN, corrupción de la microSD.
- **¿Copia de seguridad?** La microSD original con la configuración previa se conserva intacta como respaldo funcional. No consta una imagen de respaldo del estado actual (VLANs, nftables, Suricata); generarla es una mejora pendiente del plan.

- **¿Recuperación?** Sin imagen de respaldo actualizada, la recuperación implicaría reconfigurar todo manualmente — riesgo real de cara a una demo en vivo.
- ¿**Qué pasa si falla Internet/servidor/electricidad**? Si falla la WAN, la segmentación interna debería seguir funcionando — vale la pena confirmarlo con una prueba explícita (desconectar WAN y verificar que el tráfico inter-VLAN sigue bloqueado). Prueba realizada: los servicios y las reglas del gateway se restablecen tras un reinicio y tras un corte de energía. Esto se debe a que los tres servicios propios tienen reinicio automático (Restart=always) y a que, al arrancar, quillnet-control consulta al backend qué dispositivos deben seguir bloqueados y reaplica sus reglas, pues nftables no conserva reglas dinámicas entre reinicios. Además, los registros del 4 de noviembre de 2026 muestran que ante el arranque en frío del plan gratuito de Render (tiempos de espera agotados) los scripts capturan el error, lo registran y reintentan en el siguiente ciclo sin detenerse.
- **¿Redundancia?** No hay redundancia de hardware (un solo RPi4) — aceptable en un prototipo educativo, pero debe declararse como limitación conocida.

Adicionalmente, existe una línea de trabajo en evaluación sobre unificar el módem, el gateway y el AP en un solo dispositivo físico. Esto no es una vulnerabilidad de seguridad, sino una decisión de diseño con implicación directa en disponibilidad: reduciría puntos de falla por cableado externo, pero concentraría todo el sistema en un único punto de fallo total. Es un trade-off que debe evaluarse antes de implementarse, no algo que ya esté resuelto.

Cumplimiento: existe una copia funcional de respaldo de una configuración previa conocida. Problema: no hay evidencia de una imagen de respaldo actualizada del estado de configuración actual. Mejora: clonar la microSD actual una vez estabilizada la configuración, antes del Expotec (13 al 15 de noviembre de 2026).

5. Identificación de vulnerabilidades

Vulnerabilidad	Componente afectado	Consecuencia posible	Facilidad de Explotación
Suricata en modo IPS con bypass (fail-open)	Capa de prevención	Si el motor falla, el tráfico pasa sin inspección	Baja — decisión de diseño que prioriza disponibilidad
VLAN hopping	Segmentación de red	Si fuera posible, rompe el pilar central de la arquitectura	Gracias a la segmentación no fue posible
Secretos expuestos durante el desarrollo (clave JWT, clave de Groq, contraseña de la base)	Backend y servicios externos	Suplantación de sesiones, abuso de cuota o acceso a la base	Media: se rotan al final de las demostraciones para no romperlas
Falta de control de integridad en logs	eve.json / PostgreSQL	Alteración de evidencia por alguien con acceso al sistema	Media (requiere acceso previo)
Ausencia de imagen de respaldo actualizada	Todo el gateway	Recuperación lenta ante falla de hardware/SD	N/A — brecha operativa
Autenticación admin del AP/gateway sin confirmar	AP EAP110, RPi4	Acceso no autorizado a la configuración de VLANs	Depende de si se cambiaron contraseñas por defecto
Sin verificación de identidad de dispositivo por VLAN	Segmentación	Cualquier dispositivo conectado al SSID correcto hereda la confianza de esa VLAN	Media — no requiere exploit, solo cercanía física
Ausencia de cadena input en nftables	Gateway (RPi4)	Servicios de la Pi alcanzables desde segmentos IoT	Media — requiere un dispositivo comprometido en la red
Dependencia de planes gratuitos o de corto plazo (Render, Groq)	Render / PostgreSQL / Groq	Interrupciones o pérdida de datos	Ocurrió el 27 de octubre de 2026; mitigado con una renovación temporal
Detección no verificada en VLAN 20, 30 y 40	Suricata	Posibles puntos ciegos fuera de la VLAN probada	Desconocida — pendiente de prueba
Token JWT en la URL de la conexión SSE	Backend / frontend	El token podría quedar en registros de acceso	Media
Limitador de tasa en memoria de un solo proceso	Flask-Limiter	Contadores no compartidos ni persistentes	Baja

Sin revocación explícita de JWT	Sesiones	Una sesión robada sirve hasta 7 días o hasta cambiar la contraseña	Media
Remitente de correo sin dominio propio	Resend	Los correos (2FA, recuperación) solo llegan al titular de la cuenta del servicio	Media
Tráfico intra-VLAN no inspeccionado	Segmentación / Suricata	Un equipo comprometido puede atacar a otro de su misma VLAN sin que el IPS lo vea	Media: no requiere exploit, solo estar en la misma VLAN; mitigación: aislamiento de clientes en el AP
WPA2-PSK con clave compartida por VLAN y sin rotación de clave de grupo	Access point EAP110 (firmware 2022)	Quien conozca la clave de una VLAN entra a esa VLAN	Media: requiere conocer la clave; la clave es distinta en cada VLAN
Tráfico cifrado (TLS) sin inspeccionar	Suricata	Las firmas de contenido no aplican a descargas por HTTPS	Media: limitación conocida de un IPS sin inspección TLS

6. Identificación de amenazas

Amenaza	Vulnerabilidad que aprovecha	Activo afectado	Consecuencia
Ataque DDoS desde IoT comprometido (tipo Aisuru)	Ausencia de mitigación de ataques volumétricos (el IPS actúa por firmas)	Ancho de banda, disponibilidad	Saturación de la red, posible desconexión
VLAN hopping	Segmentación no validada empíricamente	Todos los activos en las 4 VLANs	Pérdida total del aislamiento entre segmentos
Robo/filtración de API key de Groq	Manejo inseguro de credenciales	Cuenta de Groq del proyecto	Abuso de cuota, gasto no autorizado
Acceso no autorizado al AP	Contraseña débil/por defecto	Configuración de VLANs	Reconfiguración maliciosa que anula la segmentación
Inyección hacia el LLM (prompt injection vía alertas)	Falta de validación de datos antes de enviar a Groq	Calidad de las alertas mostradas al usuario	Alertas falsas o manipuladas. Mitigado parcialmente: la clasificación del tipo de ataque usa reglas fijas y una lista cerrada, y a la IA solo le llegan la firma, la

Amenaza	Vulnerabilidad que aprovecha	Activo afectado	Consecuencia
Ataque DDoS desde IoT comprometido (tipo Aisuru)	Ausencia de mitigación de ataques volumétricos (el IPS actúa por firmas)	Ancho de banda, disponibilidad	Saturación de la red, posible desconexión
VLAN hopping	Segmentación no validada empíricamente	Todos los activos en las 4 VLANs	Pérdida total del aislamiento entre segmentos
Robo/filtración de API key de Groq	Manejo inseguro de credenciales	Cuenta de Groq del proyecto	Abuso de cuota, gasto no autorizado
			categoría, la acción y el protocolo.
Falla de hardware sin respaldo actualizado	Ausencia de imagen de respaldo reciente	Disponibilidad de todo el sistema	Pérdida de horas de trabajo, riesgo de falla en demo

7. Evaluación de Riesgos

Riesgo	Probabilidad	Impacto	Nivel	Justificación
Cortocircuito con el cableado del stand, que puede poner en riesgo la integridad del edificio.	Bajo	Medio	Medio	La probabilidad de que algo como esta situación se presente es baja ya que el edificio tiene un sistema de cableado pensado para soportar grandes cantidades de energía. También es bueno considerar la posibilidad de que el consumo sea mayor por los distintos proyectos que se usan. Su impacto sería bajo ya que el interruptor termomagnético se detendrá al instante. Por eso se considera que su nivel de riesgo es medio.
Problemas con la estructura del stand en el momento de la exposición, haciendo que	Baja	alto	medio	La probabilidad de que esto ocurra es baja ya que se planea que la estructura soporte la tapadera base como las lonas o la tela. Aunque puede existir el riesgo de que al colocar un

esta se derrumbe o no soporte el peso de la decoración.				objeto pesado como un televisor, este pueda colapsar. El impacto sería alto ya que supone que la estructura base se derrumbe dejando sin espacio para presentar durante la feria. Por eso su nivel de riesgo es medio ya que a pesar de que su probabilidad sea baja, su impacto en la integridad de los estudiantes y el proyecto sería alta.
Fallo en los sistemas de la raspberry Pi con los logs y la base de datos PostgreSQL	Media	Media	Media	La probabilidad de que ocurra un problema con los servicios de red que proporciona la raspberry pi es media ya que puede tratarse tanto de una mala configuración como un error humano. Su impacto será medio ya que solo se tendría que evaluar el error y corregir rápidamente mediante un corto análisis por parte del equipo técnico.
Falla en el hardware sin respaldo previo al accidente.	Bajo	Alto	Medio	La probabilidad de que ocurra un accidente que deje sin sistema al dispositivo principal sin un respaldo previo es muy baja, primero porque el dispositivo estaría protegido por una carcasa protectora además de que su configuración base para su funcionamiento natural tomaría solo un corto lapso de tiempo para poder configurarse. Su impacto en el momento sería alto porque podría afectar a los sistemas base o ya configurados en el momento de usar el dispositivo dejando a las redes desprotegidas.
Uso de VLANs o sistemas de protección y segmentación no aprobadas o verificadas.	Medio	Medio	Medio	En el dispositivo Quillnet la probabilidad de que las VLANs no estén aprobadas o no tengan un sistema verificado es media ya que se hacen pruebas continuas de pentesting para verificar su seguridad. Es cierto que también hay que considerar la posibilidad de que estas VLANs sean vulneradas por algún atacante. Su impacto sería grande ya que sería una forma de explotar una vulnerabilidad del sistema que puede ser crítico

				para los usuarios. Por eso su nivel de impacto será Medio.
Vencimiento de la base de datos gratuita de Render (27 de octubre de 2026)	Media	Alto	Alto	El riesgo se materializó el 27 de octubre de 2026 y se mitigó con una renovación de pago por tiempo limitado si esa renovación vence antes de la presentación, el backend pierde su base de datos.
Gateway accesible desde dispositivos IoT por ausencia de cadena input	Media	Alto	Alto	Un dispositivo comprometido podría intentar acceder a servicios de la Raspberry Pi.

8. Medidas de seguridad

Riesgo atendido	Medida propuesta	Tipo de medida	Responsable	Prioridad
Cortocircuito con el cableado del stand, que puede poner en riesgo la integridad del edificio.	Equipo de reemplazo	Medida de recuperación	El analista de riesgos del proyecto	Alta
Problemas con la estructura del stand en el momento de la exposición, haciendo que esta se derrumbe o no soporte el peso de la decoración.	Plan de recuperación	Medida de Recuperación	El analista de riesgos del proyecto	Alta
Fallo en los sistemas de la raspberry Pi con los logs y la base de datos PostgreSQL	Validación de datos	Medida preventiva	Los encargados de la parte técnica de desarrollo y programación.	Alta
Falla en el hardware sin respaldo previo al accidente.	Copias de seguridad	Medida de recuperación	Los encargados de la infraestructura de red del proyecto.	Alta
Uso de VLANs o sistemas de protección y	Alertas.	Medidas de detección	Los encargados de la parte de programación e	Alta

segmentación no aprobadas o verificadas.			infraestructura de red del proyecto.	
Vencimiento de la base de datos / secretos expuestos	Migrar o renovar PostgreSQL y rotar los secretos usados durante el desarrollo. Estado: base renovada; rotación de secretos programada después de la última demostración.	Medida preventiva y de recuperación	Encargados de desarrollo web	Alta
Gateway accesible desde dispositivos IoT	Cadena input con política drop y SSH por llave solo desde la VLAN de usuarios	Medida preventiva	Encargados de infraestructura de red	Alta

9. Seguridad de acceso y control de usuarios

- **¿Qué tipos de usuarios existen?** Existen dos actores autenticados y un perfil proyectado. (1) La persona del hogar: cuenta con correo y contraseña, vinculada a un gateway mediante un código de emparejamiento de 8 caracteres de un solo uso que vence en 15 minutos; puede ver el mapa, el panel y las alertas, aprobar, bloquear y desbloquear dispositivos y cambiar sus ajustes, y solo ve los datos de su propio hogar. (2) La Raspberry Pi: se autentica con su propio secreto (no con usuario y contraseña) y solo puede enviar alertas y presencia, consultar su cola de comandos y confirmarlos. (3) El técnico del ISP (perfil proyectado): tendría una consola de soporte multi-hogar; no está implementado en el prototipo y queda como trabajo futuro. Las personas invitadas que se conectan al Wi-Fi no tienen cuenta ni acceso al panel; su tráfico queda sujeto a las reglas de la VLAN a la que pertenecen.

- **¿Cómo se registran e inician sesión?** La persona crea su cuenta con correo y contraseña; el correo se valida (sintaxis, registros MX/A/AAAA del dominio, lista de dominios desechables y sugerencia de correcciones de errores comunes). Para vincular su hogar ingresa el código de emparejamiento. El inicio de sesión entrega un JWT de 7 días que incluye el identificador del gateway, y ante credenciales incorrectas se muestra un mensaje genérico.
- **¿Cómo se recuperan las contraseñas?** Con «Olvidé mi contraseña» se envía al correo un enlace de un solo uso que vence en 30 minutos. El sistema responde igual exista o no la cuenta, para no revelar qué correos están registrados; el token se guarda solo como hash SHA-256 y viaja en el fragmento de la URL. La contraseña solo puede cambiarse mediante ese enlace y, al hacerlo, se invalidan las sesiones anteriores.
- **¿Cómo se bloquea un dispositivo o cuenta?** Cualquier persona del hogar con sesión activa puede bloquear un dispositivo de inmediato (la Raspberry Pi consulta los comandos cada 5 segundos). Además, el sistema aplica cuarentena automática cuando un mismo dispositivo genera 3 alertas de bloqueo real en 5 minutos. Para desbloquear se exige verificación en dos pasos: contraseña y un código de 6 dígitos enviado por correo (10 minutos, máximo 5 intentos). La asimetría es intencional: bloquear reduce el riesgo; desbloquear lo reabre.
- **¿Qué ocurre cuando un usuario deja de utilizar el sistema?** La sesión (JWT) vence a los 7 días y el cambio de contraseña invalida las sesiones emitidas antes. Aún no existe revocación explícita de tokens individuales (limitación conocida, ver plan de mejora).
- **¿Cómo se aplica el principio de mínimo privilegio?** En tres niveles: cada cuenta solo accede a los datos de su gateway (toda consulta filtra por identificador de gateway y uno ajeno responde 404); la Raspberry Pi solo puede usar los endpoints del gateway y su secreto se guarda únicamente como hash; y en el propio gateway quillnet-agente y quillnet-vigia corren con un usuario sin privilegios, y solo quillnet-control corre como root porque necesita manipular nftables.

Función	Persona del hogar	Raspberry Pi (gateway)	Técnico ISP (proyectado, no implementado)
Ver mapa, panel y alertas de su hogar	Permitido	No aplica	Proyectado (multi-hogar)
Aprobar un dispositivo nuevo	Permitido	No	Proyectado
Bloquear un dispositivo	Permitido (directo)	Ejecuta el comando	Proyectado
Desbloquear un dispositivo	Permitido con verificación en dos pasos	Ejecuta el comando	Proyectado
Enviar alertas y presencia	No	Permitido	No
Cambiar la contraseña	Solo por enlace de un solo uso por correo	No	No
Borrar alertas	Permitido (borrado lógico; siguen contando en estadísticas)	No	Proyectado
Modificar VLAN o reglas de firewall	No (no hay pantalla para ello)	Se configuran por SSH	Proyectado

10. Protección de datos y privacidad

- **¿Qué datos personales se recopilan?** Al ser Quillnet un sistema que intercepta el tráfico de red solo recopila información del tráfico de red, no guarda los datos o peticiones de la red por motivos de seguridad. Solo los datos de envío y las IPs necesarias. Además de almacenar la configuración de las VLANs y los dispositivos conectados Datos de cuenta (correo, nombre, contraseña con hash, provincia y cantón del hogar) y datos de dispositivos (dirección MAC, IP, nombre y última vez visto).
- **¿Para que se utilizan?** Toda la información es guardada a manera de registro del tráfico para poder interceptar consultas sospechosas. Los dispositivos se guardan también a una manera de registro.

- **¿Dónde se almacenan?** En una base de datos PostgreSQL alojada en Render (proveedor en la nube), compartida entre hogares y separada lógicamente por identificador de gateway (multi-tenant). En el modelo proyectado esta base podría operar bajo responsabilidad del ISP. Los datos viajan cifrados mediante HTTPS.
- **¿Quién puede consultarlos?** Cada persona consulta únicamente los datos de su propio hogar. El equipo de desarrollo tiene acceso a la base mediante credenciales de operación.
- **¿Durante cuánto tiempo se conservarán?** Las alertas se conservan 90 días; un proceso automático de retención las elimina al vencer ese plazo, junto con los tokens y códigos de verificación vencidos hace más de 30 días. Ambos plazos son configurables.
- **¿Cómo se eliminarán de forma segura?** La persona puede borrar alertas de su bandeja: es un borrado lógico (la alerta deja de mostrarse pero se conserva para las estadísticas hasta cumplir el plazo de retención). Al vencer los 90 días, el proceso de retención las elimina de la base de datos. Los respaldos (pg_dump) conservan copias hasta que se eliminan manualmente
- **¿Qué datos no deberían recopilarse?** No deberían recopilarse datos con respecto a las cuentas de otros sistemas de los usuarios, tokens, cookies, contraseñas, peticiones a servidores, acciones concretas en una web entre otros datos que no deben de ser recopilados.

11. Desarrollo seguro

- **¿Cómo se validan los datos ingresados?** El backend valida las entradas y accede a la base mediante SQLAlchemy con consultas parametrizadas (las pautas de desarrollo del proyecto exigen parámetros y prohíben armar consultas SQL con f-strings), lo que previene la inyección SQL; la cabecera Content-Security-Policy restringe los scripts permitidos, mitigando XSS.
- **¿Cómo se manejan los errores?** Se manejan con cuidado para que el sistema no colapse si existe algún error, frecuentemente se hacen pruebas de funcionamiento del proyecto para verificar que los sistemas operan sin complicaciones.

- **¿Cómo se protegen las credenciales?** Mediante un almacenamiento seguro en las bases de datos y un sistema de hash en el cual las contraseñas son almacenadas de manera segura mediante algoritmos criptográficos estrictos.
- **¿Existen contraseñas escritas directamente en el código?** No. Los secretos se leen de variables de entorno (en Render) o de archivos .env locales excluidos del repositorio; en el gateway, el secreto está en gateway.json, fuera del repositorio. Los secretos que quedaron expuestos durante el desarrollo se rotarán al finalizar las demostraciones.
- **¿Cómo se actualizan las dependencias?** Mediante gestores de paquetes como pip en python que es el lenguaje que se usa para el backend. Manteniendo las dependencias actualizadas y verificando su compatibilidad para que no existan errores cruzados entre dos o más dependencias.
- **¿Cómo se controla el acceso al código fuente?** Absolutamente ningún usuario doméstico tiene acceso al código fuente. Los usuarios técnicos sólo tienen acceso a lo básico del software. Solo los desarrolladores principales tienen acceso al código fuente para poder modificarlo mediante un estricto control en donde deben modificar cada cambio a la infraestructura del sistema.
- **¿Qué pruebas de seguridad se realizaron?** Se realizaron pruebas manuales de detección y bloqueo desde Kali Linux (un beacon real de Covenant, MQTT sin cifrar, sqlmap y otras; ver sección 14) y 238 pruebas automatizadas del backend que incluyen casos de seguridad (por ejemplo, se comprobó que la prueba falla si el código de verificación se guarda con un hash simple). La batería formal de pentesting por VLAN, la prueba de VLAN hopping y las pruebas de carga siguen pendientes.

12. Copias de seguridad

- **¿Qué información necesita respaldo?** Las contraseñas, usuarios, correos y redes vinculadas a un dispositivo quillnet. Estos datos se guardan en la base de datos y necesitan un respaldo ya que son los que se necesitan para autenticar a los usuarios y proteger las redes con dispositivo Quillnet. Además de la base de datos, se respalda la configuración del gateway (VLAN, nftables, dnsmasq, Suricata y unidades systemd) y su archivo de identidad.

- **¿Cada cuánto se realizará la copia?** Prototipo respaldo manual con pg_dump; el último completo se realizó el 26 de octubre de 2026 . Producto (proyectado): copia automática semanal. Para así que los datos que están guardados sean almacenados seguramente en una copia. Permitiendo evitar fallos en los dispositivos que las almacenan.
- **¿Dónde se almacenará?** Se almacenará en la nube para garantizar su seguridad. Permitiendo así que no se necesiten guardar datos en dispositivos físicos pertenecientes a los ISP's. Esto permite un control más accesible para el dispositivo.
- **¿Quién será responsable?** El responsable digital de mantener los datos seguros serán los técnicos o especialistas en ciberseguridad de los ISP's. Mientras que los responsables de la seguridad física de los dispositivos que almacenan esta información son los encargados de los servidores en la nube.
- **¿Cómo se comprobará que la copia funciona?** Haciendo pruebas en el contenido de estas copias para verificar que todos los datos están siendo almacenados de manera íntegra y sin ningún tipo de cambio. Estas pruebas se realizan justo después de almacenar la copia para verificar su estado.
- **¿Cuánto tiempo tomaría recuperar el proyecto?** Backend: nuevo despliegue en Render en unos 2 minutos y restauración de la base desde pg_dump de 5 a 10 minutos . Gateway: con la microSD de respaldo, 10 a 15 minutos; sin una imagen actualizada habría que reconfigurar manualmente (varias horas).

13. Respuesta ante incidentes

- **Paso 1:** Se debe identificar el incidente, para esto se consultan con los especialistas para rastrear la raíz del problema. Posteriormente se informa al personal de seguridad informática o el SOC para que se inicien los protocolos de recuperación y documentación, además de avisar a la autoridad del ISP correspondiente
- **Paso 2:** Una vez identificado el error se debe contener el error mediante protocolos ya sean físicos (como protección de dispositivos por medio de guardas de seguridad entre otros) o métodos informáticos (cerrar puertos, bloquear accesos entre otros). A su vez, otro equipo investiga más en profundidad para monitorear las acciones del SOC y asegurarse de que no

es un Falso positivo o un Falso negativo. Se deben hacer copias de seguridad inmediatas de la información además de guardar las evidencias relacionadas con el incidente. Una vez identificada la causa se debe eliminar y prevenir más de ellas.

- **Paso 3:** Una vez eliminada la causa se deben probar todos los servicios para asegurar su integridad, en caso de que uno de ellos haya sufrido daños, se necesitará recuperarlo lo antes posible, ya sea por medio de copias de seguridad o del código. El equipo de operaciones deberá documentar un informe donde especifique cada uno de los protocolos llevados a cabo para poder recuperar el sistema. También los encargados deben informar a los respectivos departamentos para evitar que vuelvan a ocurrir estos tipos de accidentes.
- **Aplicación en Quillnet:** (1) Identificar: el agente envía la alerta y el backend la clasifica en uno de 13 tipos. (2) Contener: la cuarentena automática bloquea por dirección MAC al dispositivo tras 3 alertas de bloqueo real en 5 minutos, o la persona lo bloquea manualmente. (3) Recuperar: el desbloqueo exige verificación en dos pasos; ante sospecha de cuenta comprometida se cambia la contraseña por enlace de un solo uso, lo que invalida las sesiones anteriores; ante la filtración de un secreto se rotan la clave JWT, las claves de Groq y Resend y la contraseña de la base.

14. Análisis de resultados

Pruebas de detección y bloqueo de amenazas

Las siguientes pruebas permiten comprobar la capacidad de Quillnet para detectar y bloquear comunicaciones sospechosas generadas por un dispositivo IoT comprometido. Se utilizó Suricata en modo IPS para interceptar el tráfico y registrar los eventos de seguridad.

Las pruebas de la batería más reciente se ejecutaron el 5 de noviembre de 2026 desde una Kali (192.168.10.171) ubicada en la VLAN 10, con el framework tmNIDS (testmynids.org) y herramientas como curl, sqlmap y mosquitto_pub. La evidencia se obtuvo filtrando eve.json con jq y observando el panel y la app. Se distingue entre resultados confirmados (con línea de log) y resultados pendientes de evidencia.

Prueba	Objetivo	Resultado	Evidencia	Mejora aplicada
Ataque 1: Inyección SQL automatizada (sqlmap)	Simular el reconocimiento y la explotación automatizada que recibiría cualquier panel web expuesto en la red doméstica (ej. interfaz de administración de una cámara o dispositivo IoT).	El request se completó (sqlmap recibió respuesta del servidor de prueba). El informe del 5 de noviembre muestra que, contra un equipo de la misma VLAN 10, no se generó alerta: el tráfico dentro de una misma VLAN no pasa por el gateway y Suricata no lo inspecciona (hallazgo incluido en el plan de mejora). La detección de la firma 9000001 requiere que el tráfico atraviese el gateway; esa variante queda pendiente de repetir con evidencia.	sqlmap -u "http://192.168.10.157/?id=1" --batch --level=1	Demuestra la capacidad de detección de la red ante herramientas de explotación automatizada en tiempo real.
Ataque 2: Check-in de malware simulado	Simular el momento en que un dispositivo comprometido reporta sus datos a un servidor remoto para confirmar que sigue "vivo" y recibir instrucciones (patrón clásico de malware basado en instaladores NSIS).	La conexión no se completó; ambas firmas dispararon y el registro de eventos confirma la acción de bloqueo para las dos. El informe del 5 de noviembre registra esta prueba como pendiente de evidencia fotográfica.	curl -X POST "http://1.1.1.1/temp/gate.php" -A "NSIS_Inetc" -d "rms_id=1&right=SYSTEM&os=test&hostname=test&domain=test" --max-time 3	Demuestra la capacidad de bloqueo activo frente a un escenario de post-explotación (dispositivo intentando comunicarse hacia afuera).

Prueba 2: beacon de C2 real (Covenant)	Comprobar el bloqueo en línea de un beacon real del framework Covenant (ET MALWARE Covenant Framework HTTP Beacon, sid 2027793), generado desde la Kali en la VLAN 10.	Bloqueado en línea (acción: blocked), severidad 1 (alta); el panel lo clasificó como Malware y la app lo notificó. Confirmado.	/tmp/tmNIDS -15 (tmNIDS, testmynids.org).	Ninguna requerida; la regla está en modo drop (12,860 reglas en drop).
Prueba 5: MQTT sin cifrar (puerto 1883)	Observar qué ocurre con una publicación a un broker MQTT público sin TLS, práctica riesgosa típica en IoT.	El comando quedó colgado, el puerto 1883 fue inalcanzable y no hubo alerta; el tráfico no salió de la red. Confirmado en el informe del 5 de noviembre; no se determinó si el corte lo hizo el gateway o la red del operador móvil.	mosquitto_pub -h test.mosquitto.org -t casa/quillnet-test -m "reboot"; timeout 5 bash -c 'echo > /dev/tcp/test.mosquitto.org/1883'.	Repetir desde otra red para atribuir el corte.
Prueba 6: sqlmap contra un equipo de la misma VLAN	Comprobar si Quillnet detecta un ataque web lanzado dentro de la VLAN (regla propia 9000001).	Sin alerta contra 192.168.10.123: el tráfico entre equipos de una misma VLAN no pasa por el gateway. Confirmado (variante externa pendiente).	sqlmap contra el equipo local de la VLAN 10.	Aislamiento de clientes en el access point.
Prueba 8: dropper (descarga de ejecutable)	Detectar la descarga de un ejecutable de prueba (tmNIDS -6).	Sin alerta en el panel. Confirmado. Causas posibles según el informe: la firma que corresponde es informativa(severidad 3) y Quillnet la descarta por diseño, o la descarga viajó cifrada (HTTPS) y un IPS sin inspección TLS no puede aplicar firmas de contenido.	tmNIDS -6.	Por determinar.

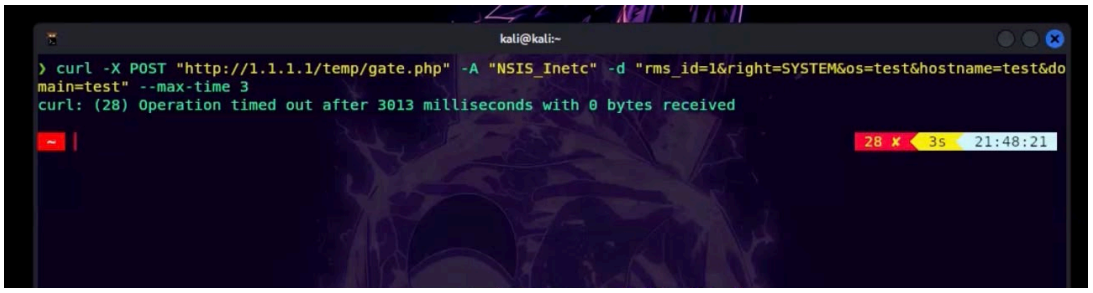
Prueba 10: respuesta «id root»	Verificar la firma GPL ATTACK_RESPONSE (sid 2100498, tmNIDS -1).	No disparó (probable ausencia de reglas GPL). Confirmado. Hipótesis del informe: el subconjunto de reglas GPL no está incluido en el conjunto ET Open cargado.	tmNIDS -1.	Verificar si debe cargarse el conjunto GPL.
Prueba 3: control remoto (regla propia 9000010)	Comprobar el bloqueo de un beacon de control remoto hacia un servidor externo.	Pendiente de evidencia (por confirmar).	curl -m 5 "http://1.1.1.1/quillnet-c2".	—
Prueba 4: tráfico sospechoso (regla propia 9000011)	Comprobar el bloqueo de tráfico anómalo sin patrón de familia conocida.	Pendiente de evidencia (por confirmar).	curl -m 5 "http://1.1.1.1/quillnet-raro".	—
Prueba 7: navegación riesgosa (Tor)	Detectar tráfico de la red Tor (tmNIDS -5).	Pendiente de evidencia (por confirmar).	tmNIDS -5.	—
Prueba 9: adware/PUP	Detectar adware (tmNIDS -14).	Pendiente de evidencia (por confirmar).	tmNIDS -14.	—
Movimiento lateral: Usuarios a Cámaras	Comprobar que un equipo de la VLAN 10 no alcanza a uno de la VLAN 20; usar un equipo real de la VLAN 20 (192.168.20.x), no 192.168.20.1, que es la propia Raspberry Pi.	Pendiente de evidencia (por confirmar).	ping y curl al equipo de la VLAN 20; sudo nft list ruleset.	—
Cuarentena	Comprobar que 3 alertas de bloqueo real en 5 minutos	Pendiente de evidencia (por	Tres líneas blocked consecutivas y la	—

automática	dejan el dispositivo aislado por su dirección MAC.	confirmar).	Kali en estado bloqueado en el mapa.	

De estas pruebas se concluye que Quillnet bloquea en línea el tráfico malicioso saliente de la VLAN de Usuarios cuando existe una regla en modo drop (12,860 de las 52,246 reglas cargadas), que el canal MQTT inseguro hacia el exterior no estuvo disponible, y que el tráfico dentro de una misma VLAN y el tráfico cifrado quedan fuera de la inspección actual. Las pruebas marcadas como «por confirmar» no se consideran logradas hasta contar con su evidencia.

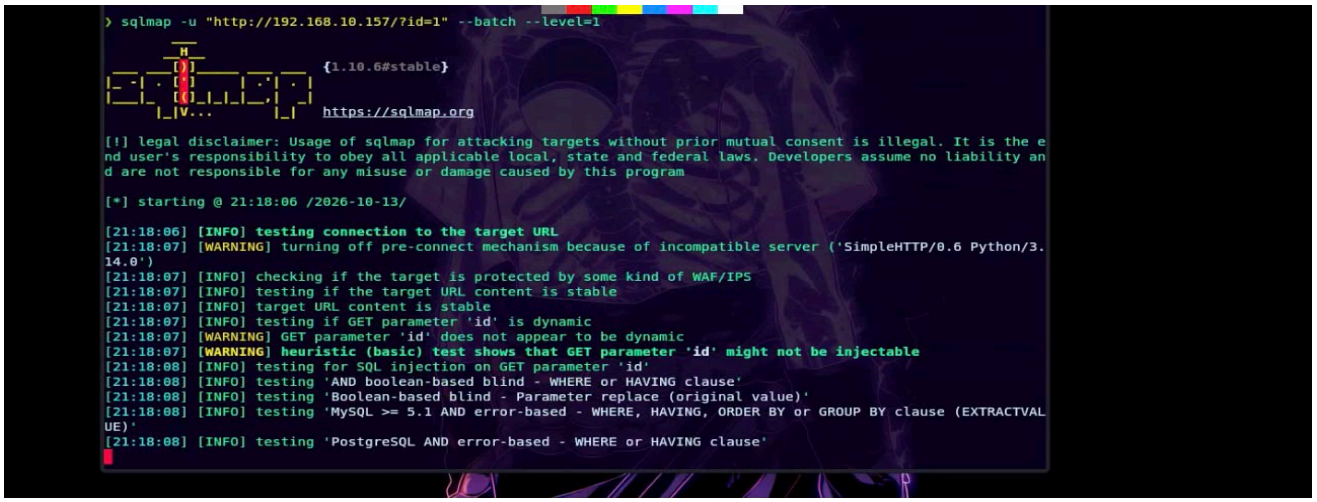
Imágenes de evidencia

Check-in de malware simulado:



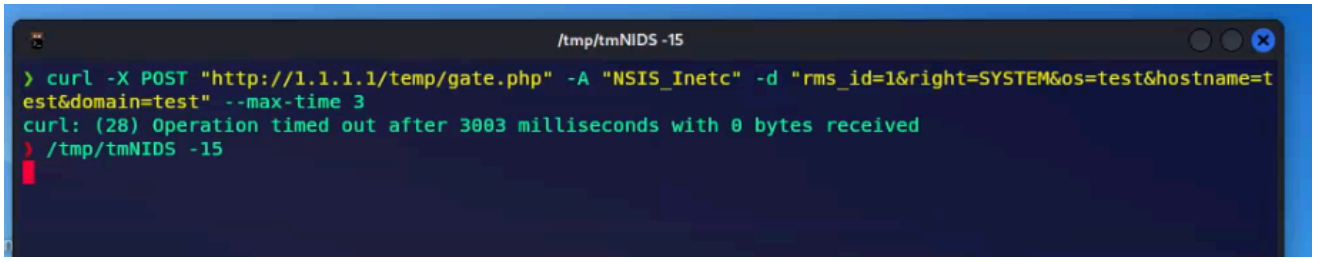
(Fuente: Propia del equipo)

Inyección SQL automatizada (sqlmap):

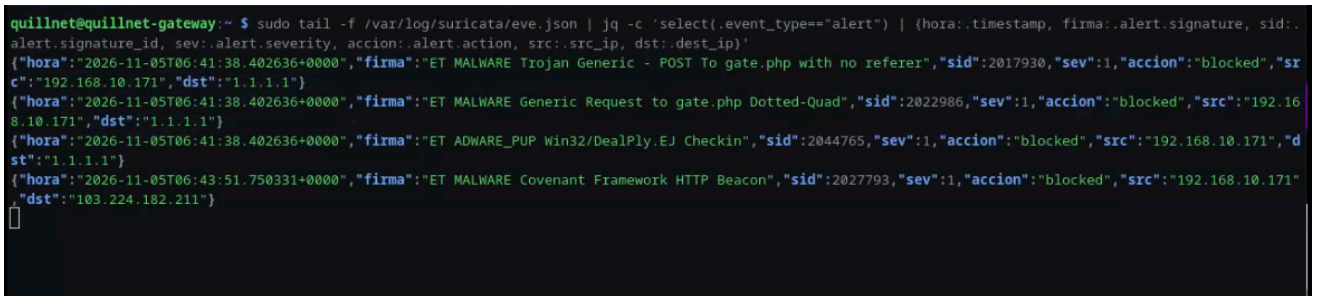


(Fuente: Propia del equipo)

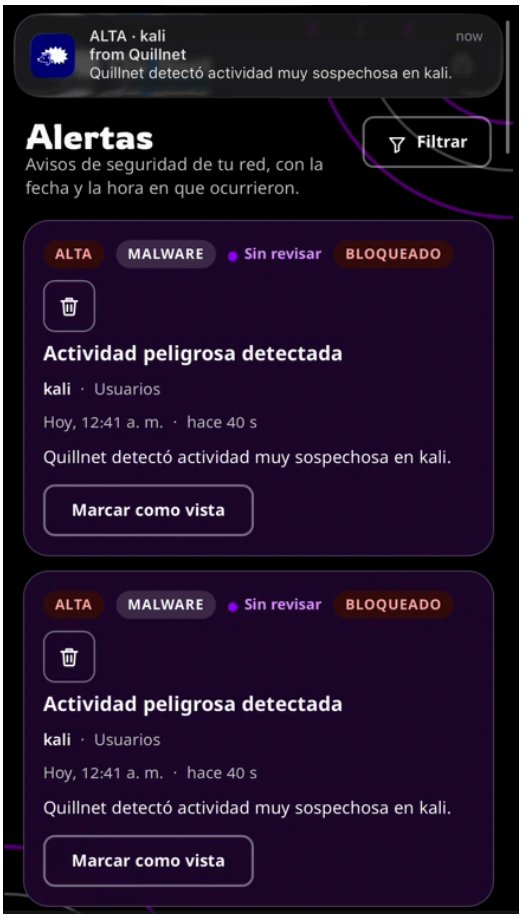
Beacon de C2 real (Covenant):



(Fuente: Propia del equipo)

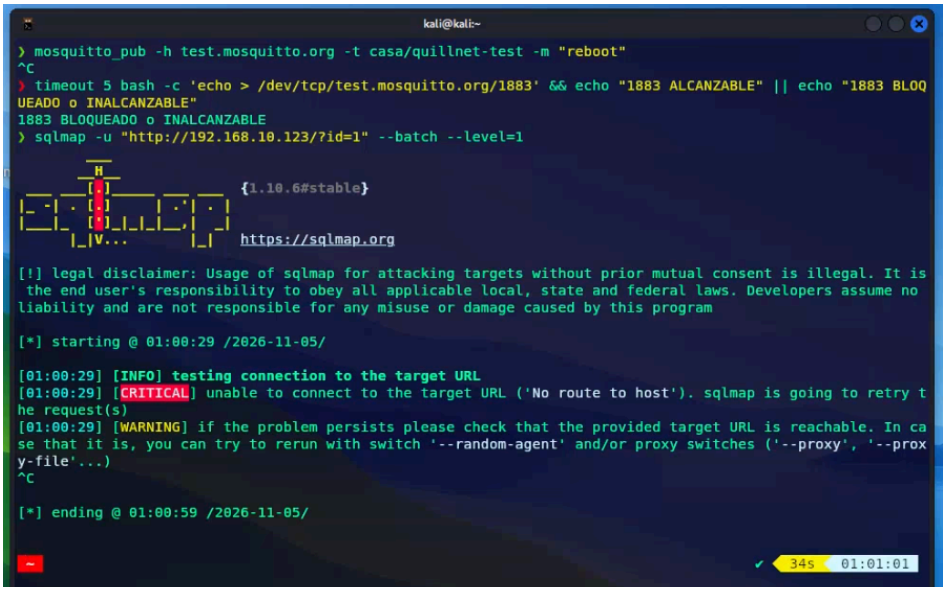


(Fuente: Propia del equipo)



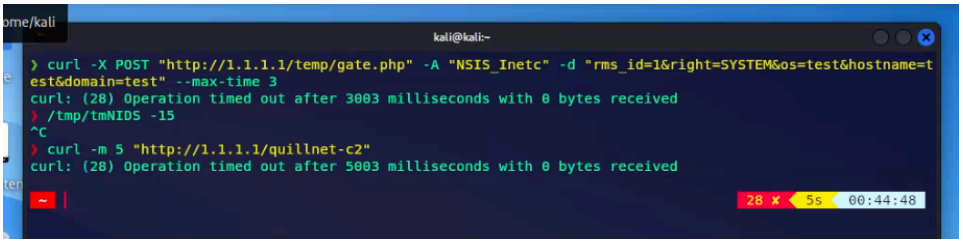
(Fuente: Propia del equipo)

sqlmap contra un equipo de la misma VLAN (sin alerta):

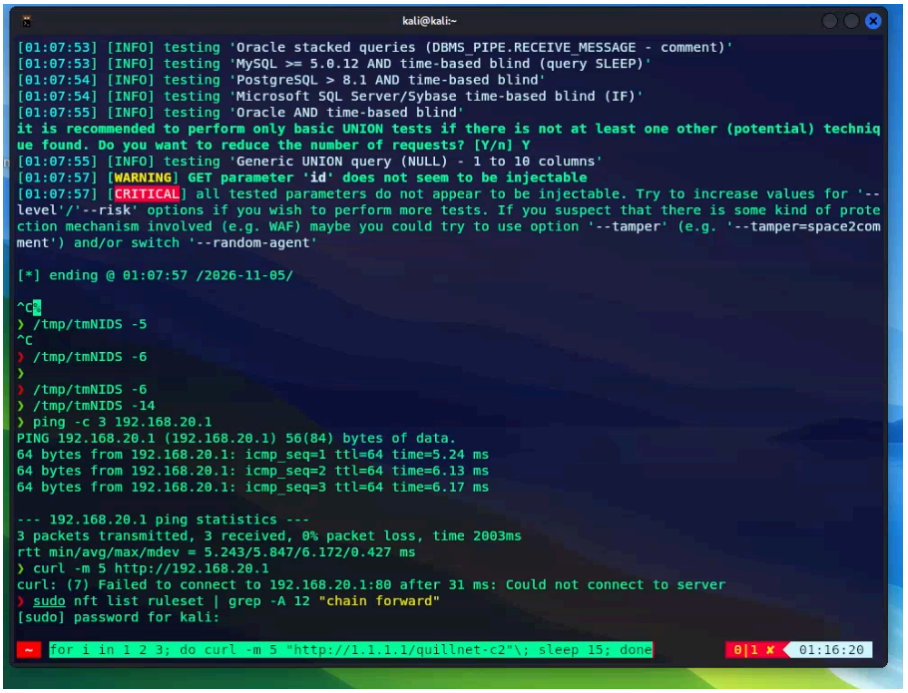


(Fuente: Propia del equipo)

Movimiento lateral Usuarios → Cámaras (pendiente):



(Fuente: Propia del equipo)



(Fuente: Propia del equipo)

Cuarentena automática (pendiente):



(Fuente: Propia del equipo)



(Fuente: Propia del equipo)

15. Plan de mejora

Mejora	Prioridad	Plazo	Responsable
Cadena input en nftables con política drop y SSH solo por llave desde la VLAN 10 (hacerla con acceso físico a la Pi por si se bloquean)	ALTA	Inmediato	Infraestructura
Clonar la microSD y respaldar la configuración del gateway	ALTA	Inmediato	Infraestructura
Probar el push en celular real (Android y iPhone con PWA) y el flujo de correo	ALTA	Inmediato	Desarrollo web
Rotar los secretos expuestos (JWT, Groq, contraseña de la base) después de la última demostración	ALTA	Inmediato	Desarrollo web
Verificar un dominio propio en Resend (o confirmar que el correo remitente es el dueño de la cuenta de Resend) para que el código 2FA del inicio de sesión llegue a cualquier usuario; mientras tanto, LOGIN_2FA=0 como interruptor de emergencia	ALTA	Inmediato	Autenticación
Migrar la base a un plan sin vencimiento o exportar/restaurar	ALTA	Corto	Desarrollo web
Pentesting formal por VLAN, prueba de VLAN hopping y detección en VLAN 20, 30 y 40	ALTA	Corto	Pentesting
Aislamiento de clientes en el access point (o microsegmentación por dispositivo) para ver el tráfico intra-VLAN	ALTA	Corto	Infraestructura
Repetir la prueba de MQTT desde otra red o enlace para saber si el corte lo hizo el gateway o el operador móvil	MEDIA	Inmediato	Pentesting
Pruebas de carga y de caída de la WAN	MEDIA	Corto	Pentesting / Infraestructura
Dominio propio verificado en el servicio de correo	MEDIA	Corto	Desarrollo web
eve.json en solo lectura e historial de cambios administrativos	MEDIA	Corto	Infraestructura
Revisar WPA2-PSK: rotación de clave de grupo y actualización de firmware del EAP110 (2022)	MEDIA	Corto	Infraestructura
Limitador de tasa en almacenamiento compartido (Redis)	MEDIA	Largo	Desarrollo web
SSE con ticket de un solo uso en lugar del JWT en la URL	MEDIA	Largo	Desarrollo web
Revocación explícita de JWT	MEDIA	Largo	Desarrollo web
Reglas basadas en metadatos TLS (JA3/SNI) y	MEDIA	Largo	Infraestructura

filtrado DNS para tráfico cifrado			
Registrar en bitácora interna las firmas de severidad baja descartadas; verificar si el conjunto GPL debe cargarse	BAJA	Corto	Pentesting
Filtro de alertas por tipo y fecha; consola de técnico del ISP	BAJA	Largo	Desarrollo web

16. Conclusiones

- **¿Cuál es el mayor riesgo del proyecto?** Hoy los mayores riesgos son: la validación pendiente del aislamiento entre VLAN (VLAN hopping) y de la detección en las VLAN 20, 30 y 40; la falta de una cadena input en el gateway; y la dependencia de servicios de plan gratuito o de corto plazo (la base de Render venció el 27 de octubre de 2026 y se renovó por tiempo limitado). La ausencia de una imagen actualizada de la microSD agrava el impacto de una falla de hardware.
- **¿Qué activo necesita mayor protección?** El activo más importante es el Raspberry Pi que controla todo el tráfico de red en los hogares.
- **¿Qué principio de la tríada CIA está mejor aplicado?** La confidencialidad en la capa de aplicación: secretos de la Pi guardados solo como hash, contraseñas con hash y cambio únicamente por enlace de correo, verificación en dos pasos para desbloquear, aislamiento por hogar y una IA que nunca recibe IPs ni MAC.
- **¿Qué necesita mejorar?** La disponibilidad y la integridad: la Raspberry Pi es un punto único de falla, no hay una imagen actualizada de la microSD, eve.json no tiene control de integridad y el servicio depende de planes gratuitos; también la confidencialidad del propio gateway hasta agregar la cadena input.
- **¿Qué medidas deberían implementarse primero?** La cadena input con política drop y SSH por llave, la imagen de la microSD y el respaldo de la configuración, la rotación de secretos al terminar las demostraciones y el pentesting formal por VLAN.

- **¿El proyecto se considera seguro para ser utilizado? ¿Por qué?:** Es seguro como prueba de concepto en un entorno controlado: se verificó el bloqueo en línea de un beacon real de Covenant, que el tráfico MQTT sin cifrar no salió de la red, el bloqueo por MAC y la persistencia tras reinicio. Sus límites conocidos: no inspecciona el tráfico entre equipos de una misma VLAN ni el tráfico cifrado con TLS. No debe considerarse listo para un despliegue comercial hasta completar la validación del aislamiento entre VLAN, las pruebas de carga y el cierre de las brechas indicadas.

Capítulo IV

Análisis de resultados

El análisis de resultados del proyecto estará dividido en dos importantes secciones, la primera se centrará en analizar los resultados de las encuestas hechas para poder interpretar la necesidad de un dispositivo como Quillnet en los hogares domésticos. La segunda parte se encargará de analizar las pruebas de seguridad para el dispositivo.

La encuesta hecha recopiló información de 112 hogares diferentes que permite tener información amplia y variada acerca de la seguridad y los dispositivos en los hogares de la sociedad costarricense. El siguiente análisis está hecho en base a las respuestas proporcionadas en el formulario, la muestra fue de 112 hogares de personas costarricenses entre la clase media y alta.

En el primer enunciado se muestra la provincia de proveniencia de los participantes. En los resultados se obtuvo que un 81.3% de los encuestados viven actualmente en la provincia de San José.



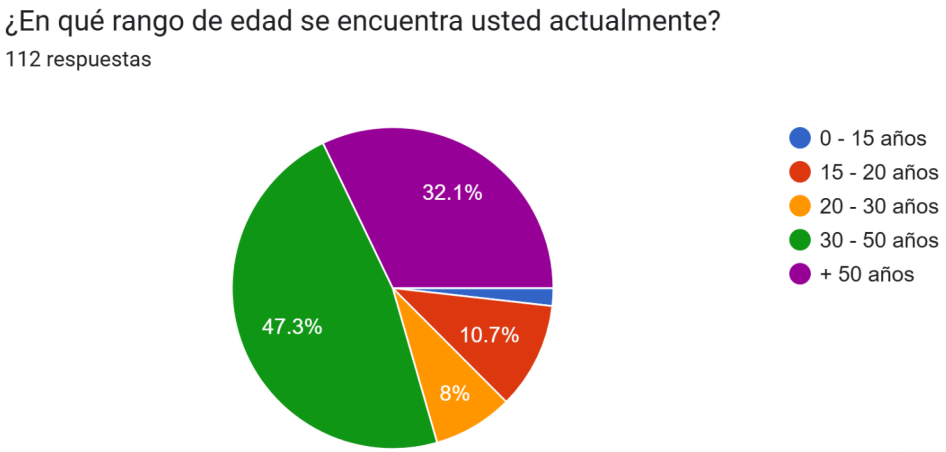
Provincia de residencia de los encuestados. Tomado de el formulario Proyecto Quillnet 2026

En el segundo enunciado se consultaba acerca de cuántas personas residen en el hogar. De estos datos se puede concluir que un 61.1% de la muestra vive en un hogar junta a un rango de entre 1 a 3 personas.



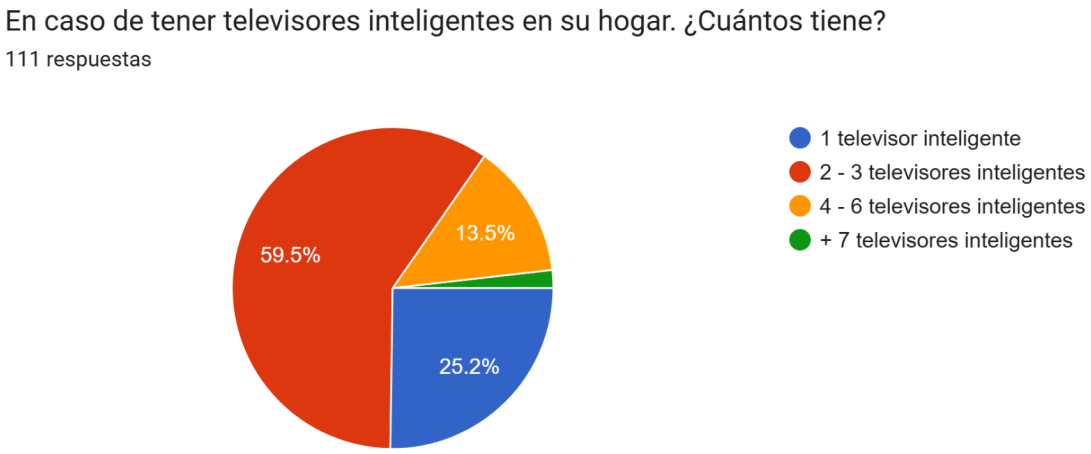
Cantidad de personas viviendo en el hogar. Tomado de el formulario Proyecto Quillnet 2026

El cuarto enunciado preguntaba cuál era el rango de edad de los encuestados. Tomando en base los datos mostrados en la encuesta se puede concluir que un 47.3% de los integrantes de los hogares ronda la edad de entre los 30 a 50 años.

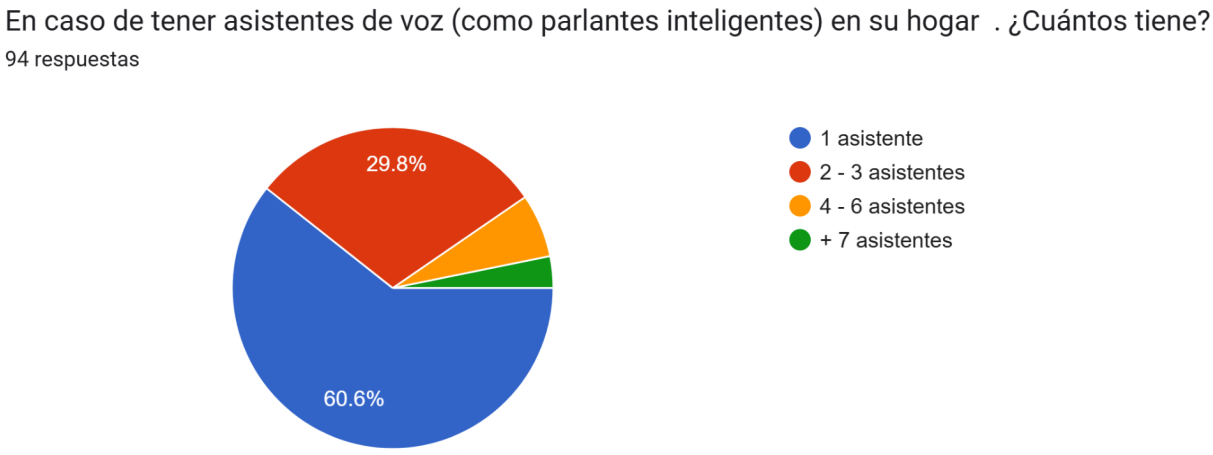


Rango de edad de las personas encuestadas. Tomado de el formulario Proyecto Quillnet 2026

Las siguientes preguntas en el formulario buscan conocer cuales son los dispositivos IoT más comunes en los hogares costarricenses entre la clase media y alta. Mediante las respuestas de los participantes se puede concluir que los dispositivos más comunes en los entornos domésticos son, los televisores inteligentes, los celulares, las computadoras entre otros. Lo cual refleja que los sistemas conectados a internet son más comunes entre la población de Costa Rica.



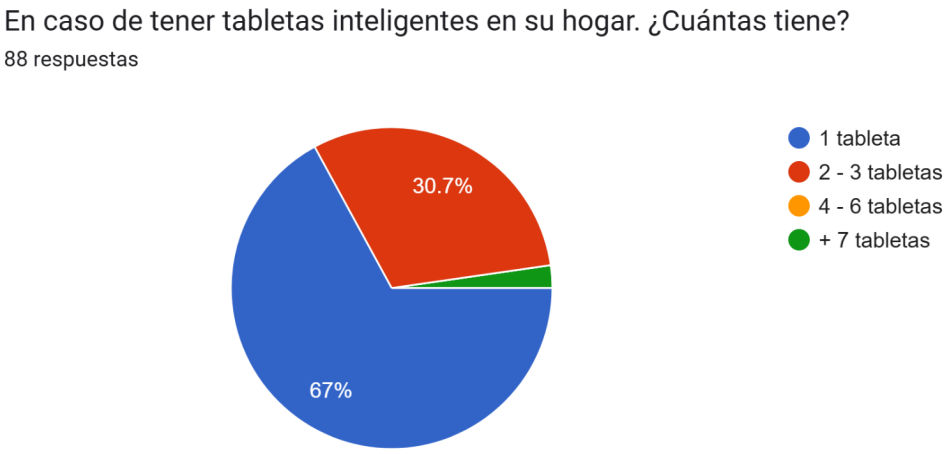
Cantidad de televisores inteligentes en el hogar. Tomado de el formulario Proyecto Quillnet 2026



Cantidad de asistentes de voz en el hogar. Tomado de el formulario Proyecto Quillnet 2026



Cantidad de celulares en el hogar. Tomado de el formulario Proyecto Quillnet 2026

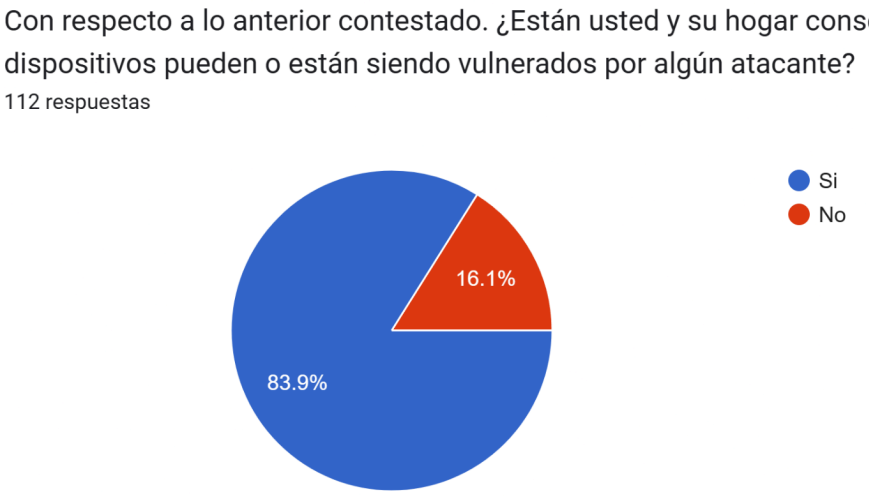


Cantidad de tabletas inteligentes en el hogar. Tomado de el formulario Proyecto Quillnet 2026

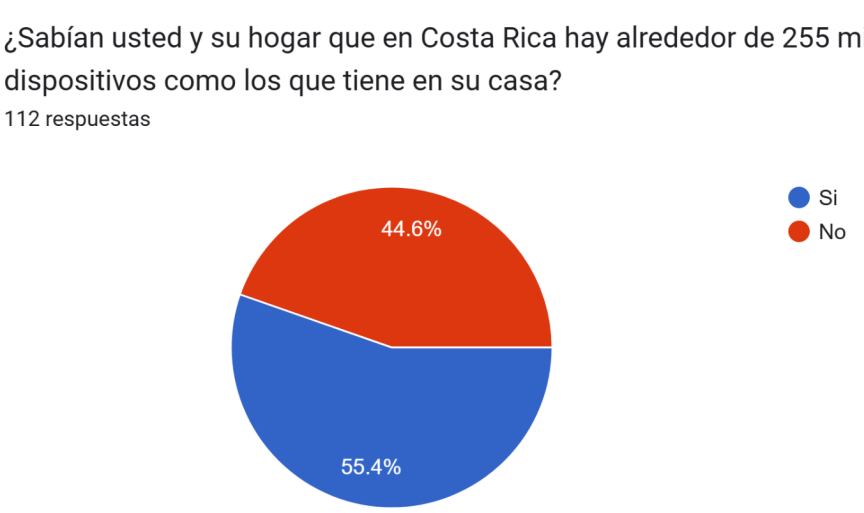


Cantidad de computadoras en el hogar. Tomado de el formulario Proyecto Quillnet 2026

Las dos siguientes preguntas qué fueron contestadas por la muestra buscaba encontrar que tan conscientes eran los participantes acerca de su vulnerabilidad tecnológica frente a ataques cibernéticos en su hogar.



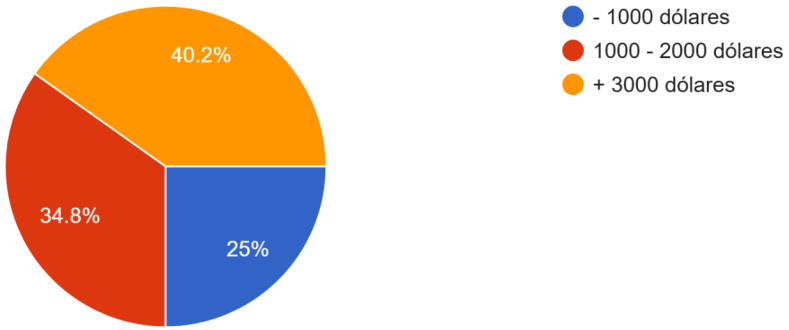
Conciencia de la vulnerabilidad de los dispositivos tecnológicos. Tomado de el formulario Proyecto Quillnet 2026



Conciencia de la vulnerabilidad de los dispositivos tecnológicos. Tomado de el formulario Proyecto Quillnet 2026

Para la pregunta siguiente es importante considerar que los datos proporcionados en esta encuesta son totalmente confidenciales. Se le consultó a los participantes acerca de de sus ingresos mensuales con el fin de determinar la clase social o los ingresos de los participantes para evaluar la posibilidad de instalar Quillnet en hogares tanto de bajos recursos como altos recursos.

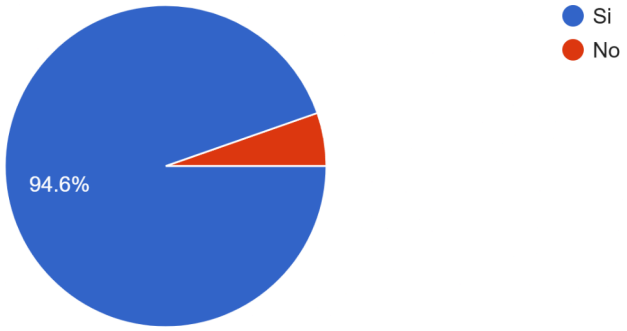
¿Cuántos son los ingresos mensuales de su hogar? Recuerde que toda la información compartida aquí es totalmente confidencial y no se registraran datos personales de cada hogar.
112 respuestas



Ingresos mensuales de los encuestados. Tomado de el formulario Proyecto Quillnet 2026

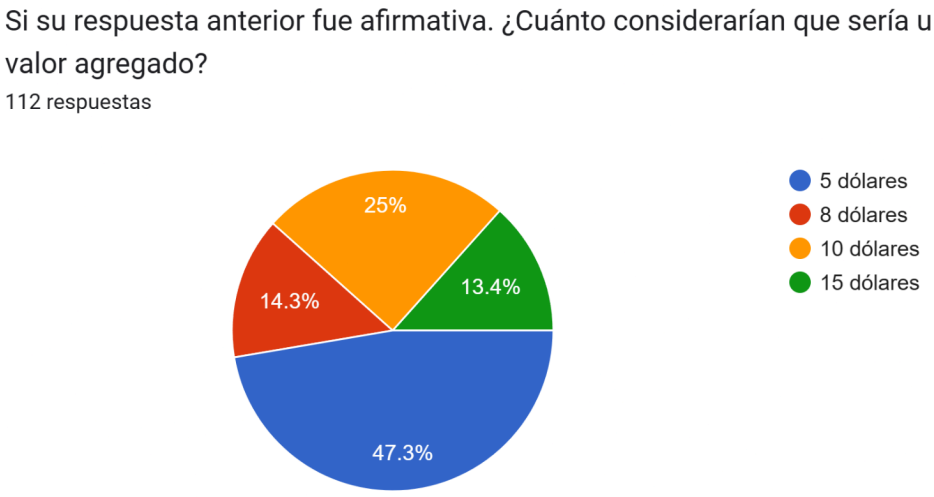
La antepenúltima fue hecha con el objetivo de evaluar el proyecto Quillnet como un negocio que sea posible y rentable. En esta se les preguntó a los evaluados si ellos usaron un servicio que les proporciona internet y que además les permitiera obtener datos de su red y protegerla de ataques.

Si existiera un servicio que pudiera proteger sus redes de estos ataques y darle información relevante del estado de sus conexiones integrado a...ternet por un pequeño valor agregado, ¿lo usaría?
112 respuestas



Validación de Quinet como idea de negocio. Tomado de el formulario Proyecto Quillnet 2026

En la última pregunta se les evaluó a los participantes cuál sería una buena suma mensual para proteger las redes de sus hogares con un dispositivo Quillnet. Por medio de esto se puede concluir que un precio aceptable para los usuarios será aumentar el servicio a 5 dolares mas.



Precio para Quillnet como servicio. Tomado de el formulario Proyecto Quillnet 2026

De toda esta información se podría concluir e interpretar que la población costarricense entre los 20 a 50 años de edad que viven mayoritariamente en la provincia de San José son conscientes de los peligros cibernéticos que pueden sufrir sus dispositivos conectados a internet. Además se puede concluir que la mayoría de usuarios estarían dispuestos a pagar entre 5 a 10 dólares más por proteger su internet de amenazas. Este rango incluye el precio proyectado de ₡3,500 mensuales (Capítulo III, Gestión Administrativa y Financiera), por lo que el modelo no requeriría un subsidio del ISP para resultar accesible a los hogares encuestados. Limitación de la muestra: las 112 respuestas provienen de una muestra no probabilística, concentrada en San José (81.3 %) y en hogares de clase media y alta, por lo que los resultados describen a esa población y no pueden generalizarse a todos los hogares de Costa Rica.

Conclusiones Generales

1. **La seguridad de los dispositivos IoT residenciales representa una necesidad importante.** La investigación realizada permitió identificar que los dispositivos conectados a internet pueden presentar vulnerabilidades que comprometen la privacidad de los usuarios. Por esta razón, QUILLNET plantea una alternativa orientada a fortalecer la protección de los hogares mediante mecanismos de seguridad de red.
2. **La combinación de Zero Trust y segmentación de red constituye una base sólida para la protección de los dispositivos.** La arquitectura propuesta permite separar los dispositivos IoT de la red utilizada por los usuarios y controlar las comunicaciones entre segmentos. Esto contribuye a reducir las posibilidades de movimiento lateral de un atacante y limita el impacto que podría tener el compromiso de un dispositivo.
3. **La integración de herramientas de detección, análisis y respuesta representa uno de los principales aportes de QUILLNET.** El proyecto contempla el uso de Suricata para analizar el tráfico de red, una arquitectura de backend para procesar los eventos y una interfaz que permita presentar las alertas de forma comprensible. Además, la metodología Purple Team proporciona un enfoque para evaluar las defensas mediante ataques simulados y mejorar las medidas de protección según los resultados obtenidos.
4. **La facilidad de uso es un elemento fundamental para acercar la ciberseguridad a los hogares.** La propuesta de una interfaz de autogestión, con información visual, alertas y explicaciones en lenguaje sencillo, busca que las personas puedan comprender el estado de su red sin necesitar conocimientos técnicos avanzados. De esta manera, QUILLNET procura acercar mecanismos de seguridad utilizados en entornos empresariales a un contexto doméstico.
5. **El desarrollo del prototipo permitió completar avances y reconocer desafíos.** Entre los avances se encuentran el gateway físico con Raspberry Pi OS Lite, las cuatro VLAN con firewall Zero Trust, Suricata 7.0.10 en modo IPS en línea, el bloqueo en línea verificado de un beacon real del framework Covenant (con 12,860 reglas de Emerging Threats Open en modo drop), el bloqueo por dirección MAC con cuarentena automática y persistencia tras

reinicio, el backend en la nube con traducción de alertas por inteligencia artificial y clasificación en 13 tipos de ataque, el frontend de autogestión (mapa, panel y alertas en tiempo real) y 238 pruebas automatizadas. Como desafíos, quedan por completar la batería formal de pentesting por VLAN, la prueba de VLAN hopping, la detección en las VLAN 20, 30 y 40 y la prueba de rendimiento bajo carga.

6. **QUILLNET presenta potencial de crecimiento, aunque todavía requiere validación antes de una implementación a gran escala.** Su diseño permite plantear futuras mejoras en automatización, monitoreo, gestión de múltiples instalaciones y respuesta ante amenazas. No obstante, para evolucionar de una prueba de concepto a un servicio comercial será necesario comprobar su eficacia mediante pruebas controladas, reforzar la confidencialidad de los datos, evaluar el rendimiento y garantizar la estabilidad de sus componentes.

Recomendaciones para la actualidad del proyecto

1. **Fortalecer la confidencialidad de la información.** Se recomienda revisar los mecanismos de autenticación, la gestión de sesiones y el almacenamiento de credenciales, así como proteger las claves de las API y evitar que se expongan dentro del código fuente. También es importante verificar que los datos enviados a servicios externos, como la API de Groq, no incluyan información sensible innecesaria.

2. **Proteger el propio gateway y la continuidad del backend.** Se recomienda agregar una cadena input en nftables con política drop, restringir SSH por llave solo desde la VLAN de usuarios, rotar los secretos expuestos durante el desarrollo y migrar la base de datos PostgreSQL a un plan sin vencimiento, pues la base gratuita venció el 27 de octubre de 2026 y su renovación actual es temporal.
3. **Completar y documentar las pruebas de seguridad.** Se deben ampliar las pruebas controladas de pentesting para comprobar la detección y el bloqueo de amenazas, la efectividad de las reglas del firewall y el aislamiento entre las VLAN. Asimismo, es necesario

documentar los resultados de cada prueba, las alertas generadas y las correcciones aplicadas para contar con evidencia verificable de la efectividad del sistema.

4. **Implementar copias de seguridad actualizadas.** Se recomienda crear respaldos de la configuración del gateway, las reglas de firewall, las VLAN, la base de datos PostgreSQL y los demás componentes esenciales. También se deben realizar pruebas de restauración para comprobar que el sistema pueda recuperarse ante fallos de hardware, corrupción de la tarjeta microSD o pérdida de información.
5. **Mejorar el monitoreo y la respuesta ante amenazas.** Es conveniente continuar ajustando las reglas de detección para reducir los falsos positivos y mejorar la identificación de actividades sospechosas. Las acciones de bloqueo deben verificarse mediante pruebas controladas, procurando que las medidas de seguridad no interrumpan las comunicaciones legítimas de los dispositivos.
6. **Verificar la integridad de los registros y las configuraciones.** Se recomienda restringir los permisos de acceso a los registros de seguridad, mantener un historial de los cambios administrativos y establecer mecanismos para detectar modificaciones no autorizadas. Esto facilitaría la investigación de incidentes y evitaría que la evidencia de una amenaza sea alterada o eliminada.
7. **Mantener actualizados los componentes tecnológicos.** Es importante revisar periódicamente las versiones del sistema operativo, las herramientas de seguridad, las dependencias de software y las bibliotecas utilizadas. Antes de actualizar, se deben comprobar la compatibilidad entre componentes y el funcionamiento de las configuraciones existentes para evitar nuevas interrupciones.
8. **Continuar mejorando la experiencia del usuario.** La interfaz debe presentar las alertas mediante mensajes claros, niveles de riesgo comprensibles y recomendaciones prácticas. También se recomienda verificar que las acciones disponibles para el usuario sean seguras y que la información mostrada coincida con el estado real de la red.

9. **Fortalecer la gestión de sesiones y la capa de acceso.** Se recomienda mover el limitador de tasa a un almacenamiento compartido (Redis), sustituir el token en la URL de la conexión

SSE por un ticket de un solo uso, implementar la revocación explícita de JWT y verificar un dominio propio en el servicio de correo.

Recomendaciones para el desarrollo futuro

1. **Desarrollar un sistema escalable de gestión y monitoreo.** A futuro, se recomienda evaluar la incorporación de un SIEM que permita centralizar, correlacionar y analizar eventos de seguridad provenientes de distintas instalaciones. Esto facilitaría la identificación de patrones de ataque y permitiría ampliar las capacidades de monitoreo de QUILLNET.
2. **Ampliar la automatización de la respuesta ante incidentes.** Quillnet ya aísla automáticamente por dirección MAC a los dispositivos que generan 3 alertas de bloqueo real en 5 minutos; se propone ampliar esa automatización con más criterios de decisión, aplicar reglas de seguridad adicionales y restaurar las condiciones normales cuando corresponda. Estas acciones deberán contar con criterios de autorización y mecanismos de verificación para evitar bloqueos incorrectos.
3. **Evaluar la implementación de QUILLNET como servicio gestionado por un proveedor de internet.** Una línea de desarrollo consiste en estudiar cómo desplegar y administrar el sistema desde la infraestructura de un proveedor de servicios de internet (ISP). Para ello, será necesario analizar los costos, la compatibilidad con distintos equipos de red, la administración remota segura y el mantenimiento de múltiples instalaciones.
4. **Mejorar la capacidad de adaptación a diferentes dispositivos IoT.** Se recomienda ampliar las pruebas con cámaras IP, televisores inteligentes, asistentes de voz y otros dispositivos de distintos fabricantes. Esto permitiría identificar diferencias de comportamiento, ajustar las políticas de segmentación y comprobar que la protección funcione en diferentes entornos domésticos.
5. **Optimizar el rendimiento y la disponibilidad.** En futuras versiones se deberá evaluar la capacidad de procesamiento y memoria del hardware, los tiempos de respuesta ante alertas y el impacto del monitoreo sobre la conectividad. También conviene estudiar alternativas de recuperación ante fallos y configuraciones que reduzcan la dependencia de un único punto de fallo.

6. **Ampliar la validación mediante Purple Team.** Se recomienda establecer un ciclo periódico de simulación de ataques, detección, análisis de resultados y mejora de las defensas. Este proceso permitirá comprobar el comportamiento del sistema frente a nuevas amenazas y mantener una evaluación continua de sus capacidades de protección.
7. **Evaluar alternativas para reducir la dependencia de servicios externos.** Se propone estudiar opciones para la interpretación de alertas que permitan mantener la funcionalidad del sistema cuando un servicio externo de inteligencia artificial no esté disponible. Cualquier alternativa deberá evaluarse según su precisión, costo, privacidad y consumo de recursos. Como primer paso, la versión actual ya clasifica el tipo de ataque con reglas fijas antes de recurrir a la IA y muestra un texto de respaldo por severidad si la IA no responde.
8. **Realizar un estudio de viabilidad comercial.** Antes de considerar una distribución a gran escala, se recomienda analizar los costos del hardware, el mantenimiento, las actualizaciones, el soporte técnico y la operación del servicio. También se deben identificar posibles alianzas con proveedores de internet y establecer un modelo de costos accesible para los hogares costarricenses. El modelo proyectado en el Capítulo III (₡3,500 mensuales y recuperación de la inversión en 14 meses) debe validarse con datos reales de costos de operación.

En términos generales, se recomienda mantener un proceso de desarrollo continuo, documentado y basado en evidencia. La evolución de QUILLNET deberá priorizar la protección de la información, la confiabilidad del sistema, la facilidad de uso y la validación de sus mecanismos de seguridad, de manera que cada nueva funcionalidad responda a una necesidad identificada y contribuya al objetivo principal de proteger los dispositivos IoT domésticos.

Bibliografía

3CORESec. (s.f.). *tmNIDS (testmynids.org)*. Repositorio 3CORESec/testmynids.org en GitHub. https://testmynids.org

Akamai. (2026). *Akamai Guardicore Segmentation*. https://www.akamai.com/resources/product-brief/akamai-segmentation

Bitdefender. (2025, marzo 27). *Securing your smart home: Step-by-step network segmentation*. https://bitdefender.in/securing-your-smart-home-step-by-step-network-segmentation/

Cisco. (2024). *Cisco IoT threat defense: Securing the IoT at scale*. https://blogs.cisco.com/security/cisco-iot-threat-defense-securing-the-iot-at-scale

Cisco. (2026, enero 30). *What is network segmentation?* https://www.cisco.com/site/us/en/learn/topics/security/what-is-network-segmentation.html

Cloudflare. (2026). *Cloudflare 2025 Q4 DDoS threat report*. https://blog.cloudflare.com/ddos-threat-report-2025-q4/

ColorTokens. (2025, mayo 8). *ColorTokens + Nozomi Networks partnership for OT/IoT security*. https://colortokens.com/news/nozomi-iot-ot-security-vendors/

Cybersecurity and Infrastructure Security Agency (CISA). (s.f.). *Known exploited vulnerabilities catalog*. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Droms, R. (1997). *Dynamic Host Configuration Protocol* (RFC 2131). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2131

Federal Bureau of Investigation, Internet Crime Complaint Center (IC3). (2025, 5 de junio). *Home Internet Connected Devices Facilitate Criminal Activity* (Alerta I-060525-PSA). https://www.ic3.gov/PSA/2025/PSA250605

Forescout Research – Vedere Labs. (2025). *The riskiest devices of 2025*. https://www.forescout.com/research-labs/the-riskiest-devices-of-2025/

Fortinet. (2024). *FortiNAC — Network access control*.

https://www.fortinet.com/products/network-access-control

Fortinet. (s.f.). *¿Mejores prácticas de seguridad del IoT? Cómo proteger los dispositivos del IoT*. https://www.fortinet.com/lat/resources/cyberglossary/iot-best-practices

Groq. (s.f.). *GroqCloud documentation*. https://console.groq.com/docs

Help Net Security. (2024, 19 de septiembre). *FBI forced Flax Typhoon to abandon its botnet*. https://www.helpnetsecurity.com/2024/09/19/flax-typhoon-botnet-disrupted/

Institute of Electrical and Electronics Engineers. (2022). *IEEE standard for local and metropolitan area networks—Bridges and bridged networks* (IEEE Std 802.1Q-2022). IEEE.

IoT Analytics. (2025, 30 de octubre). *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*. IoT Business News. https://iotbusinessnews.com/?p=44499

IoT For All. (2024, diciembre 2). *10 tips to secure your IoT devices from hackers*. https://www.iotforall.com/10-tips-to-secure-your-iot-devices-from-hackers

Ip_Admin. (2024, agosto 28). *10 common IoT hacks and how to defend against them*. Device Authority. https://deviceauthority.com/10-common-iot-hacks/

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Jouly, R. (s.f.). *Reducción de la superficie de ataque en entornos IoT*. ManageEngine. https://www.manageengine.com/latam/blog/general/reduccion-de-la-superficie-de-ataque-en-entornos-ti.html

JumpCloud. (2025). *IoT device vulnerability report*. https://jumpcloud.com

Kelley, S. (s.f.). *Dnsmasq: Network services for small networks*. https://thekelleys.org.uk/dnsmasq/doc.html

McRae. (s.f.). *Comprendiendo los pilares clave de la tecnología IoT*. SpinTel. https://www.spintel.net.au/blog/pillars-of-iot

Meta. (2024). *Llama 3.3 model card*. https://www.llama.com

Microsoft Azure. (s.f.). *¿Qué es IoT?* https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-iot

MITRE. (2024). *MITRE ATT&CK framework*. https://attack.mitre.org/

MITRE Caldera. (2024). *Caldera: Automated adversary emulation*. https://caldera.mitre.org/

Netfilter Project. (s.f.). *nftables wiki*. https://wiki.nftables.org/wiki-nftables/index.php/What_is_nftables%3F#What_is_nftables?

Netgear / Bitdefender. (2025). *2025 IoT threat report*. https://www.netgear.com/hub/network/security/iot-threat-report-25/

National Institute of Standards and Technology (NIST). (2020). Zero Trust Architecture (SP 800-207). https://www.nist.gov/publications/zero-trust-architecture

National Institute of Standards and Technology (NIST). (2025). Digital Identity Guidelines (SP 800-63-4).

National Institute of Standards and Technology (NIST). (2025). Authentication and Authenticator Management (SP 800-63B-4).

MDN Web Docs. (2025). *API*. https://developer.mozilla.org

MDN Web Docs. (2025). *What is a web server?* https://developer.mozilla.org

Open Web Application Security Project (OWASP). (2025). *Password Storage Cheat Sheet*. https://owasp.org

Open Web Application Security Project (OWASP). (2025). *OWASP Top 10: Cryptographic Failures*. https://owasp.org

Open Information Security Foundation. (2025). *Suricata User Guide*. https://suricata.io

Pallets Projects. (2025). *Handling Application Errors*. https://flask.palletsprojects.com

Palo Alto Networks, Unit 42. (2020). *2020 Unit 42 IoT Threat Report*. https://unit42.paloaltonetworks.com/iot-threat-report-2020/

Palo Alto Networks. (2024). *Enterprise IoT security*. https://www.paloaltonetworks.com/network-security/device-security

Percival, C., & Josefsson, S. (2016). *The scrypt password-based key derivation function* (RFC 7914). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc7914/

PostgreSQL Global Development Group. (2025). *PostgreSQL documentation*.

https://www.postgresql.org/docs/

Proofpoint. (s.f.). *Emerging Threats Open ruleset*. https://rules.emergingthreats.net

Raspberry Pi Ltd. (2025). *Raspberry Pi 4 Model B: Product brief and documentation*. https://www.raspberrypi.com/documentation/

Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., & Lear, E. (1996). *Address allocation for private internets* (RFC 1918). Internet Engineering Task Force www.rfc-editor.org/info/rfc1918/

SentinelOne. (2025, octubre 23). *Los 10 principales riesgos de seguridad del IoT y cómo mitigarlos*.https://www.sentinelone.com/es/cybersecurity-101/data-and-ai/iot-security-risks/

SQLite Project. (2025). *About SQLite*. https://sqlite.org

Srisuresh, P., & Egevang, K. (2001). *Traditional IP network address translator (Traditional NAT)* (RFC 3022). Internet Engineering Task Force. www.rfc-editor.org/rfc/rfc3022

Transforma Insights. (2026). *IoT connections forecast 2024-2035*. https://transformainsights.com/research/forecast/highlights

Trend Micro. (2023, 2 de mayo). *Mirai botnet exploits TP-Link router vulnerability: What to know and how to protect your device*. https://news.trendmicro.com/2023/05/02/mirai-botnet-exploits-tp-link-router-vulnerability

WHATWG. (s.f.). *HTML living standard: Server-sent events*. https://html.spec.whatwg.org/multipage/server-sent-events.html

WifiTalents. (2026). *IoT default credentials and update statistics*. https://wifitalents.com

Zscaler. (2024). *IoT Security Solutions*. https://www.zscaler.com/products-and-solutions/iot-security

¿Qué es una superficie de ataque? (s.f.). IBM. https://www.ibm.com/es-es/think/topics/attack-surface

Anexos

Anexo 1. Estadísticas adicionales

El riesgo promedio de los dispositivos conectados en los diez países de mayor riesgo pasó de 6.53 en 2024 a 9.1 en 2025 (Forescout, 2025). En septiembre de 2024, el FBI, la NSA y la Cyber National Mission Force revelaron una botnet de más de 260,000 dispositivos (routers, cámaras IP, grabadores de video y equipos de almacenamiento en red) controlada por la empresa china Integrity Technology Group (Help Net Security, 2024).

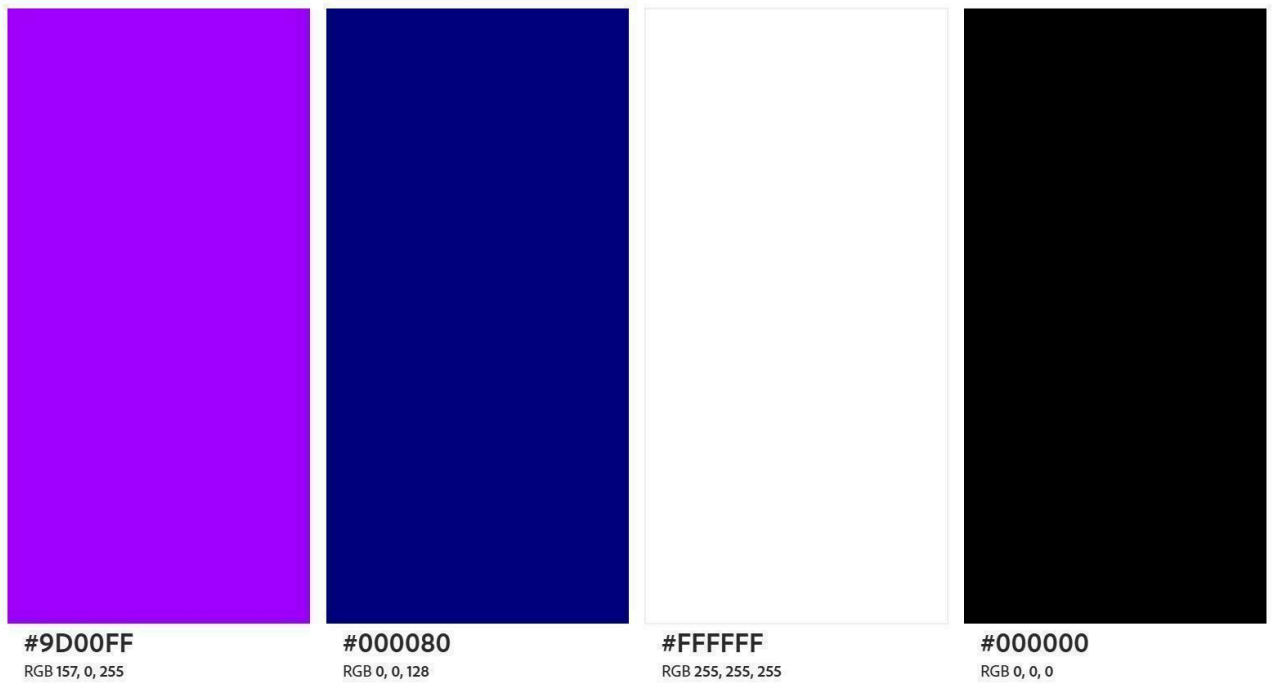
Anexo 2. Glosario técnico abreviado

IoT (Internet of Things), CVE (Common Vulnerabilities and Exposures), MQTT (Message Queuing Telemetry Transport), VLAN (Virtual Local Area Network), IDS (Intrusion Detection System), SIEM (Security Information and Event Management), ISP (Internet Service Provider), ZTA (Zero Trust Architecture), 2FA (Two-Factor Authentication), DHCP (Dynamic Host Configuration Protocol), DNS (Domain Name System), ET Open (Emerging Threats Open, conjunto público de reglas de Suricata), GPL (conjunto de reglas heredado de Snort/Sourcefire para Suricata), HMAC (Hash-based Message Authentication Code), IPS (Intrusion Prevention System), JA3 (huella de la negociación TLS del cliente), JWT (JSON Web Token), KEV (Known Exploited Vulnerabilities, catálogo de CISA), MAC (Media Access Control), NFQUEUE (cola de nftables que entrega paquetes a Suricata en modo inline), PSK (Pre-Shared Key), PWA (Progressive Web App), scrypt (función de derivación de claves costosa en memoria), SNI (Server Name Indication), SSE (Server-Sent Events), SSID (Service Set Identifier), TLS (Transport Layer Security), VAPID (Voluntary Application Server Identification), Web Push (estándar de notificaciones push para la web).

Anexo 3. Stack técnico del proyecto

Tecnología	Uso principal	Uso en el proyecto
Suricata IPS	Sistema de detección de intrusos	Analiza el tráfico de red, bloquea ataques y genera alertas cuando detecta algo sospechoso
Kali Linux	Simulación de ataques y pentesting	Se encarga de simular los ataques a cada dispositivo
API de Groq	Interfaz de programación para conectar modelos de inteligencia artificial	Traduce las alertas técnicas de la red a lenguaje simple para el usuario
Raspberry Pi	Ordenador de placa única de tamaño reducido	Es el hardware físico donde corre todo el sistema
Raspberry Pi OS Lite	Sistema operativo oficial de Raspberry Pi, basado en Debian	Sistema del gateway; reemplazó a OpenWRT, que no permitió instalar Suricata
Python Flask	Microframework para levantar aplicaciones y APIs	Es la API que se conecta con la aplicación web para mostrar los datos al usuario
PostgreSQL	Base de datos relacional de código abierto	Base de datos en la nube (Render) con datos separados por hogar
HTML/CSS/JavaScript	Tecnologías fundamentales de la World Wide Web	Forman el frontend de la aplicación web para mostrarle los datos al usuario
nftables y dnsmasq	Firewall de Linux y servicio DHCP/DNS	Política Zero Trust (forward drop) y un ámbito DHCP/DNS por VLAN
Render	Plataforma de despliegue en la nube	Aloja el backend Flask y la base de datos PostgreSQL
Resend	Servicio de correo	Recuperación de contraseña, código de verificación en dos pasos (inicio de sesión y desbloqueo) y respaldo de alertas altas
Web Push (VAPID) y PWA	Notificaciones y aplicación instalable	Aviso de alertas y de dispositivos nuevos

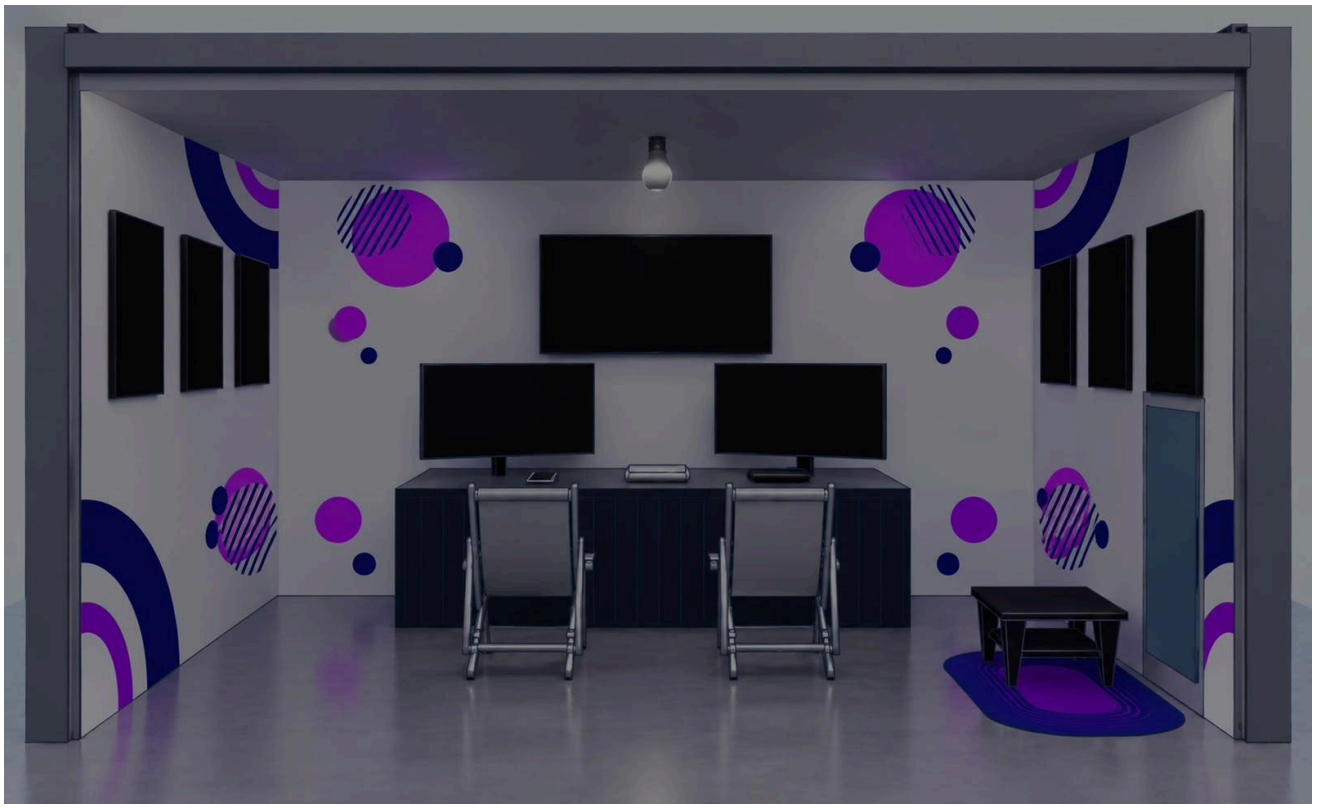
Anexo 4. Paleta de Colores



Anexo 5. Logo



Anexo 6. Diseño del stand.



Anexo 7. Vista de una VLAN desde el celular en modo oscuro.



Anexo 8. Vista de las alertas desde el celular en modo claro.



Anexo 9. Vista de una VLAN desde el celular en modo claro.



Anexo 10. Vista del panel de estado desde el celular en modo oscuro.



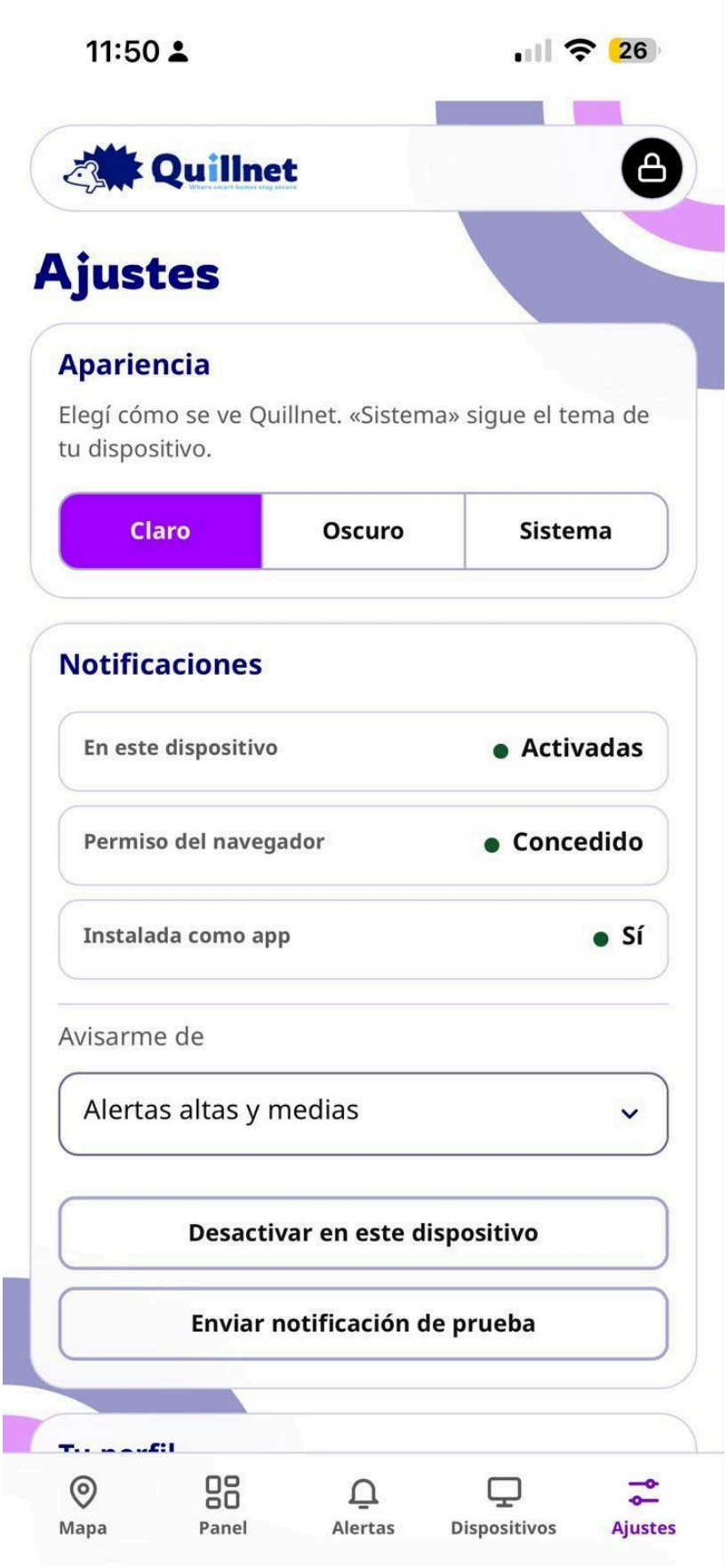
Anexo 11. Vista del panel de estado desde el celular en modo claro.



Anexo 12. Vista de la configuración desde el celular en modo oscuro.



Anexo 13. Vista de la configuración desde el celular en modo claro.



Anexo 14. Vista de las alertas desde el celular en modo oscuro.



Anexo 15. Alertas al momento.



Anexo 16. Alerta y descripción del dispositivo.



Anexo 17. Muestra de ataque en Kali Linux (sqlmap)



Anexo 18. Muestra de los logs de suricata.

